

UNIT- II

2

Client Side Technologies : JavaScript and DOM

Syllabus

JavaScript : Introduction to JavaScript, JavaScript in perspective, basic syntax, variables and data types, statements, operators, literals, functions, objects, arrays, built in objects, JavaScript debuggers.

DOM : Introduction to Document Object Model, DOM history and levels, intrinsic event handling, modifying element style, the document tree, DOM event handling, jQuery, Overview of Angular JS.

Contents

2.1	Introduction to JavaScript	
2.2	JavaScript in Perspective	
2.3	Basic Syntax	
2.4	Identifiers, Keywords and Comments	
2.5	Data Types	May-18, 19, Marks 5
2.6	Literals	
2.7	Other Primitive Types	
2.8	Variables	
2.9	Operators	
2.10	Input and Output	
2.11	Statements	
2.12	Querying and Handling Properties	
2.13	Functions	April-18, Dec.-18, Marks 5
2.14	Built in Objects	
2.15	Creation of Object	Dec.-18, Marks 5
2.16	Arrays	May-18, 19, Marks 5
2.17	Strings	

2.18	JavaScript Debuggers	
2.19	Introduction to Document Object Model (DOM)	 Dec.-19 ,..... Marks 5
2.20	DOM History and Levels	
2.21	The Document Tree	 April-18, May-19 ,..... Marks 5
2.22	Modifying Element Style	 May-18, Dec.-19, March-20 ,..... Marks 5
2.23	DOM Event Handling	 April-18, Dec.-18, May-19, March-20 , Marks 5
2.24	Form Validation in JavaScript	 Dec.-18 ,..... Marks 5
2.25	jQuery	 May-19 , Marks 5
2.26	Overview of Angular JS	 May-18, 19, Dec.-18, 19 ,..... Marks 6
2.27	Multiple Choice Questions	

Part - I : JavaScript

2.1 Introduction to JavaScript

JavaScript was developed by Netscape in 1995. At that time its name was LiveScript. Later on Sun Microsystems joined the Netscape and then they developed **LiveScript**. And later on its name is changed to **JavaScript**.

2.1.1 Difference between Java and JavaScript

Sr. No.	Java	JavaScript
1.	Java is a programming language.	JavaScript is a scripting language.
2.	Java is an object oriented programming language.	Javascript is not an object oriented programming language. Its object model is far different than a typical object oriented programming language such as C++ or Java.
3.	Java is strongly typed language and the type checking is done at compile time.	In JavaScript variables need not be declared before their use. Checking the compatibility of type can be done dynamically.
4.	Objects in Java are static. That means the number of data members and method are fixed at compile time.	In JavaScript the objects are dynamic. That means we can change the total data members and method of an object during execution.

2.2 JavaScript in Perspective

Following are some features of JavaScript.

1. **Browser Support** : For running the JavaScript in the browser there is no need to use some plug-in. Almost all the popular browsers support JavaScripting.
2. **Structure Programming Syntax** : The Javascript supports much commonly the structured language type syntax. Similar to C-style the programming blocks can be written.
3. It automatically inserts the semicolon at the end of the statement, hence there is no need to write semicolon at the end of the statement in JavaScript.

4. **Dynamic Typing** : It supports dynamic typing, that means the data type is bound to the value and not to the variable. For example one can assign integer value to a variable say 'a' and later on can assign some string value to the same variable in JavaScript.
5. **Run Time Evaluation** : Using the **eval** function the expression can be evaluated at run time.
6. **Support for Object** : JavaScript is object oriented scripting language. However handling of objects in JavaScript is somewhat different than the conventional object oriented programming languages. JavaScript has a small number of in-built objects.
7. **Regular Expression** : JavaScript supports use of regular expressions using which the text-pattern matching can be done. This feature can be used to validate the data on the web page before submitting it to the server.
8. **Function Programming** : In JavaScript functions are used. One function can accept another function as a parameter. Even, one function can be assigned to a variable just like some data type. The function can be run without giving the name.

2.3 Basic Syntax

The JavaScript can be directly embedded within the HTML document or it can be stored as external file.

Directly embedded JavaScript

The syntax of directly embedding the JavaScript in the HTML is

```
<script type="text/javascript">
    ...
    ...
    ...
</script>
```

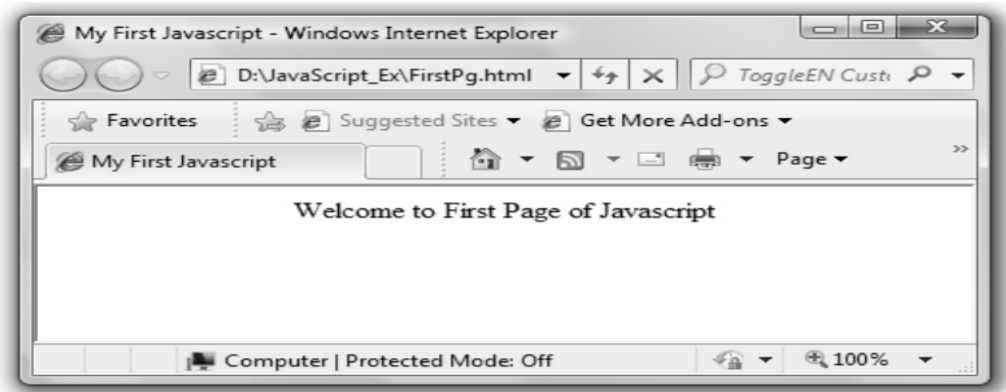
Example 2.3.1 Write a JavaScript to display welcome message in JavaScript.

Solution :

```
<!DOCTYPE html >
<html >
  <head>
    <title> My First Javascript </title>
  </head>
```

```
<body>
  <center>
    <script type="text/javascript">
      /*This is the First JavaScript*/
      document.write(" Welcome to First Page of Javascript");
    </script>
  </center>
</body>
</html>
```

Output



Script Explanation :

In above script

- 1) We have embedded the javaScript within

```
<script type="text/javascript">
...
</script>
```

- 2) A comment statement using /* and */. Note that this type of comment will be recognized only within the <script> tag. Because, JavaScript supports this kind of comment statement and not the XHTML document.
- 3) Then we have written document.write statement, using which we can display the desired message on the web browser.

Indirectly Embedding JavaScript

If we want to embed the JavaScript indirectly, that means if the script is written in some another file and we want to embed that script in the HTML document then we must write the script tag as follows -

```
<script type="text/javascript" src="MyPage.js">
...
...
...
</script>
```

JavaScript is which is to be embedded is in the file **MyPage.js**

We will follow the following steps to use the external JavaScript file.

Step 1 : Create an XHTML document as follows -

XHTML Document[FirstPg.html]

```
<!DOCTYPE html >
<html>
  <head>
    <title> My First Javascript </title>
  </head>
  <body>
    <center>
      <script type="text/javascript" src="my_script.js">
      </script>
    </center>
  </body>
</html>
```

This is an external javascript file; it can be specified with the attribute **src**

Step 2 :

JavaScript[my_script.js]

```
/*This is the First JavaScript*/
document.write(" Welcome to First Page of Javascript");
```

Step 3 : Open the HTML document in Internet Explorer and same above mentioned output can be obtained.

Advantages of indirectly embedding of JavaScript

1. Script can be hidden from the browser user.
2. The layout and presentation of web document can be separated out from the user interaction through the JavaScript.

Disadvantages of indirectly embedding of JavaScript

1. If small amount of JavaScript code has to be embedded in XHTML document then making a separate JavaScript file is meaningless.

2. If the JavaScript code has to be embedded at several places in XHTML document then it is complicated to make separate JavaScript file and each time invoking the code for it from the XHTML document.

2.4 Identifiers, Keywords and Comments

1. Identifiers

Identifiers are the names given to the variables. These variables hold the data value. Following are some conventions used in JavaScript for handling the identifiers -

1. Identifiers must begin with either letter or underscore or dollar sign. It is then followed by any number of letters, underscores, dollars or digits.
2. There is no limit on the length of identifiers.
3. The letters in the identifiers are case-sensitive. That means the identifier INDEX, Index, index, inDex are considered to be distinct.
4. Programmer defined variable names must not have upper case letters.

2. Reserved words

Reserved words are the special words associated with some meaning. Various reserve words that are used in JavaScript are enlisted as below -

break	Continue	delete	for	in	return	throw	var	with
case	default	else	function	instanceof	switch	try	void	
catch	do	finally	if	new	this	typeof	while	

3. Comments

JavaScript supports following comments.

1. The // i.e a single line comment can be used in JavaScript.
2. The /* and */ can be used as a multi-line comment.
3. The XHTML <!--> and <--> is also allowed in JavaScript

4. Semicolon

While writing the JavaScript the web programmer must give semicolon at the end of the statements.

2.5 Data Types

SPPU : May-18, 19, Marks 5

JavaScript uses six types of values - number, string, boolean, null, object and function

Number : Number is a numeric value that can be integer or float. It can be used calculation.

String : String is a collection of characters. It is enclosed within single quote or double quote. A string may contain numbers but these numbers can not be used for calculations.

Boolean : The Boolean values are **true** and **false**. These values can be compared with the variables or can be used in assignment statement.

Null : The null value can be assigned by using the reserved **word null**. The **null** means no value. If we try to access the null value then a runtime error will occur.

Object : Object is for an entity that represents some value. For example - window object is a value using which the window is created. Form is an object upon which some components such as button, checkbox, radio buttons can be placed and used.

Review Questions

1. Explain the different data types available in JavaScript.

SPPU : May-18, Marks 5

2. What are different data types available in JavaScript ? Explain with correct syntax of declaration of it.

SPPU : May-19, Marks 5

2.6 Literals

There are two types of literals used in JavaScript and those are numeric literals and string literals. The numeric literals are called **numbers**. These numbers can include integer values, floating point or double precision values. For example -

10

10.3

10.0

10.

10E3

10.2E4

10.e2

10e-3

are all valid numeric literals.

The string literals are the sequence of characters. It can be written in double quotes " " or in single quotes ' '. For example

'Rain Rain come soon '

2.7 Other Primitive Types

The other primitive types are -

- **Boolean**

The Boolean values are **true** and **false**. These values can be compared with the variables or can be used in assignment statement.

- **Null**

The null value can be assigned by using the reserved word **null**. The **null** means no value. If we try to access the **null** value then a runtime error will occur.

- **Undefined**

If a variable is explicitly declared and not assigned any value to it then it is an undefined value. If we try to display the undefined value then on the browser the word "undefined" will be displayed. There is no reserved word for undefined primitive type.

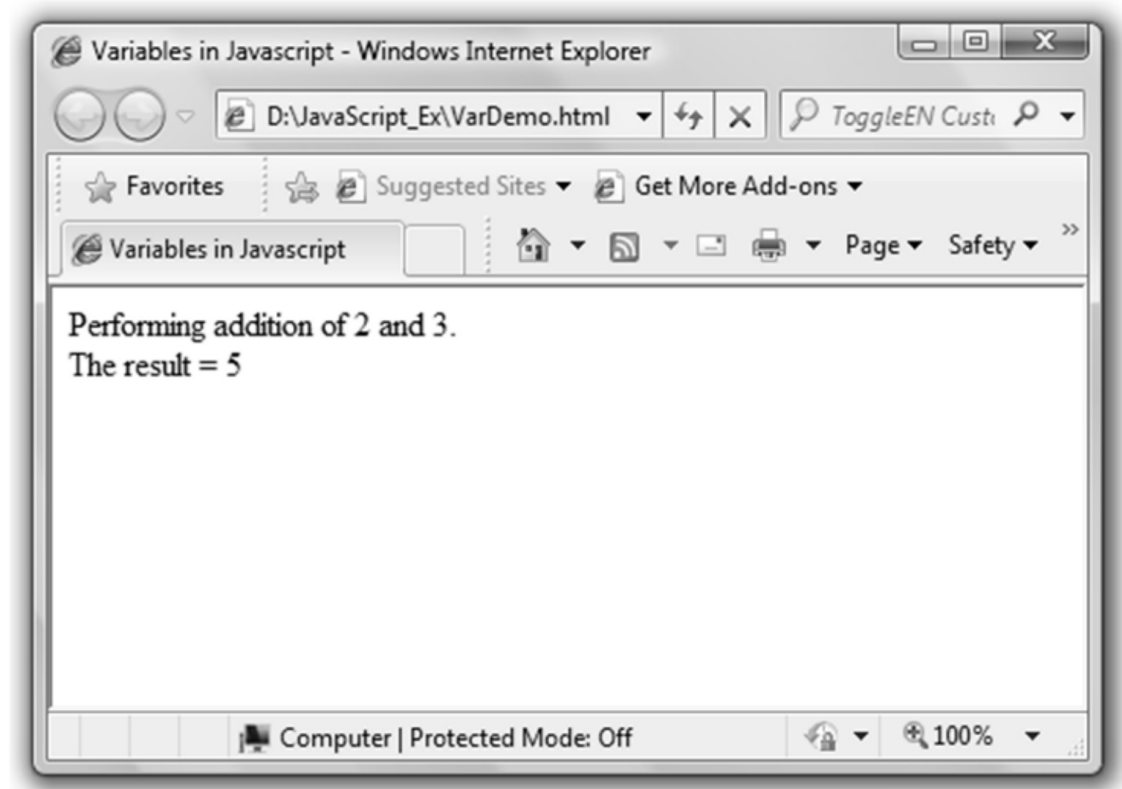
2.8 Variables

In JavaScript we can declare the variable using the reserved word **var**. The value of this variable can be anything; it can be numeric or it can be string or it can be a Boolean value.

JavaScript[VarDemo.html]

```
<!DOCTYPE html >
<html >
<head>
<title> Variables in Javascript </title>
</head>
<body>
<script type="text/javascript">
var a,b,c;
var string;
a=2;
b=3;
c=a+b;
string ="The result = ";
document.write("Performing addition of 2 and 3. "+"<br/>");
document.write(string);
document.write(c);
</script>
</body>
</html>
```

Variable declaration is done using **var**.
Note that there is no data type required
for handling variables.

Output

Note that using **var** we can define the variable which is of type numbers (2, 3 or 5) as well as the string "The result".

2.9 Operators

Various operators used by JavaScript are as shown in following Table -

Type	Operator	Meaning	Example
Arithmetic	+	Addition or unary plus	c = a + b
	-	Subtraction or unary minus	d = - a
	*	Multiplication	c = a*b
	/	Division	c = a/b
	%	Mod	c = a%b
Relational	<	Less than	a < 4
	>	Greater than	b > 10

	<=	Less than equal to	b <= 10
	>=	Greater than equal to	a >= 5
	==	Equal to	x == 100
	!=	Not equal to	m != 8
Logical	&&	And operator	0 && 1
		Or operator	0 1
Assignment	=	Is assigned to	a = 5
	+=	add a value then assign	a += 5 means a = a + 5
	-=	subtract a value then assign	a -= 10 means a = a - 10
	/=	divide the value then assign	a /= 2
	*=	multiply and then assign	a *= 5
Increment	++	Increment by one	++i or i++
Decrement	--	Decrement by one	--k or k--

Conditional operator

The conditional operator is ? The syntax of conditional operator is

Condition?expression1:expression 2

Where expression1 denotes the true condition and expression2 denotes false condition.

For example :

a > b ? true : false

This means that if a is greater than b then the expression will return the value true otherwise it will return false.

Expressions

JavaScript makes use of mathematical statements using operators and operands.

These statements are called expressions.

For example - Following Fig. 2.9.1 shows a simple expression.

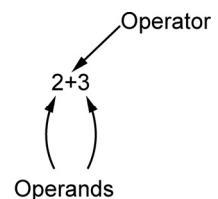


Fig. 2.9.1 An expression

2.10 Input and Output

2.10.1 The document.write

- For displaying the message on the web browser the basic method being used is **document.write**. To print the desired message on the web browser we write the message in double quotes inside the document.write method.
- **For example**

```
document.write("Great India");
```
- We can pass the variable instead of a string as a parameter to the document.write.

For example

```
var my_msg="Great India";  
document.write(my_msg);
```

Note that the variable my_msg should not be passed in double quotes to document.write otherwise the result the string "my_msg" will be printed on the browser instead of the string "Great India".

- We can combine some HTML tags with the variable names in document.write so that the formatted display can be possible on the web browser.

For example - if we want to print the message "Great India" on the web browser in bold font then we can write following statements -

```
var my_msg="Great India";  
document.write("<strong>" + my_msg + "</strong>");
```

2.11 Statements

2.11.1 If Statement

As a selection statement we use various types of if statements. These statements are -

1. **if** - The syntax of if is

```
if(condition)
```

statement if the condition is true

2. **if...else** - The syntax of if...else is

```
if(condition)
```

statement if the condition is true

```
else
```

statement if the condition is false

3. **if...else if** - The syntax of if...else if is

```
if(condition)
```

statement if the condition is true

else if(condition)

statement if another condition is true

else if(condition)

statement if another condition is true

...

else

statement

Some times we can have **nested if...**statements, which work similar to C or C++. Here is a sample JavaScript.

JavaScript[IfDemo.html]

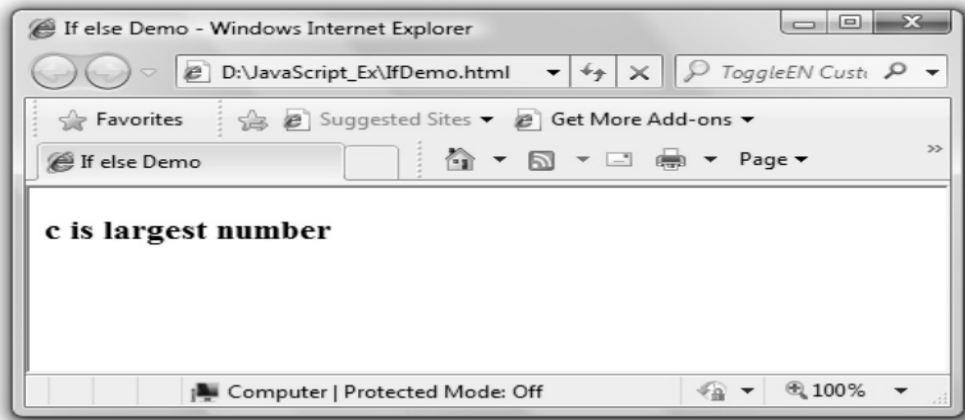
```
<!DOCTYPE html>
<html>
<head>
  <title>If else Demo</title>
</head>
<body>
<script type="text/javascript">
  var a,b,c;
  a=10;b=20;c=30;
  if(a>b)
  {
    if(a>c)
      document.write("<h3>a is largest number</h3>");
    else
      document.write("<h3>c is largest number</h3>");
  }
  else
  {
    if(b>c)
      document.write("<h3>b is largest number</h3>");
    else
      document.write("<h3>c is largest number</h3>");
  }
</script>
</body>
</html>
```

Script Explanation

The above document is to find the largest number from the given three numbers. We have used nested if...else statements. The script is pretty simple, in which first of all we

have compared a with b, if a is greater than b then we have compared a with c. Similarly in the else part b is compared with c. Thus all the three numbers get compared with each other and the largest number is found out among the three. Since already in our program we have set the a, b and c values as 10, 20 and 30 respectively the output should be c is largest number and here comes the output on web browser.

Output



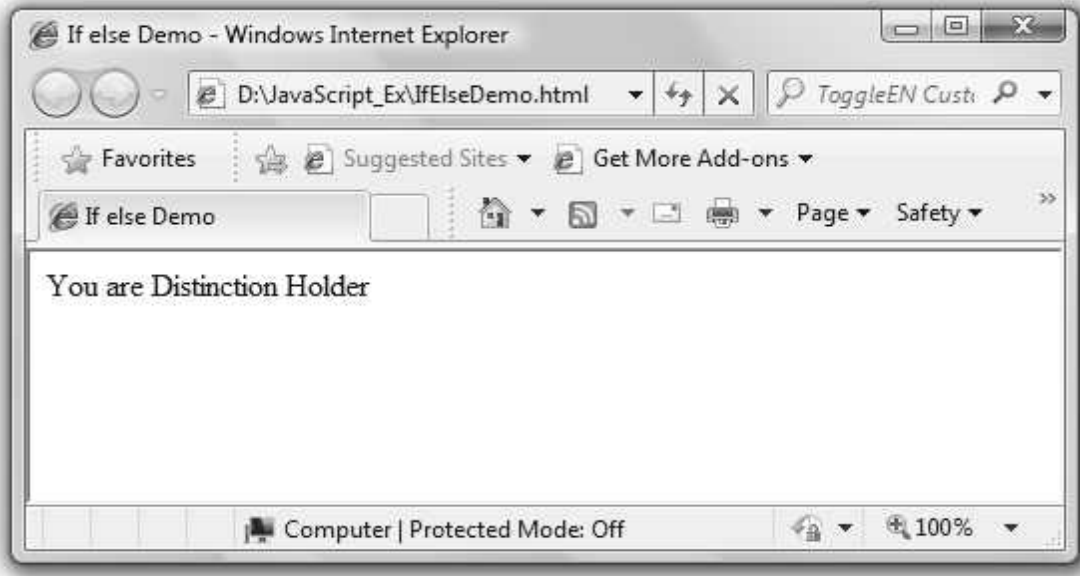
We can use various operators such as arithmetic operator, logical operator, relational operators and so on in the if statement. Following JavaScript makes use of such operators.

JavaScript[IfElseDemo.html]

```
<!DOCTYPE html>
<html>
<head>
  <title>If else Demo</title>
</head>
<body>
  <script type="text/javascript">
    var marks;
    marks=80;
    if(marks<40)
      document.write("You are failed");
    else if(marks>=40 && marks<50)
      document.write("You are passed");
    else if(marks>=50 && marks<60)
      document.write("You have got Second class");
    else if(marks>=60 && marks<66)
      document.write("You have got First class");
    else
      document.write("You are Distinction Holder");
```

```
</script>
</body>
</html>
```

Output



In above script if we change the values of the marks variable then appropriate message will get displayed on the web browser.

2.11.2 Switch Statement

Switch case statement is basically to execute the desired choice. The syntax of this control structure is similar to that in C or C++.

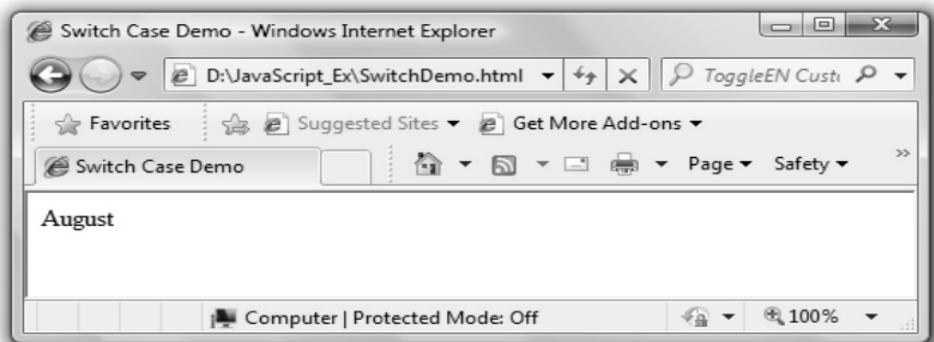
```
switch(choice)
{
    case identifier: statement
        break
    ...
    default: statement
}
```

JavaScript[SwitchDemo.html]

```
<!DOCTYPE html>
<html>
<head>
    <title>Switch Case Demo</title>
</head>
<body>
```

```
<script type="text/javascript">
d=new Date();
ch=d.getMonth();
switch(ch)
{
case 0: document.write("January");
break;
case 1: document.write("February");
break;
case 2: document.write("March");
break;
case 3: document.write("April");
break;
case 4: document.write("May");
break;
case 5: document.write("June");
break;
case 6: document.write("July");
break;
case 7: document.write("August");
break;
case 8: document.write("September");
break;
case 9: document.write("October");
break;
case 10: document.write("November");
break;
case 11: document.write("December");
break;
}
</script>
</body>
</html>
```

Output



Script Explanation : In this script,

- 1) We have used the object **Date** which returns the value of the current date.
- 2) Then using **getMonth()** method the month value can be obtained. The month value starts from 0 to 11 representing January to December. This value is basically obtained from system date function. The month is taken in variable **ch** according to the value in **ch** the corresponding case will get executed.

2.11.3 Loop Statement

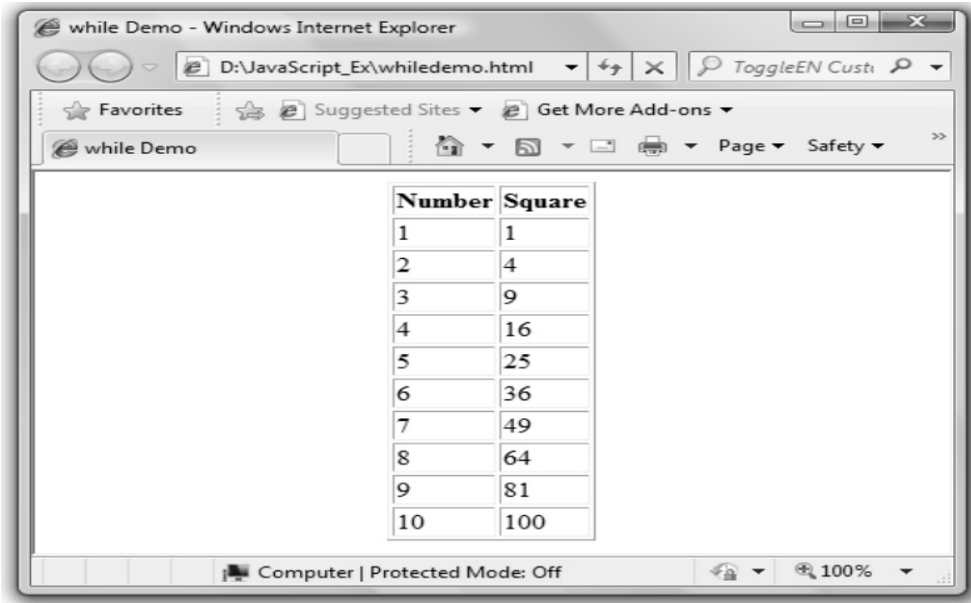
2.11.3.1 while Loop

while statements help us in implementing the iterative logic of the program. The syntax of **while** is as follows -

```
Some initial condition;
while(terminating condition)
{
    some statements;
    stepping condition;
}
```

The **while** is implemented by following JavaScript.

```
<!DOCTYPE html>
<html>
<head>
    <title>while Demo</title>
</head>
<body>
    <table border=1 align="center">
        <th>Number</th> <th>Square</th>
        <script type="text/javascript">
            i=1;
            while (i<=10)
            {
                document.write("<tr><td>" + i + "</td><td>" + (i*i) + "</td></tr>");
                i++;
            }
        </script>
    </table>
</body>
</html>
```

Output**2.11.3.2 do..while Loop**

The do-while loop is similar to the while loop, the only difference is that the do-while executes at least once. The syntax of do...while is

```
do
{
...
}while(condition);
```

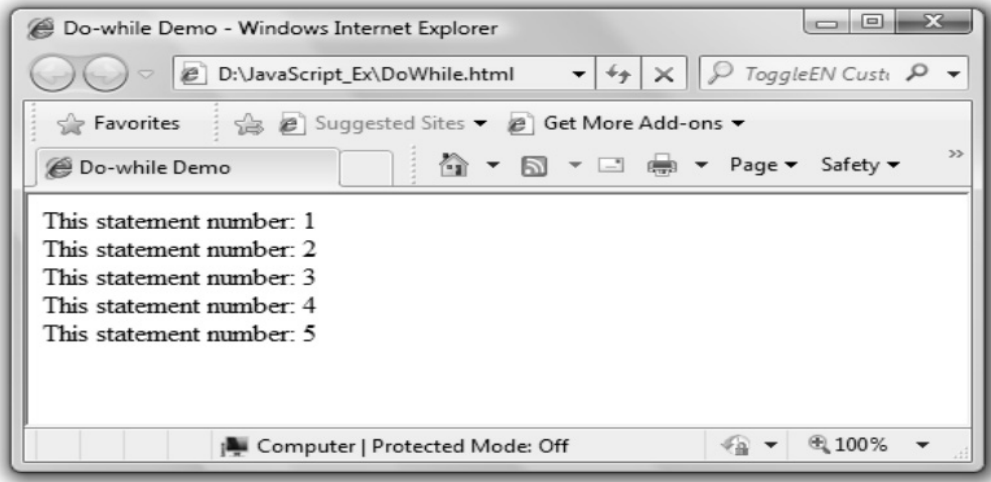
The following JavaScript illustrates use of do...while

JavaScript[DoWhile.html]

```
<!DOCTYPE html>
<html>
<head>
  <title>Do-while Demo</title>
</head>
<body>
  <script type="text/javascript">
    counter=1;
    do
    {
      document.write("This statement number: "+counter);
      document.write("<br>");
      counter++;
    }
```

```
}while(counter<=5);  
</script>  
</body>  
</html>
```

Output



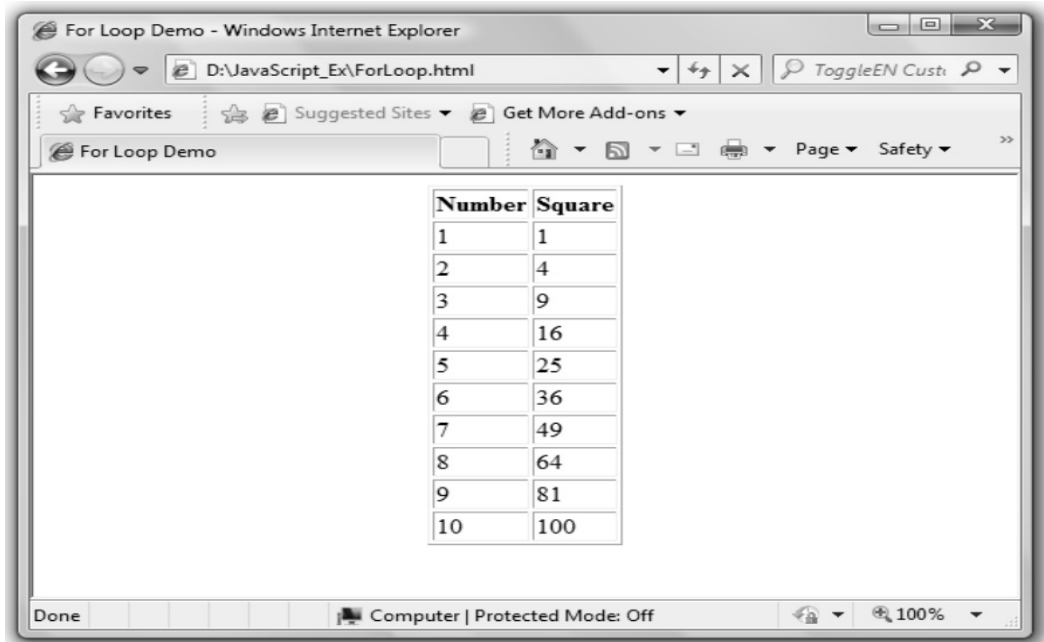
2.11.3.3 for Loop

This is the most commonly used programming construct. The syntax of for loop is
for(initial condition; terminating condition; stepping condition)

Here is a JavaScript which makes use of for loop

JavaScript[ForLop.html]

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>For Loop Demo</title>  
</head>  
<body>  
  <table border=1 align="center">  
    <th>Number</th> <th>Square</th>  
    <script type="text/javascript">  
      for (i=1; i<=10; i++)  
      {  
        document.write("<tr><td>"+i+"</td><td>"+(i*i)+"</td></tr>");  
      }  
    </script>  
  </table>  
</body>  
</html>
```

Output

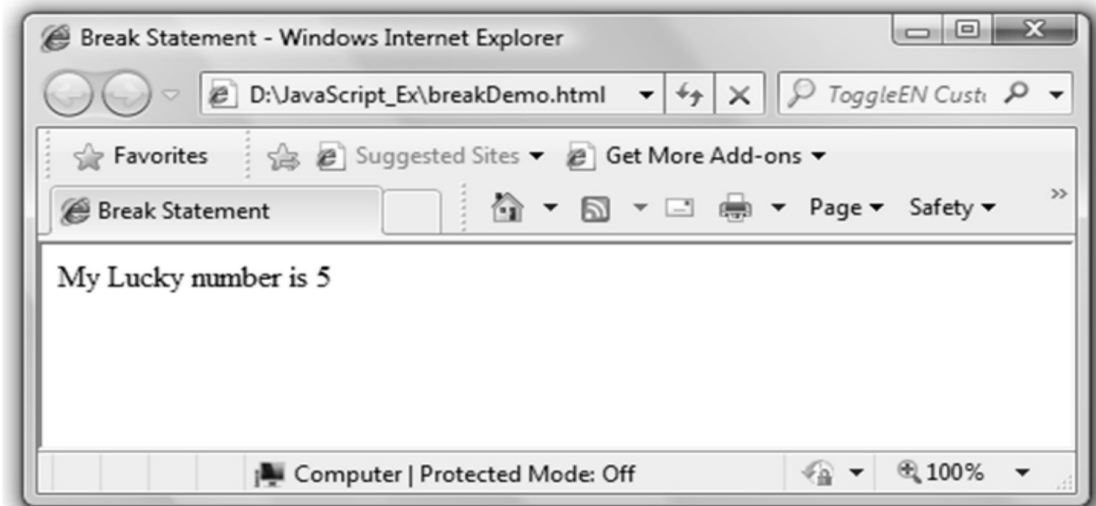
Number	Square
1	1
2	4
3	9
4	16
5	25
6	36
7	49
8	64
9	81
10	100

2.11.3.4 break Statement

Similar to C or C++, the break statement is used to break the loop. It is given by the keyword **break**. The following script shows the use of break.

JavaScript[breakDemo.html]

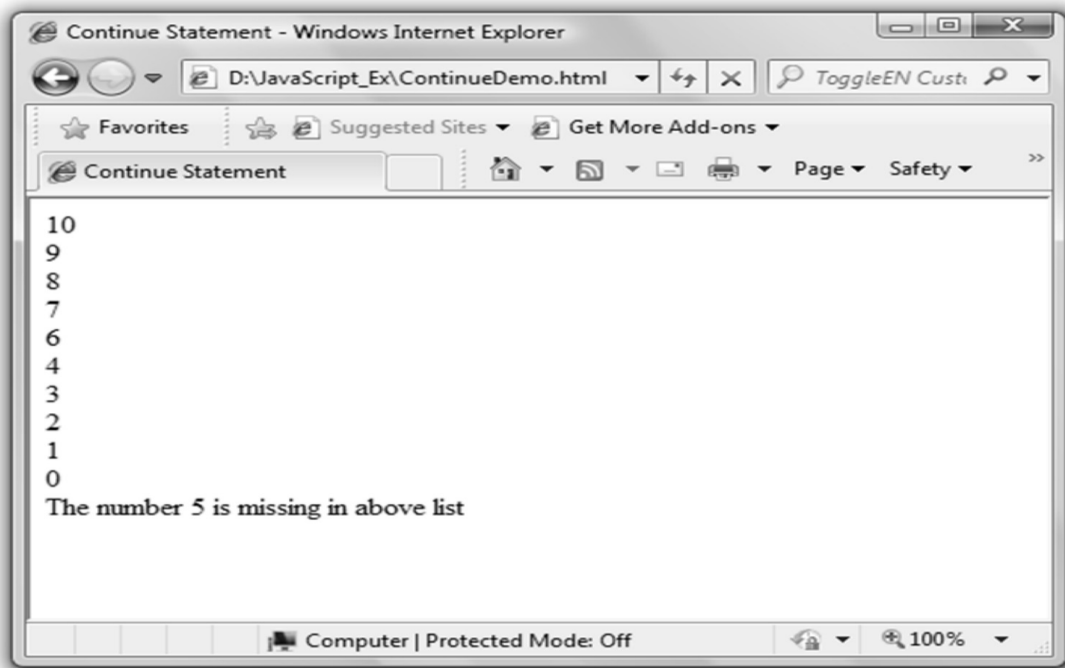
```
<!DOCTYPE html>
<html>
<head>
  <title>Break Statement</title>
</head>
<body>
  <script type="text/javascript">
    for(i=10;i>=0;i--)
    {
      if(i==5)
        break;
    }
    document.write("My Lucky number is "+i);
  </script>
</body>
</html>
```

Output**2.11.3.5 The continue Statement**

The continue statement is used in a loop in order to continue (skip). The keyword **continue** is used to make use of continue statement in a loop.

Javascript Program[ContinueDemo.html]

```
<!DOCTYPE html>
<html>
<head>
<title>Continue Statement</title>
</head>
<body>
<script type="text/javascript">
  for(i=10;i>=0;i--)
  {
    if(i==5)
    {
      x=i;
      continue;
    }
    document.write(i);
    document.write("<br>");
  }
  document.write("The number "+x+" is missing in above list" );
</script>
</body>
</html>
```

Output**2.12 Querying and Handling Properties**

- One of the important features of JavaScript is its **interactivity** with the user.
- There are **three types of popup boxes** used in JavaScript by which user can interact with the browser.

alert box : In this type of popup box some message will be displayed.



confirm box : In this type of popup box in which the message about confirmation will be displayed. Hence it should have two buttons **Ok** and **Cancel**.



Prompt box is a type of popup box which displays a text window in which the user can enter something. Hence it has two buttons **Ok** and **Cancel**.

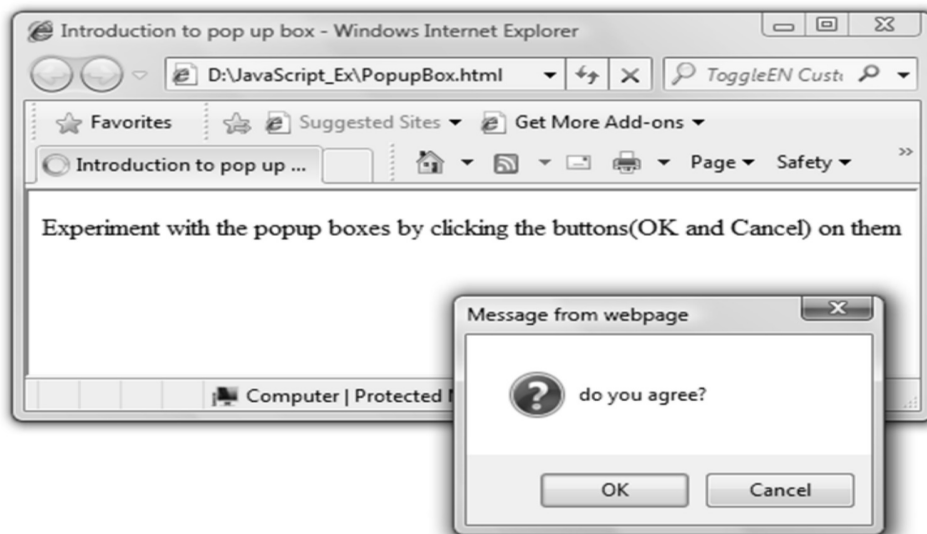


JavaScript[PopupBox.html]

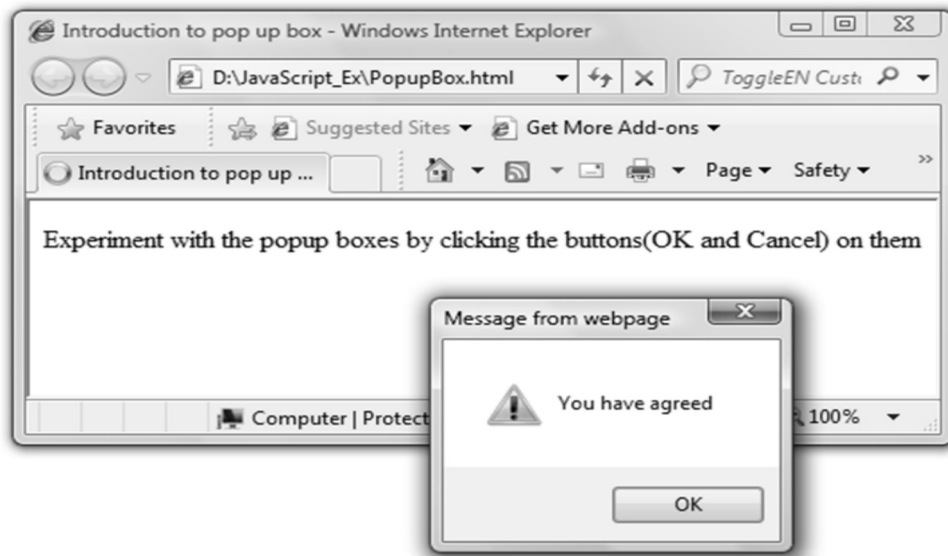
```
<!DOCTYPE html>
<html >
<head>
  <title>Introduction to pop up box</title>
</head>
<body>
<p>Experiment with the popup boxes by clicking the buttons(OK and Cancel) on them</p>

<script type="text/javascript">
  if(confirm("do you agree?"))
    alert("You have agreed");
  else
    input_text=prompt("Enter some string here...", "");
  /*the value entered in prompt box is returned
  and stored in the variable text */
    alert("Hi "+input_text);
</script>
</body>
</html>
```

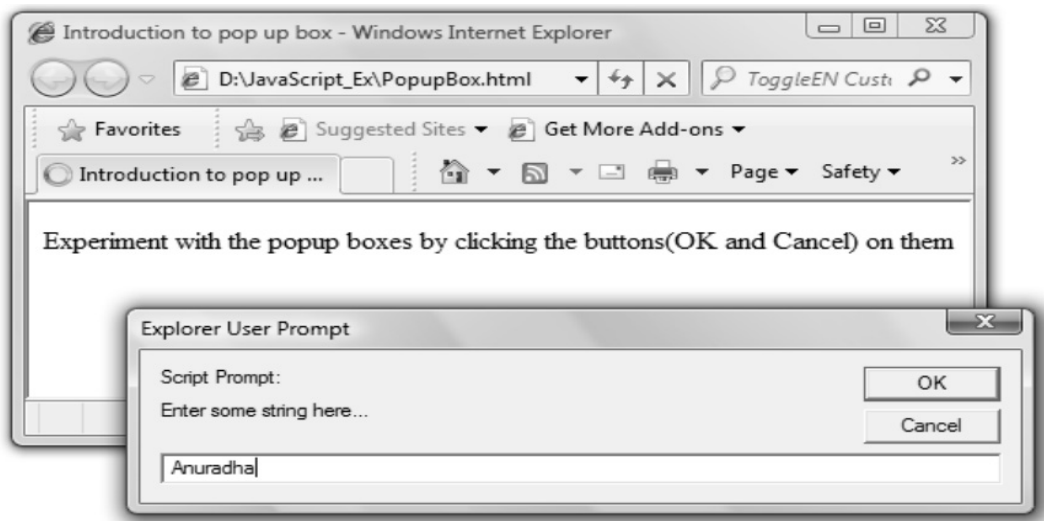
Output



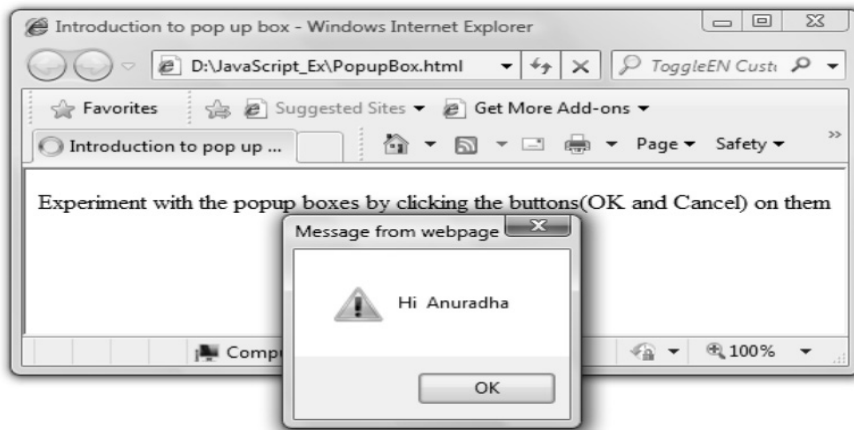
If we click on OK button then an alert box will appear.



On load, the confirm box appears and if we click on Cancel button then the prompt box will appear. We can type some string within it.



Click OK button and we will get the alert box as follows -



Thus alert, confirm and prompt boxes cause the browser to wait for user response. The user responds by clicking the button on these popup boxes.

Script Explanation

On loading this script using web browser first of all a confirm box will be displayed. If we click on OK button an alert box will appear. Otherwise a prompt box will be displayed. Again if we click on the OK button of prompt box again an alert box will appear which will display the string which you have entered on prompt box. If you run the above script you will get all these effects.

Example 2.12.1 *Develop a javascript to generate 'ARMSTRONG NUMBERS' between the ranges 1 to 100. [e.g. : 153 is an Armstrong number, since sum of the cube of the digits is equal to the number i.e. $1^3 + 5^3 + 3^3 = 153$].*

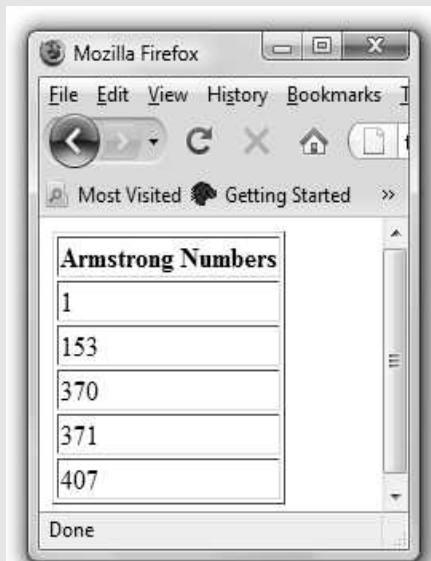
Solution :

```
<html>
<body>
<table border="1" align="center">
<th>Armstrong Numbers</th>
<script type="text/javascript">
var num,i,temp,sum;
var n=0;
i=1;
do
{
  num=i;
  sum=0;
  while(num>0)
  {
```

```

n=num%10;
n=parseInt(n);
num=num/10;
num=parseInt(num);
sum=sum+(n*n*n);
}
if(sum==i)
{
document.write("<tr><td>"+i+"</td></tr>");
}
i++;
}while(i<=1000);
</script>
</table>
</body>
</html>

```



Example 2.12.2 Write a JavaScript that displays table as per following : (Calculate the squares and cubes of the numbers from 0 to 10).

Number	Square	Cube
1	1	1
2	4	8
3	9	27
4	16	64
5	25	125
6	36	216
7	49	343
8	64	512
9	81	729
10	100	1000

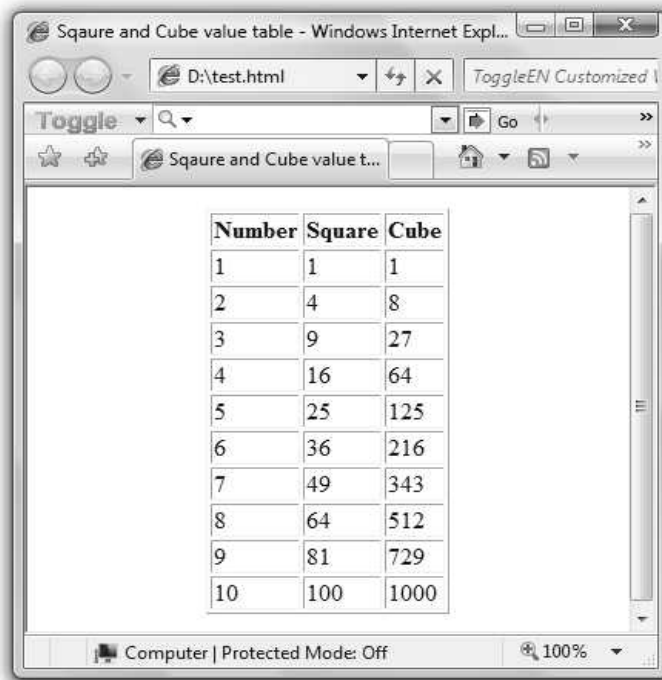
Solution :

```

<html>
<head>
<title>Sqaure and Cube value table</title>
</head>
<body>
<table border=1 align="center">
<th>Number</th><th>Square</th><th>Cube</th>
<script type="text/javascript">
for (i=1; i<=10; i++)
{
document.write("<tr><td>"+i+"</td><td>"+(i*i)+"</td><td>"+(i*i*i)+"</td></tr>");

```

```
}  
</script>  
</table>  
</body>  
</html>
```



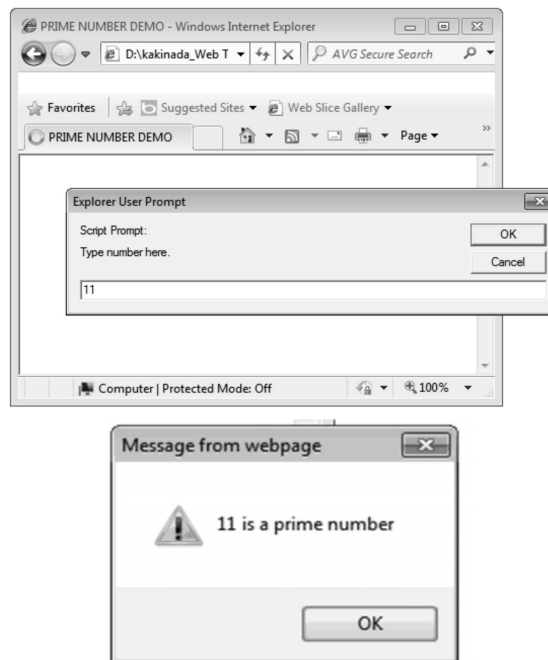
Example 2.12.3 Write a script that reads an integer and displays whether it is a prime number or not.

Solution :

```
<html>  
<head>  
  <title>PRIME NUMBER DEMO</title>  
</head>  
<body>  
  <script type="text/javascript">  
    var num=prompt("Type number here.", "");  
    var b;  
    var flag=1;  
    for(i=2;i<num;i+ +)  
    {  
      b=num%i;  
      if(b==0)  
      {
```

```
    flag=0;
    break;
  }
}
if(flag==0)
    alert(num+" is not a prime number");
else
    alert(num+" is a prime number");
</script>
</body>
</html>
```

Output



2.13 Functions

SPPU : April-18, Dec.-18, Marks 5

- We can write the functions in the JavaScript for bringing the modularity in the script.
- Separate functions can be created for each separate task. This ultimately helps in finding the bug from the program efficiently.

2.13.1 Defining a Function

- We can define the function anywhere in the script either in head or body section or in both. But it is a standard practice to define the function in the head section and call that function from the body section.
- The keyword **function** is used while defining the function.
- The syntax for defining the function is

```
function name_of_function (arg1,arg2,...argn)
{
    ...
    Statements
}
```

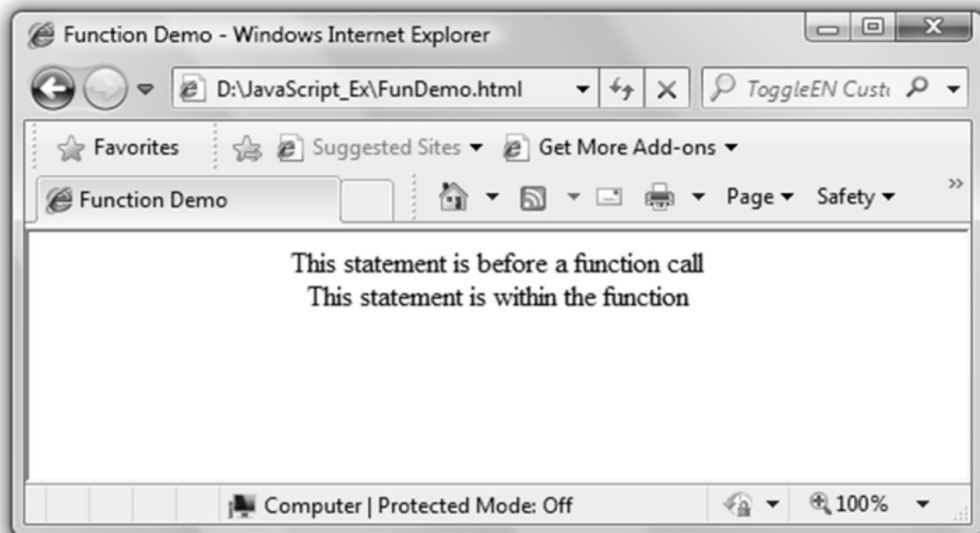
2.13.2 Writing a Function

Here is a simple illustration in which we have written a function **my_fun()**

JavaScript[FunDemo.html]

```
<!DOCTYPE html>
<html>
<head>
<title>Function Demo</title>
<script type="text/javascript">
    function my_fun()
    {
        document.write("This statement is within the function");
    }
</script>
</head>
<body>
<center>
    <script type="text/javascript">
        document.write("This statement is before a function call");
        document.write("<br>");
        my_fun(); // call to the function
    </script>
</center>
</body>
</html>
```



Output**Script Explanation**

The above code is pretty simple. We have defined one function named **my_fun** in the **head** section of the HTML document and called it from the **body** section. The corresponding **write** statements are used to display the messages on the browser window.

2.13.3 Adding an Arguments

- We can pass some arguments to the function.
- The syntax is

```
function function_name(argument1, argument2...,argumentn)
{
    //body of function
}
```

2.13.4 Scope of Variable and Argument

- Scope is the block or area of program in which particular variable or argument is accessible.
- The scope is defined using two types of variables - **Local Scope** and **Global Scope**.

Local Scope

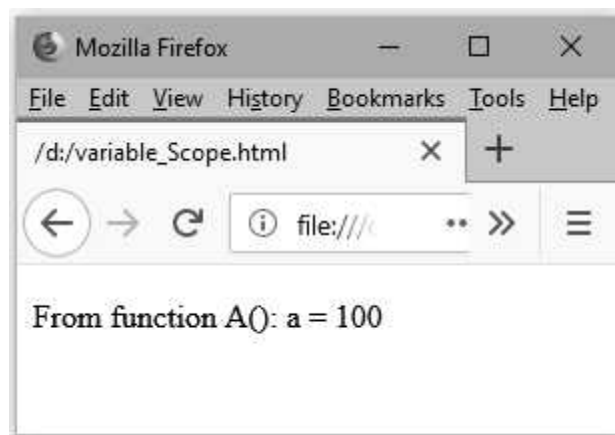
If a variable is defined inside a function then that variable is a local variable and its scope is a local scope. That also means that the local variable is accessible only within a function in which it is defined. It is not accessible outside that function.

Following program shows the use of local variable.

JavaScript example for demonstrating scope of local variable

```
<!DOCTYPE html>
<html>
  <head>
    <script type="text/javascript">
      function A()
      {
        var a=100//local variable
        document.getElementById("id1").innerHTML=
        "From function A(): a = "+a
      }
      function B()
      {
        document.getElementById("id2").innerHTML=
        "From function B(): a = "+a
      }
    </script>
  </head>
  <body>
    <p id="id1"></p>
    <p id="id2"></p>
    <script type="text/javascript">
      A()
      B()
    </script>
  </body>
</html>
```

Output



Script Explanation :

In above JavaScript,

- 1) We have created two functions namely A() and B().
- 2) Inside the **A()** function variable a is declared and initialized to the value 100. The value of this variable is displayed using the element<p>...</p>
- 3) Inside the **B()** function, we are trying to display the value of same variable a using <p>...</p> . But note that the output does not display value of variable a through function B(). This is because variable a is a local variable declared in function A(). Hence its scope is upto the function A(). Outside this function, the variable **a** is undefined.

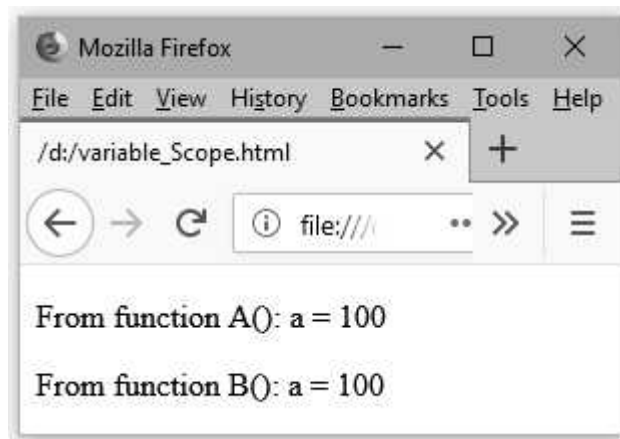
Global Scope

A variable is called **global variable**, if it is defined outside the function. The variable having global scope is accessible by any function.

Following example illustrates the use of global variable.

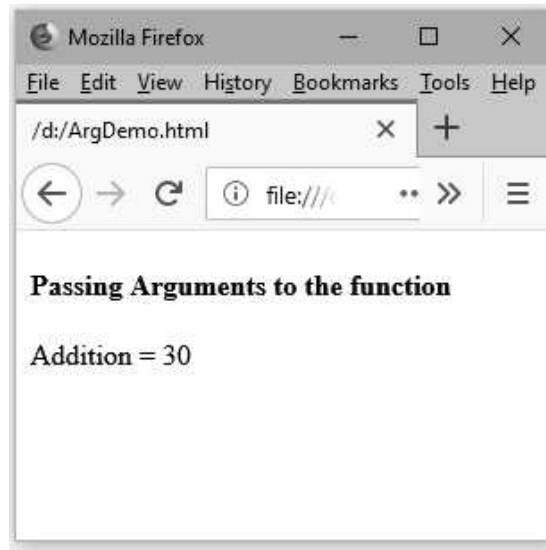
JavaScript example for demonstrating the scope of global variable

```
<!DOCTYPE html>
<html>
  <head>
    <script type="text/javascript">
      var a=100//Global variable
      function A()
      {
        document.getElementById("id1").innerHTML
        ="From function A(): a = "+a
      }
      function B()
      {
        document.getElementById("id2").innerHTML
        ="From function B(): a = "+a
      }
    </script>
  </head>
  <body>
    <p id="id1"> </p>
    <p id="id2"> </p>
    <script type="text/javascript">
      A()
      B()
    </script>
  </body>
</html>
```


Output**2.13.5 Calling a Function from with Argument**

In JavaScript, we can pass an argument to the function. Following JavaScript shows how to pass an argument to the function. In the following program I have written a simple function for addition of two numbers. The values of two numbers are 10 and 20 respectively. Inside this function the addition of these two arguments is carried out and result is displayed on the Browser page.

```
<!DOCTYPE html>
<html>
<head>
  <script type="text/javascript">
    function add(a,b)//function to which arguments a and b are passed
    {
      c=a+b
      document.write("Addition = "+c)
    }
  </script>
</head>
<body>
  <h4> Passing Arguments to the function </h4>
  <script type="text/javascript">
    var x=10;
    var y=20
    add(x,y)
  </script>
</body>
</html>
```

Output**2.13.6 Calling a Function without an Argument**

A function can also be called without passing any argument. In this case, all the required variables are declared and used within that function. Following program shows how to call a function without passing an argument.

JavaScript Document

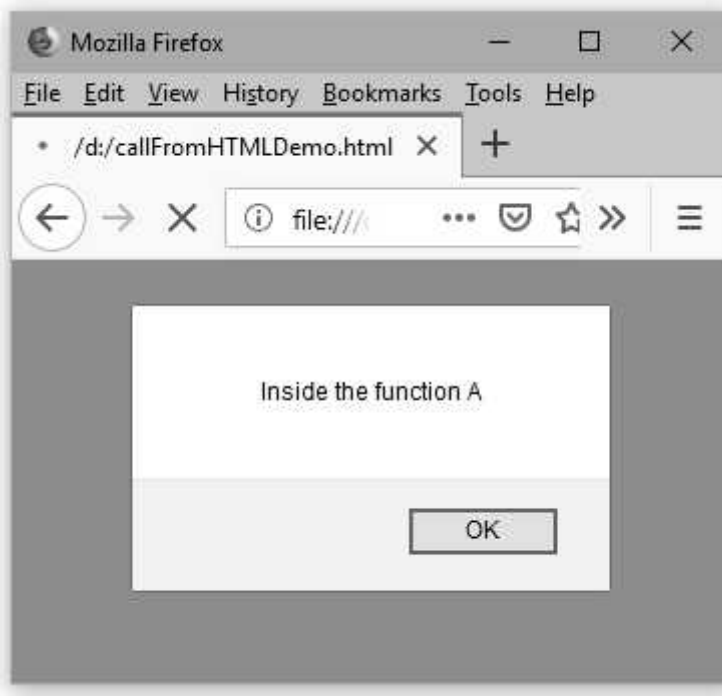
```
<!DOCTYPE html>
<html><head>
  <script type="text/javascript">
    function add()
    {
      var a=10
      var b=20
      c=a+b
      document.write("Addition = "+c)
    }
  </script>
</head>
<body>
  <h4> Function without passing argument </h4>
  <script type="text/javascript">
    add()
  </script>
</body>
</html>
```

Output**2.13.7 Calling a Function from HTML**

For calling a function from HTML normally, JavaScript events are used. For example -

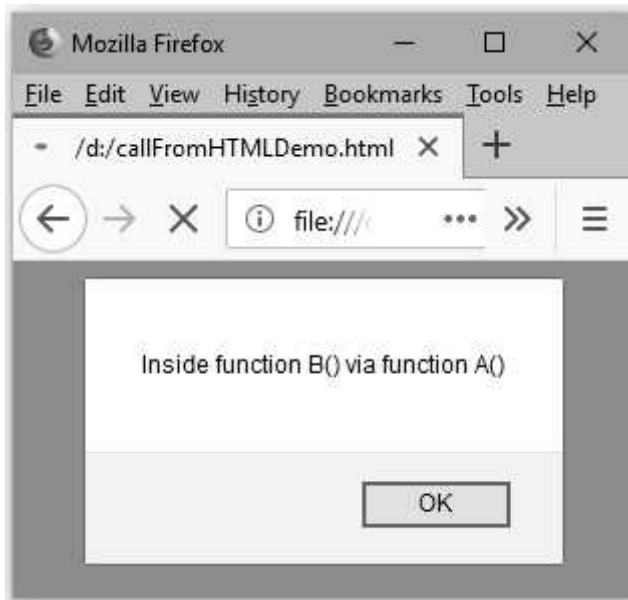
JavaScript Document

```
<!DOCTYPE html>
<html>
  <head>
    <script type="text/javascript">
      function A()
      {
        alert("Inside the function A");
      }
    </script>
  </head>
  <body onload="A()">
  </script>
</body>
</html>
```

Output**2.13.8 Function Calling another Function**

We can call one function from another function. This is called nested functions. In the following code, there is a call for function B() from function A().

```
<!DOCTYPE html>
<html>
  <head>
    <script type="text/javascript">
      function A()
      {
        B();//calling another function
      }
      function B()
      {
        alert("Inside function B() via function A()");
      }
    </script>
  </head>
  <body onload="A()">
  </script>
</body>
</html>
```

Output**2.13.9 Returning a Value from Function**

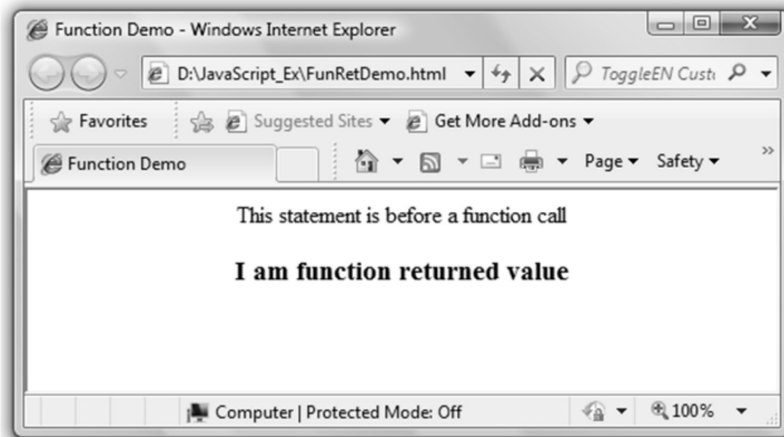
- We can return some value from the function using a keyword **return**.
- This return value is either stored in variable or directly displayed on the browser window.
- Following is a simple illustration in which the value is returned from the function and is directly displayed on the browser window using **document.write**.

JavaScript[FunRetDemo.html]

```
<!DOCTYPE html>
<html>
<head>
<title>Function Demo</title>
<script type="text/javascript">
  function my_fun()
  {
    str="I am function returned value";
    return str;
  }
</script>
</head>
<body>
<center>
  <script type="text/javascript">
```

```
document.write("This statement is before a function call");  
document.write("<br>");  
document.write("<h3>" + my_fun() + "<h3>");  
</script>  
</center>  
</body>  
</html>
```

Output



Review Questions

1. List and brief any five functions in JavaScript.
2. Explain how to call JavaScript function from HTML page.

SPPU : April-18, Marks 5**SPPU : Dec.-18, Marks 5**

2.14 Built in Objects

In JavaScript object is a collection of properties. These properties are nothing but the members of the classes from Java or C++. For instance - in JavaScript the object Date() is used which happens to be the member of the class in Java.

2.14.1 Math Objects

- For performing the mathematical computations there are some useful methods available from **math** object.
- For example if we want to find out minimum of two numbers then we can write -

```
document.write(math.min(4.5,7.8));
```

The above statement will result in 4.5. Thus using various useful methods we can perform many mathematical computations.

- Here are some commonly used methods from **math** object.

Method	Meaning
sqrt(num)	This method finds the square root of num.
abs(num)	This method returns absolute value of num.
ceil(num)	This method returns the ceil value of num. For example ceil(10.3) will return 11.
floor(num)	This method returns the floor value of num. For example floor(10.3) will return 10.
log(num)	This method returns the natural logarithmic value of num. For example log(7.9) will return 2.
pow(a,b)	This method will compute the ab. For example pow(2,5) will return 32.
min(a,b)	Returns the minimum value of a and b.
max(a,b)	Returns the maximum value of a and b.
sin(num)	Returns the sine of num.
cos(num)	Returns the cosine of num.
tan(num)	Returns the tangent of num.
exp(num)	Returns the exponential value i.e. e^{num} .

JavaScript Program[MathDemo.html]

```
<!DOCTYPE html>
<html >
<head>
  <title>Math Demo</title>
</head>
<body>
  <center>
    <h3>Square root of 100 is </h3>
    <script type="text/javascript">
      var num=100;
      document.write("<h3>" + Math.sqrt(num) + "</h3>");
    </script>
  </center>
</body>
</html>
```

Output**2.14.2 Number Objects**

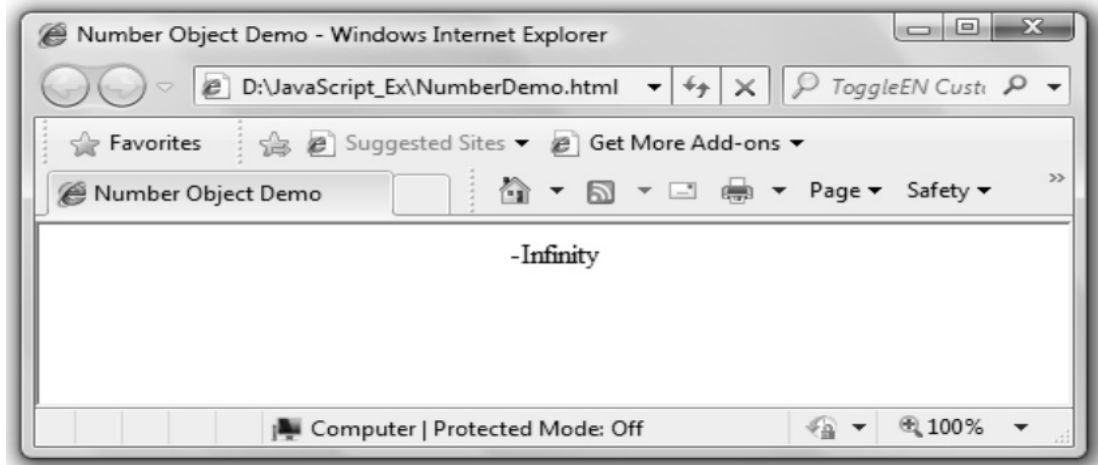
Various properties of number object are -

Property	Meaning
MAX_VALUE	Largest possible number gets displayed.
MIN_VALUE	Smallest possible number gets displayed.
NaN	When not a number then NaN is displayed.
PI	The value of PI gets displayed.
POSITIVE_INFINITY	The positive infinity gets displayed.
NEGATIVE_INFINITY	The negative infinity gets displayed.

Using **Number.property_name** we can display the property value. Following JavaScript uses the property of negative infinity.

JavaScript[NumberDemo.html]

```
<!DOCTYPE html>
<html>
<head>
  <title>Number Object Demo</title>
</head>
<body>
  <center>
    <script type="text/javascript">
      document.write(Number.NEGATIVE_INFINITY);
    </script>
  </center>
</body>
</html>
```


Output**2.14.3 Date Objects**

- This object is used for obtaining the date and time.
- This date and time value is based on computer's local time (system's time) or it can be based on GMT (Greenwich Mean Time).
- Nowadays this GMT is also known as UTC i.e. **Universal Co-ordinated Time**. This is basically a world time standard.
- Following are the commonly used methods of Date object.

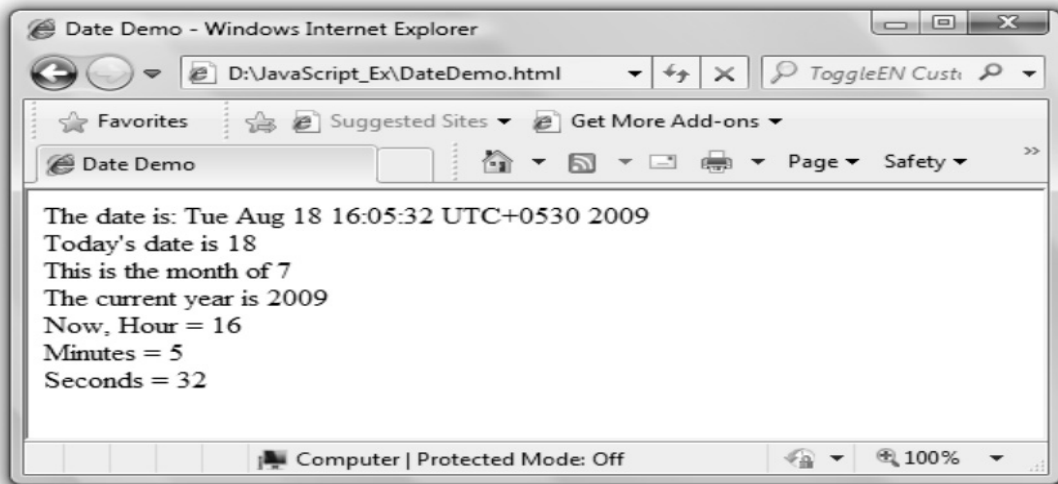
Method	Meaning
getTime()	It returns the number of milliseconds. This value is the difference between the current time and the time value from 1st January 1970.
getDate()	Returns the current date based on computers local time.
getUTCDate()	Returns the current date obtained from UTC.
getDay()	Returns the current day. The day number is from 0 to 6 i.e. from Sunday to Saturday.
getUTCDay()	Returns the current day based on UTC. The day number is from 0 to 6 i.e. from Sunday to Saturday.
getHours()	Returns the hour value ranging from 0 to 23, based on local time.
getUTCHours()	Returns the hour value ranging from 0 to 23, based on UTC timing zone.
getMilliseconds()	Returns the milliseconds value ranging from 0 to 999, based on local time.

getUTCMilliseconds()	Returns the milliseconds value ranging from 0 to 999, based on UTC timing zone.
getMinutes()	Returns the minute value ranging from 0 to 59, based on local time.
getUTCMinutes()	Returns the minute value ranging from 0 to 59, based on UTC timing zone.
getSeconds()	Returns the second value ranging from 0 to 59, based on local time.
getUTCSeconds()	Returns the second value ranging from 0 to 59, based on UTC timing zone.
setDate(value)	This function helps to set the desired date using local timing or UTC timing zone.
setHour(hr,minute,second,ms)	This function helps to set the desired time using local or UTC timing zone. The parameters that can be passed to this function are hour, minute, seconds and milliseconds. Only hour parameter is compulsory and rest all are the optional parameters.

In the following web document we are making use of **Date()** object and some useful methods of it.

JavaScript[DateDemo.html]

```
<!DOCTYPE html>
<html>
<head>
  <title>Date Demo</title>
</head>
<body>
  <script type="text/javascript">
    var my_date=new Date();
    document.write("The date is: "+my_date.toString()+"<br>");
    document.write("Today's date is "+my_date.getDate()+"<br>");
    document.write("This is the month of "+my_date.getMonth()+"<br>");
    document.write("The current year is "+my_date.getFullYear()+"<br>");
    document.write("Now, Hour = "+my_date.getHours()+"<br>");
    document.write("Minutes = "+my_date.getMinutes()+"<br>");
    document.write("Seconds = "+my_date.getSeconds()+"<br>");
  </script>
</body>
</html>
```

Output**Script Explanation :**

1. In the above script we have created an instance *my_date* of the object **Date()**.
2. Then using **my_date.toString()** method we can display the current date along with the time.
3. Then using the functions like `getDate()`, `getMonth()`, `getFullYear()` we are displaying the date, month and year separately.
4. Similarly using the functions `getHours()`, `getMinutes()` and `getSeconds()` we can display the current hour, minute and seconds separately.

2.14.4 Boolean Objects

- This object is the simplest kind of object which is used especially when we want to represent **true** and **false** values.
- Here is a simple JavaScript in which the Boolean type variable is used -

Javascript Program

```
<html>
<body>
<script type="text/javascript">
var temp=new Boolean(false);
document.write("<b>"+"The boolean value is: ");
document.write(temp.toString());
</script>
</body>
</html>
```

Output



2.14.5 String Objects

- String is a collection of characters.
- In JavaScript using **string** object many useful string related functionalities can be exposed off.
- Some commonly used methods of string object are concatenating two strings, converting the string to upper case or lower case, finding the substring of a given string and so on.
- Here is a listing of some methods of string.

Method	Meaning
concat(str)	This method concatenates the two strings. For example - s1.concat(s2) will result in concatenation of string s1 with s2.
charAt(index_val)	This method will return the character specified by value index_val.
substring(begin,end)	This method returns the substring specified by begin and end character.
toLowerCase()	This function is used to convert all the uppercase letters to lower case.
toUpperCase()	This function is used to convert all the lowercase letters to upper case.
valueOf()	This method returns the value of the string.

There is one important property of string object and that is **length**. For example

```
var my_str="Hello";
var len;
len=my_str.length;
```

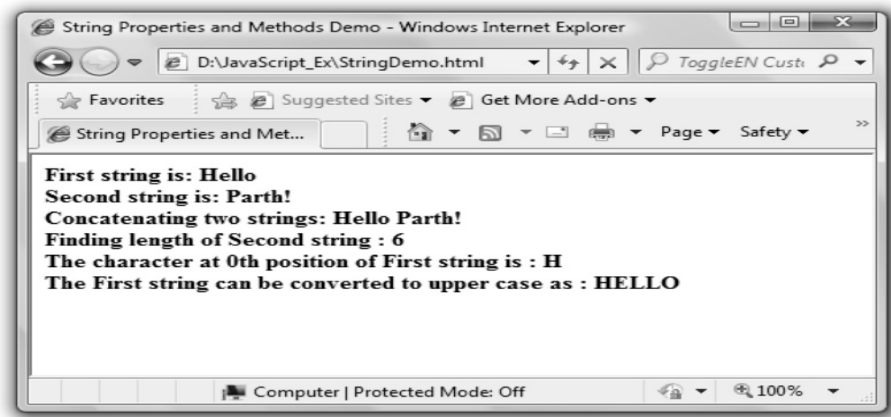
Length of the string "Hello" will be stored in the variable len

Here is sample program in which some methods of string object are used.

JavaScript[StringDemo.html]

```
<!DOCTYPE html>
<html>
<head>
  <title>String Properties and Methods Demo</title>
</head>
<body>
<strong>
  <script type="text/javascript">
    var s1="Hello ";
    var s2="Parth!";
    document.write("First string is: "+s1+"<br>");
    document.write("Second string is: "+s2+"<br>");
    document.write("Concatenating two strings: "+s1.concat(s2)+"<br>");
    document.write("Finding length of Second string : "+s2.length+"<br>");
    document.write("The character at 0th position of First string is: "+s1.charAt(0)+"<br>");
    document.write("The First string can be converted to upper case as : "+s1.toUpperCase());
  </script>
</strong>
</body>
</html>
```

Output



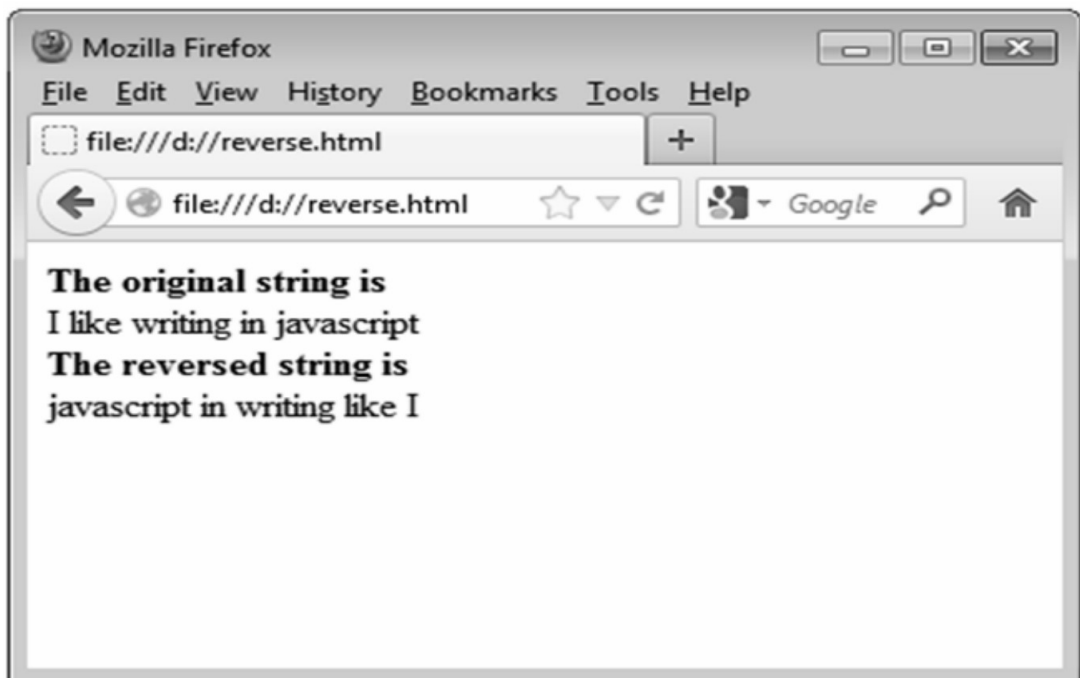
Example 2.14.1 Write a script that inputs a line of text, tokenizes it with String method split and displays the tokens in reverse order.

Solution :

```
<html>
<body>
<script type="text/javascript">
```

```
var str=new String("I like writing in javascript");
var a=new Array();
a=str.split(' ');
document.write("<strong>The original string is</strong>"+"<br/>");
for(i=0;i<a.length;i++)
document.write(a[i]+" ");
document.write("<br/>");
document.write("<strong>The reversed string is</strong>"+"<br/>");
for(i=a.length-1;i>=0;i--)
document.write(a[i]+" ");
</script>
</body>
</html>
```

Output



2.15 Creation of Object

SPPU : Dec.-18, Marks 5

- The web designer can create an object and can set its properties as per his requirements.
- The object can be created using new expression.
- Initially the object with no properties can be set using following statements.

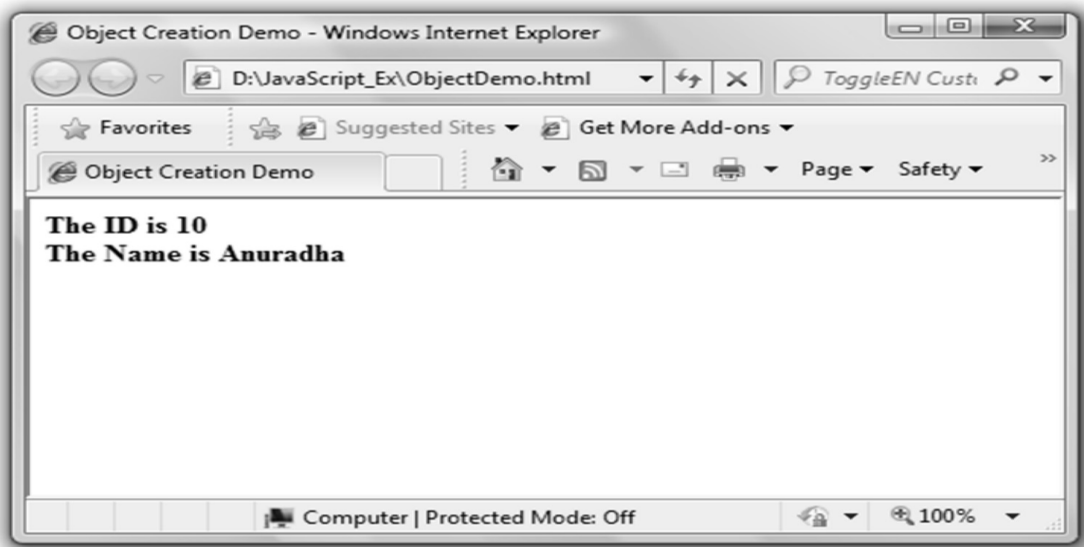
```
Myobj=new Object();
```

- Then using dot operator we can set the properties for that object.
- The object can then be modified by assigning the values to this object.

JavaScript[ObjectDemo.html]

```
<!DOCTYPE html>
<html>
<head>
  <title>Object Creation Demo</title>
</head>
<body>
<strong>
  <script type="text/javascript">
    var Myobj;
    var n,i;
    //creating an object
    Myobj=new Object();
    //setting the propertis for newly created object
    Myobj.id=10;
    Myobj.name="Anuradha";
    document.write("The ID is "+Myobj.id+"<br/>");
    document.write("The Name is "+Myobj.name);
  </script>
</strong>
</body>
</html>
```

Output



Alternative method of creating an object and modifying the properties is as given below -

```
var Myobj={name:"Anuradha",id:10};
```

Then using **document.write** statement we can display the property values as follows -

```
document.write("The ID is "+Myobj.id+"<br/>");  
document.write("The Name is "+Myobj.name);
```

The property of created object can be deleted using an expression **delete**. Normally the property of an object is deleted in order to free the allocated memory so that this memory can be reused by other process.

Review Question

1. How to create an object in JavaScript ? Explain with suitable examples.

SPPU : Dec.-18, Marks 5

2.16 Arrays

SPPU : May-18,19, Marks 5

- **Definition of Arrays :** Arrays is a collection of similar type of elements which can be referred by a common name.
- Any element in an array is referred by an array name followed by “[“followed by position of the element followed by]”.
- The particular position of element in an array is called array index or subscript.

For example -

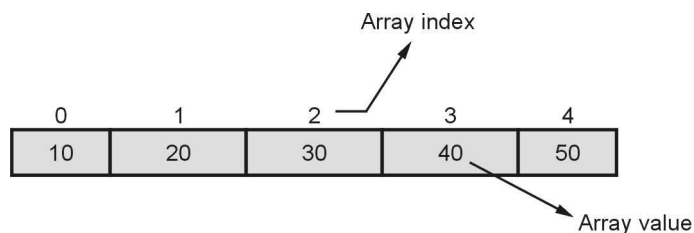


Fig. 2.16.1 Arrays

- Normally the first element in an array is stored at 0th location, however we can start storing the element from any position.

2.16.1 Declaring an Array

- In JavaScript the array can be created using **Array** object.
- Suppose, we want to create an array of 10 elements then we can write,

```
var ar = new Array(10);
```


- Using new operator we can allocate the memory dynamically for the arrays.
- In the brackets the size of an array is mentioned and the var ar denotes the name of the array. Thus by the above sentence an array ar will be created in which we can store 10 elements at the most. Sometimes the above statement can be written like this

```
var ar;  
ar=new Array(10);
```

2.16.2 Initializing an Array

- **Initialization** is the process of assigning a value when either a variable or array is declared. For example

```
int count=10//variable initialization
```

- While initializing an array -
 - 1) Declare name of the array
 - 2) Make use of the keyword **new**
 - 3) Each value within an array as an 'array element' must be separated by comma.
- For example -

var mylist = new Array (10,20,30,40,50)

Name of the array New operator Array elements separated by comma

Fig. 2.16.2

2.16.3 Defining an Array Elements

- The elements in the array are stored from index 0.
- For example - elements in array 10, 20, 30, 40, 50 are stored as follows -

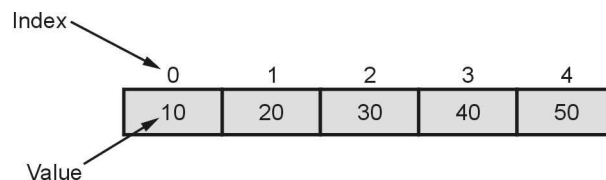


Fig. 2.16.3

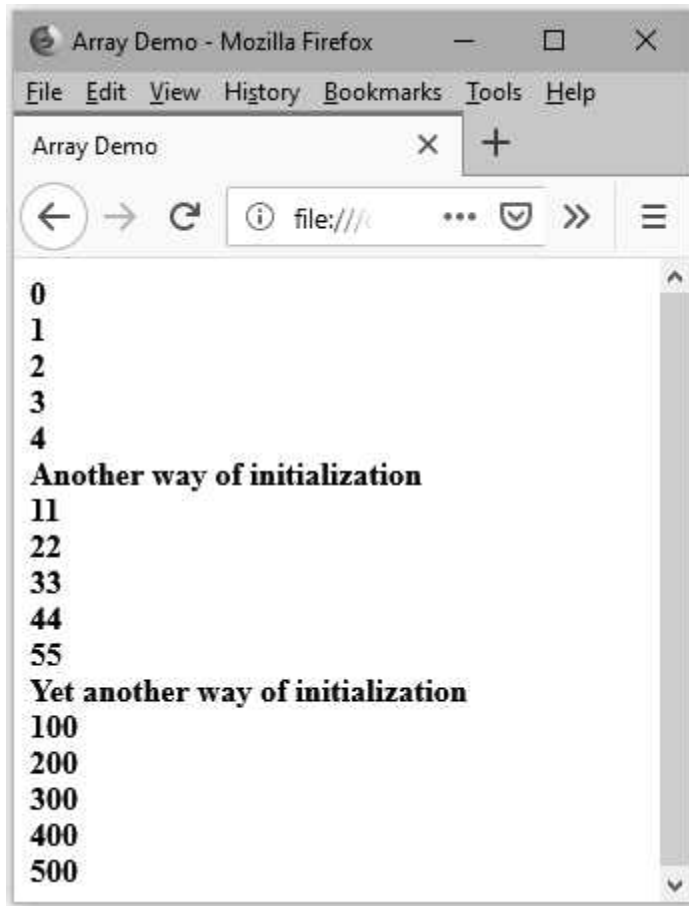
- Following is a simple JavaScript that illustrates the initialization of array by defining the array elements.

JavaScript Document

```
<!DOCTYPE html>
<html>
<head>
  <title>Array Demo</title>
</head>
<body>
<strong>
  <script type="text/javascript">
    a=new Array(5);//creation of array
    for(i=0;i<5;i++)
    {
      a[i]=i;
      document.write(a[i]+"<br>");//displaying array
    }
    document.write("Another way of initialization"+"<br>");
    b=new Array(11,22,33,44,55);//creation of array
    for(i=0;i<5;i++)
    {
      document.write(b[i]+"<br>");//displaying array
    }
    document.write("Yet another way of initialization"+"<br>");
    var c=[100,200,300,400,500];//creation of array
    for(i=0;i<5;i++)
    {
      document.write(c[i]+"<br>");//displaying array
    }
  </script>
</strong>
</body>
</html>
```

Script Explanation : In above JavaScript, as you can notice that, an array can be initialized in three different ways which is shown by boldface. Hence an output of above script will be

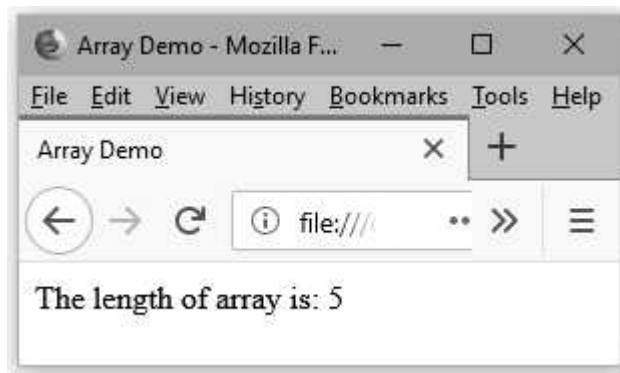
Output



Example 2.16.1 Write a JavaScript to define the array elements and to find the length of array.

Solution :

```
<!DOCTYPE html>
<html>
<head>
  <title>Array Demo</title>
</head>
<body>
  <script type="text/javascript">
    a=new Array(11,22,33,44,55);//creation of array
    var len=a.length
    document.write("The length of array is: "+len)
  </script>
</body>
</html>
```

Output

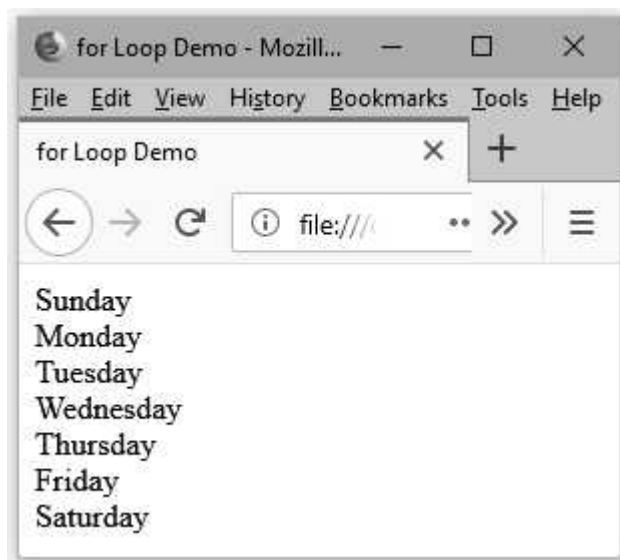
Script Explanation : In above JavaScript, We have used **length** property to calculate length of the array. This actually gives total number of elements in the array.

2.16.4 Looping an Array

- Looping an array means visiting each element present in the array.
- Following JavaScript illustrates how to loop or iterate through the elements of array -

JavaScript Document

```
<!DOCTYPE html>
<html>
<head>
  <title>for Loop Demo</title>
</head>
<body>
  <script type="text/javascript">
    Days=new Array();
    Days[0]="Sunday";
    Days[1]="Monday";
    Days[2]="Tuesday";
    Days[3]="Wednesday";
    Days[4]="Thursday";
    Days[5]="Friday";
    Days[6]="Saturday";
    for(i=0;i<Days.length;i++)
    {
      document.write(Days[i]+"<br>");
    }
  </script>
</body>
</html>
```

Output**2.16.5 Adding an Array Element**

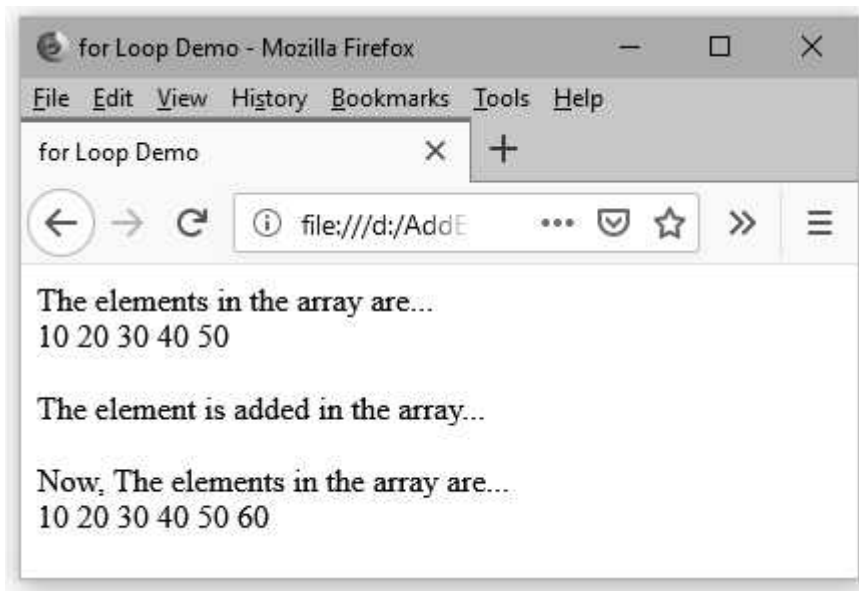
- We can add the element in the array at the end. This increases the size of the array.
- Following example illustrates this idea -

JavaScript Document

```
<!DOCTYPE html>
<html>
<head>
  <title>for Loop Demo</title>
</head>
<body>
  <script type="text/javascript">
    a=new Array();
    a[0]=10;
    a[1]=20;
    a[2]=30
    a[3]=40
    a[4]=50;
    document.write("The elements in the array are...<br/>");
    for(i=0;i<a.length;i++)
    {
      document.write(a[i]+" ");
    }
  </script>
</body>
</html>
```

```
document.write("<br/><br/>The element is added in the array...<br/>");  
a[a.length]=60;  
document.write("<br/>Now, The elements in the array are...<br/>");  
for(i=0;i<a.length;i++)  
{  
    document.write(a[i]+" ");  
}  
</script>  
</body>  
</html>
```

Output



2.16.6 Sorting an Array Element

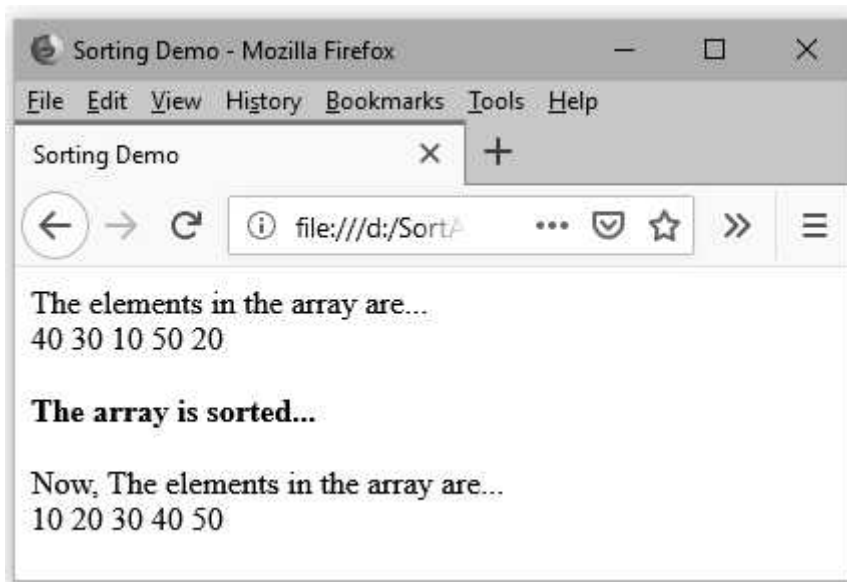
- The elements can be sorted using the built in function **sort**.
- Sorting is basically a process of arranging the elements in ascending order or increasing order.
- For example -

JavaScript Document

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Sorting Demo</title>  
</head>
```

```
<body>
<script type="text/javascript">
  a=new Array();
  a[0]=40;
  a[1]=30;
  a[2]=10
  a[3]=50
  a[4]=20;
  document.write("The elements in the array are...<br/>");
  for(i=0;i<a.length;i++)
  {
    document.write(a[i]+" ");
  }
  document.write("<br/><br/><b>The array is sorted...</b><br/>");
  a.sort();
  document.write("<br/>Now, The elements in the array are...<br/>");
  for(i=0;i<a.length;i++)
  {
    document.write(a[i]+" ");
  }
</script>
</body>
</html>
```

Output



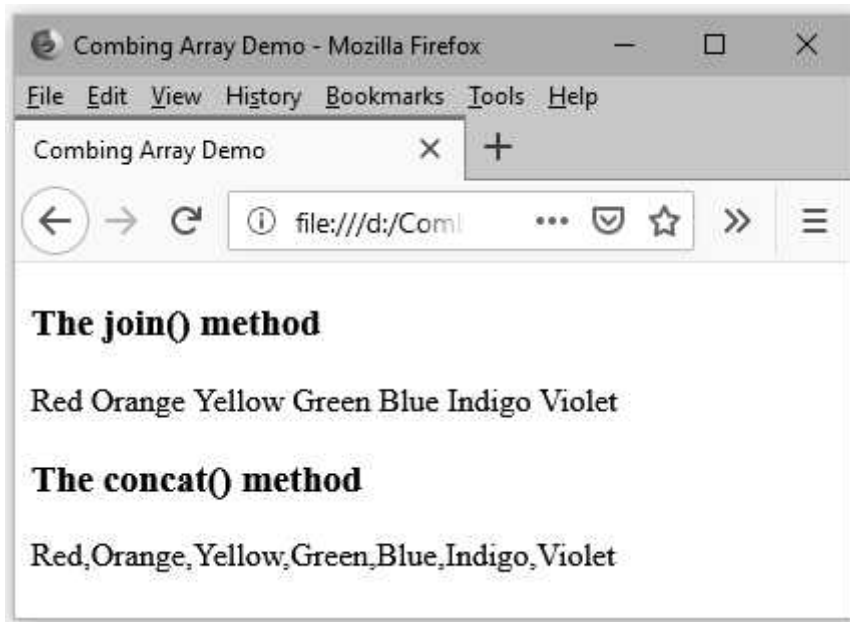
2.16.7 Combining Array Elements into String

- In JavaScript it is possible to combine the array elements into a string.
- We need to use two functions for combining array elements into string and those are -
 - 1) Join()
 - 2) concat()
- There is **difference** between join() and concat() function. The **concat()** method separates each value with a comma. The **join()** method also uses a comma to separate values, but you can specify a character other than a comma to separate values.

JavaScript Document

```
<!DOCTYPE html>
<html>
<head>
  <title>Combing Array Demo</title>
</head>
<body>
  <script type="text/javascript">
    a=new Array();
    a[0]="Red";
    a[1]="Orange";
    a[2]="Yellow"
    a[3]="Green"
    a[4]="Blue";
    a[5]="Indigo";
    a[6]="Violet";
    document.write("<h3> The join() method</h3>");
    var str1=a.join(' ');
    document.write(str1);
    document.write("<h3> The concat() method</h3>");
    var str2=a.concat();
    document.write(str2);

  </script>
</body>
</html>
```


Output**2.16.8 Changing Elements of an Array**

There are various methods that help in changing the elements of array.

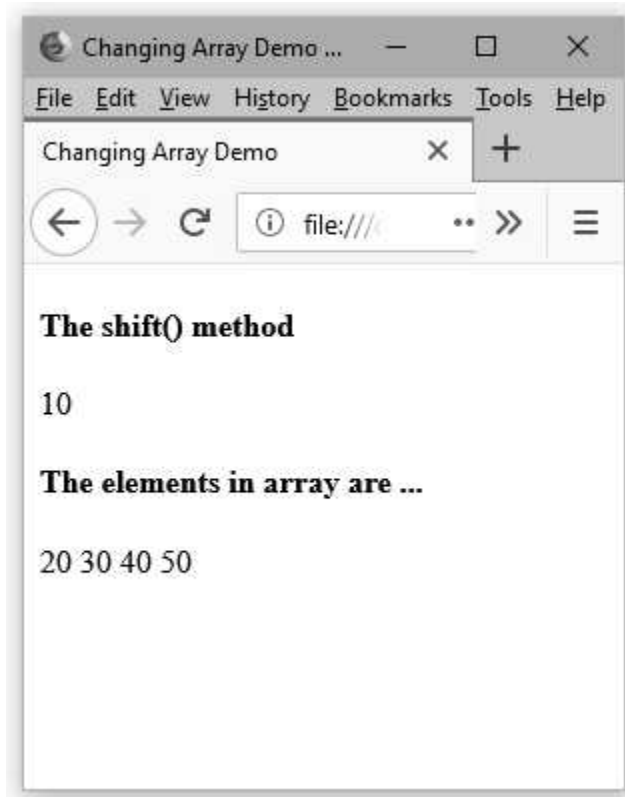
1) Shift Method

This method removes the first element of the array. For example -

```
<!DOCTYPE html>
<html>
<head>
  <title>Changing Array Demo</title>
</head>
<body>
  <script type="text/javascript">
    a=new Array();
    a[0]=10
    a[1]=20
    a[2]=30
    a[3]=40
    a[4]=50
    document.write("<h4> The shift() method</h4>");
    var num=a.shift();
    document.write(num);
    document.write("<h4>The elements in array are ...</h4>");
    for(i=0;i<a.length;i++)
```

```
document.write(a[i] + " ");  
</script>  
</body>  
</html>
```

Output



2) Push Method

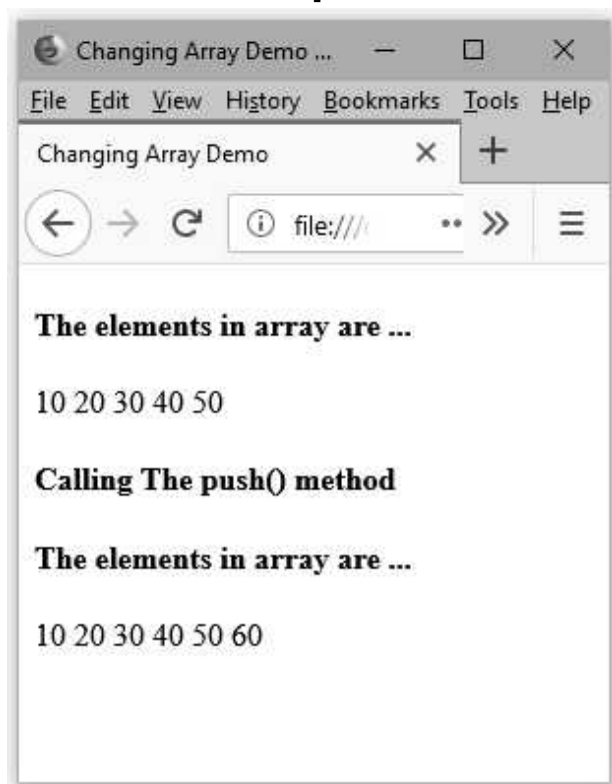
The push method is used to create a new element at the end of the array.

For example -

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Changing Array Demo</title>  
</head>  
<body>  
  <script type="text/javascript">  
    a=new Array();  
    a[0]=10  
    a[1]=20  
    a[2]=30
```

```
a[3]=40
a[4]=50
document.write("<h4>The elements in array are ...</h4>");
for(i=0;i<a.length;i++)
document.write(a[i]+" ");
document.write("<h4> Calling The push() method</h4>");
a.push(60);
document.write("<h4>The elements in array are ...</h4>");
for(i=0;i<a.length;i++)
document.write(a[i]+" ");
</script>
</body>
</html>
```

Output



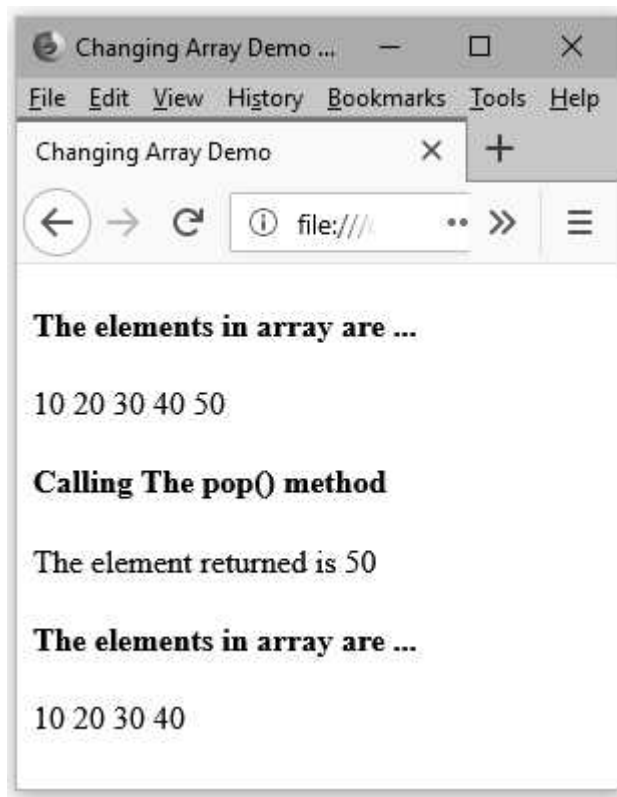
3) Pop Method

This method returns and removes the last element of the array. For example -

```
<!DOCTYPE html>
<html>
<head>
  <title>Changing Array Demo</title>
</head>
```

```
<body>
  <script type="text/javascript">
    a=new Array();
    a[0]=10
    a[1]=20
    a[2]=30
    a[3]=40
    a[4]=50
    document.write("<h4>The elements in array are ...</h4>");
    for(i=0;i<a.length;i++)
      document.write(a[i]+" ");
    document.write("<h4> Calling The pop() method</h4>");
    var val=a.pop();
    document.write("The element returned is "+val)
    document.write("<h4>The elements in array are ...</h4>");
    for(i=0;i<a.length;i++)
      document.write(a[i]+" ");
  </script>
</body>
</html>
```

Output

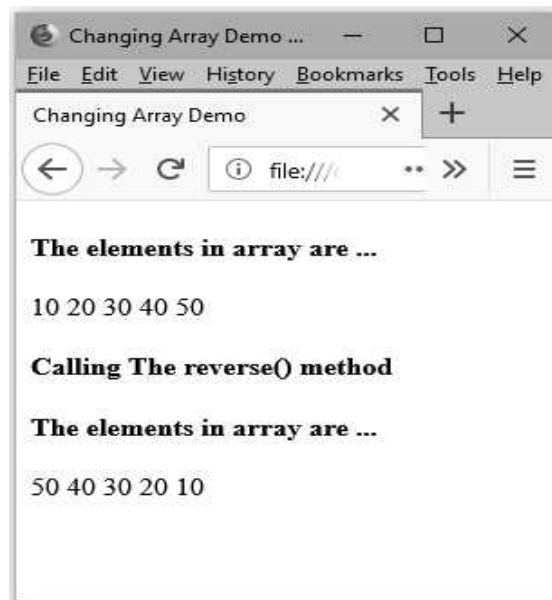


4) Reverse Method

The reverse method is used to reverse the elements present in the array. For example

```
<!DOCTYPE html>
<html>
<head>
  <title>Changing Array Demo</title>
</head>
<body>
  <script type="text/javascript">
    a=new Array();
    a[0]=10
    a[1]=20
    a[2]=30
    a[3]=40
    a[4]=50
    document.write("<h4>The elements in array are ...</h4>");
    for(i=0;i<a.length;i++)
      document.write(a[i]+" ");
    document.write("<h4> Calling The reverse() method</h4>");
    a.reverse();
    document.write("<h4>The elements in array are ...</h4>");
    for(i=0;i<a.length;i++)
      document.write(a[i]+" ");
  </script>
</body>
</html>
```

Output



Review Question

1. Explain how to create and read array elements in JavaScript.

SPPU : May-18,19, Marks 5**2.17 Strings**

- String is a collection of characters.
- Some commonly used methods of string object are concatenating two strings, converting the string to upper case or lower case, finding the substring of a given string and so on.
- The string is written within the quotes- either single quote or double quote. For example -

```
var myname='ABC'
```

2.17.1 Manipulating a String

Manipulating a string means, changing one string to another, joining two strings, changing the string from upper case to lower case or from lower case to upper.

There are some commonly used methods of string

Method	Meaning
concat(str)	This method concatenates the two strings. For example s1.concat(s2) will result in concatenation of string s1 with s2.
charAt(index_val)	This method will return the character specified by value index_val.
substring(begin,end)	This method returns the substring specified by begin and end character.
toLowerCase()	This function is used to convert all the uppercase letters to lower case.
toUpperCase()	This function is used to convert all the lowercase letters to upper case.
valueOf()	This method returns the value of the string.

There is one important property of string object and that is **length**. For example

```
var my_str="Hello";  
var len;  
len=my_str.length;
```

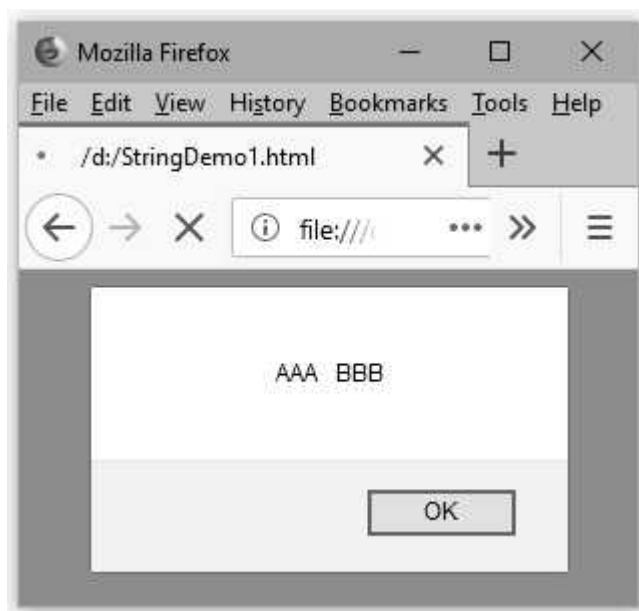
Length of the string "Hello" will be stored in the variable len

2.17.2 Joining a String

We can join two strings using + operator. For example -

```
<!DOCTYPE html>
<html>
  <script type="text/javascript">
    var Firstname="AAA"
    var Lastname=" BBB"
    var name=Firstname+Lastname
    alert(name);
  </script>
</body>
</html>
```

Output



2.17.3 Receiving a Character from Given Position

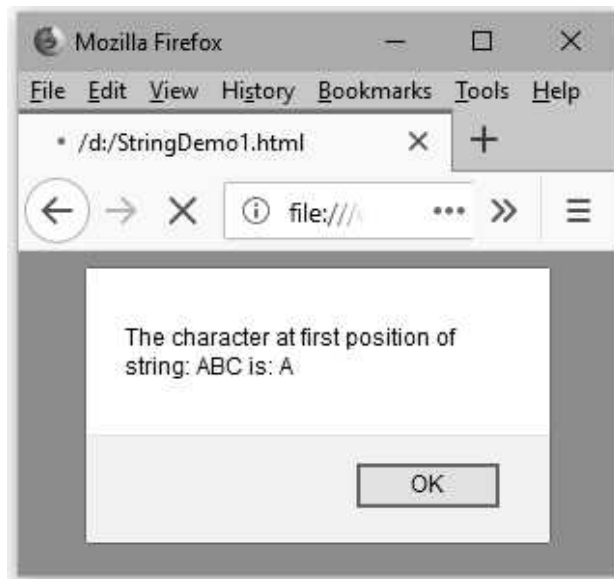
The **charAt()** is a method that returns the character from the specified index. The index of a first character is 0; the second character is 1 and so on. Javascript characters in a string are indexed from left to right.

If no index is provided to **charAt()** method, then the default is 0. Following program illustrates how to receive character from given position.

JavaScript Document

```
<!DOCTYPE html>
<html>
  <script type="text/javascript">
    var str="ABC"
    alert("The character at first position of string: "+str+" is: "+str.charAt(0))
  </script>
</body>
</html>
```

Output



2.17.4 Retrieving a Position of Character in a String

- For finding the position of particular character we use indexOf function.
- It returns the position of the first occurrence of a specified value in a string.
- This method is case sensitive.
- **Example**

```
<!DOCTYPE html>
<html>
  <script type="text/javascript">
    var str="I love India - my country"
    var n=str.indexOf("India");
    alert("The word 'India' is from index: "+n+" in the text: "+str)
  </script>
</body>
</html>
```


Output**2.17.5 Dividing Text**

- Any text is made up of words. We can divide the text into a collection of these words. For example - User may enter his/her complete name as first name, middle name and surname. By dividing the text, we can get firstname, middle name and surname separated.
- Usually **split()** function is used to divide the text.
- The **split()** method creates a new array and then copies portions of the string, called a substring, into its array element.
- **Syntax :** The split method is used in the following manner -

```
var newStrArray=textname.split(delimiter character)
```

JavaScript Document

```
<!DOCTYPE html>
<html>
  <script type="text/javascript">
    var Name="Shivaji Shahaji Bhosale"
    var list=Name.split(' ')
    alert("First Name: "+list[0]+"\\nMiddle Name: "+list[1]+"\\nLast Name: "+list[2])
  </script>
</body>
</html>
```

Output**2.17.6 Copying a Substring**

There are two useful methods for copying a substring - i) `substring()` and ii) `substr()`

1. Use of substring

The syntax of `substring` is

```
substring(start, end)
```

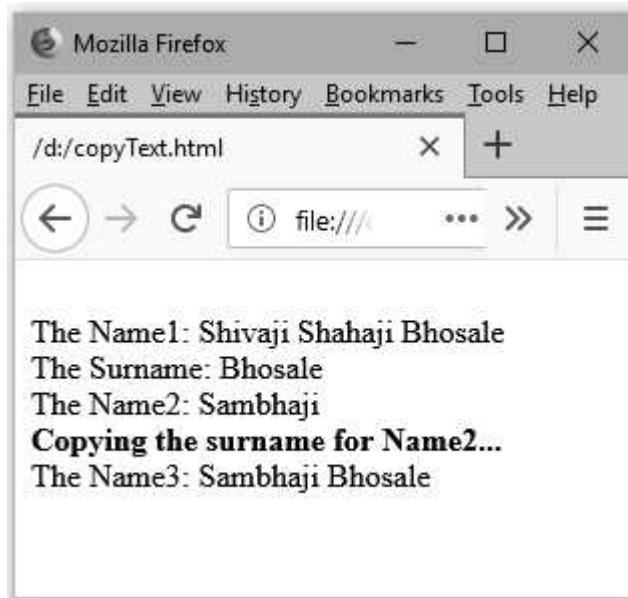
where `start` indicates the starting index and `end` indicates the ending index for extracting substring.

Following program shows use of `substring()` function for retrieving the substring and then copying it to some other string at the end.

```
<!DOCTYPE html>
<html>
  <script type="text/javascript">
    var Name1="Shivaji Shahaji Bhosale"
    var Name2="Sambhaji"
    var sub_str=Name1.substring(16,23)
    document.write("<br/>The Name1: "+Name1)
    document.write("<br/>The Surname: "+sub_str)
    document.write("<br/>The Name2: "+Name2)
    document.write("<br/><b>Copying the surname for Name2...</b>")
    var Name3=Name2+" "+sub_str
```

```
document.write("<br/>The Name3: "+Name3)
</script>
</body>
</html>
```

Output



2. Use of substr

The **substr** method is also used to extract the substring from the text. The syntax for substr is as follows

```
substr(start,length)
```

where start indicates the starting index and length indicates the number of characters to be extracted.

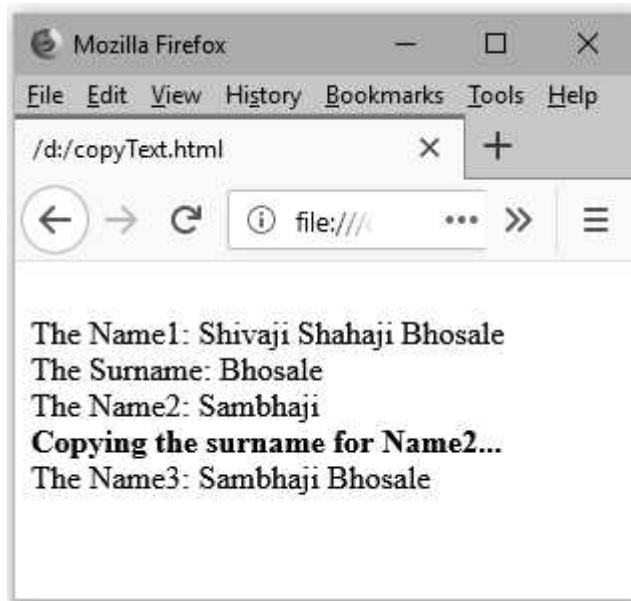
Following example shows how to extract some substring using **substr()** method and then copy this substring to another text.

JavaScript Document

```
<!DOCTYPE html>
<html>
  <script type="text/javascript">
    var Name1="Shivaji Shahaji Bhosale"
    var Name2="Sambhaji"
    var sub_str=Name1.substr(16,7)
    document.write("<br/>The Name1: "+Name1)
    document.write("<br/>The Surname: "+sub_str)
```

```
document.write("<br/>The Name2: "+Name2)
document.write("<br/><b>Copying the surname for Name2...</b>")
var Name3=Name2+" "+sub_str
document.write("<br/>The Name3: "+Name3)
</script>
</body>
</html>
```

Output



2.17.7 Converting String to Number and Number to String

1. Converting String to Number

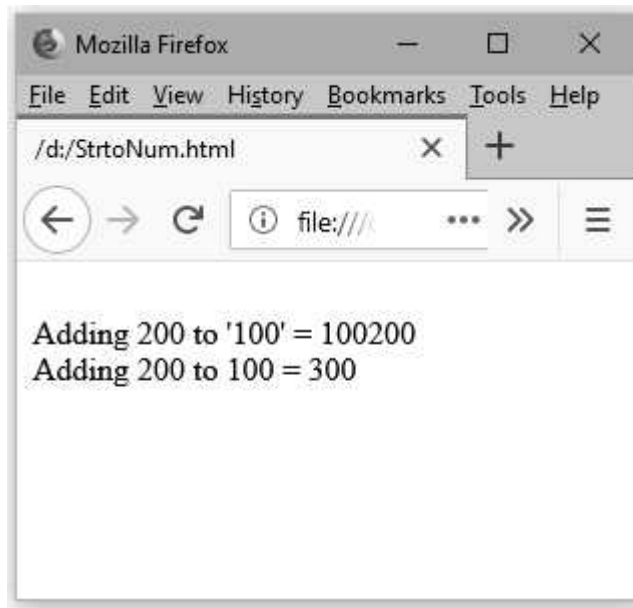
For converting string to number we have different types of functions based on type of numbers - That means if string is to be converted to integer(a number without decimal point) then the method **parseInt()** is used. If string is to be converted to float value then the method **parseFloat()** is used.

Example

```
<!DOCTYPE html>
<html>
  <script type="text/javascript">
    var str="100";
    var a=parseInt(str)//converting string 100 to number 100
    var b=200;
    result=str+b
```

```
document.write("<br/>Adding 200 to '100' = "+result)
result=a+b
document.write("<br/>Adding 200 to 100 = "+result)
</script>
</body>
</html>
```

Output



2. Converting Number to String

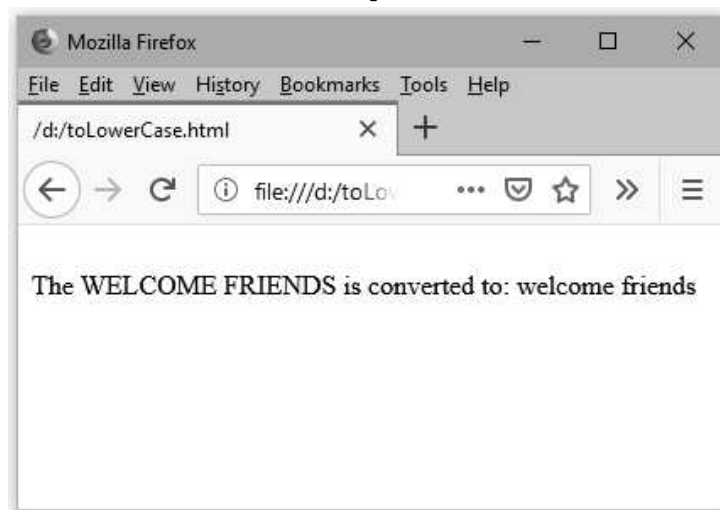
Using **toString()** method we can convert a number to string. For example -

```
<!DOCTYPE html>
<html>
  <script type="text/javascript">
    var num=100;
    var str=num.toString()
    document.write("<br/>The 100 value is converted to string "+str)
  </script>
</body>
</html>
```

Output**2.17.8 Changing Case of String**

Using `toLowerCase()`, we can convert the capital letters in the string to small letter. For example

```
<!DOCTYPE html>
<html>
  <script type="text/javascript">
    var str="WELCOME FRIENDS";
    var newstr=str.toLowerCase()
    document.write("<br/>The "+str+" is converted to: "+newstr)
  </script>
</body>
</html>
```

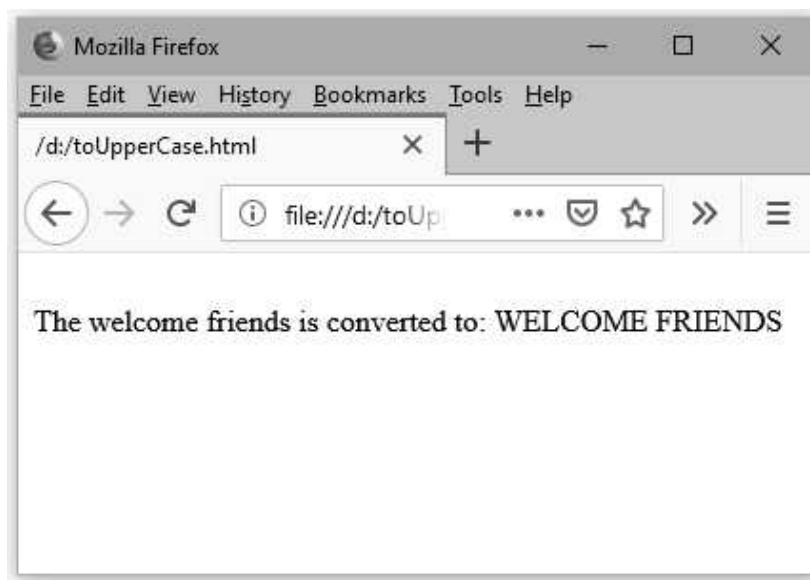
Output

Using **toUpperCase()** method we can convert the small letters to capital letters.

For example -

```
<!DOCTYPE html>
<html>
  <script type="text/javascript">
    var str="welcome friends";
    var newstr=str.toUpperCase()
    document.write("<br/>The "+str+" is converted to: "+newstr)
  </script>
</body>
</html>
```

Output



2.17.9 Finding Unicode of a Character

- Computer can not understand characters. It understands only numbers. Hence whenever user types some letter, it gets converted automatically to a number called **Unicode number**. The computer can understand this Unicode number only.
- Unicode is a standard that assigns a number to every character, number, and symbol that can be displayed on a computer screen.
- The **charCodeAt()** is a methods that returns Unicode of a string. For example -

```
<!DOCTYPE html>
<html>
  <script type="text/javascript">
```

```
var ch='a'  
var n=ch.charCodeAt()  
document.write("<br/>The letter '"+ch+"' has a unicode as: "+n)  
</script>  
</body>  
</html>
```

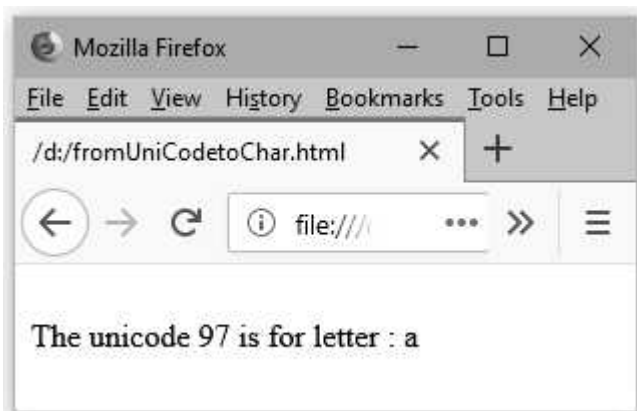
Output



- Similarly, we can obtain the letter or character from a Unicode using the method **fromCharCode()**. For example

```
<!DOCTYPE html>  
<html>  
  <script type="text/javascript">  
    var n=97  
    var ch=String.fromCharCode(n)  
    document.write("<br/>The unicode '"+n+"' is for letter : '"+ch)  
  </script>  
</body>  
</html>
```

Output



2.18 JavaScript Debuggers

- When any syntactical error occurs in the web document then we get nothing on the output. That is neither output nor any diagnostic message is getting displayed on the browser window. This becomes annoying for the developer to design the desired web page in proper direction. We can receive the error messages by making some settings in browser window. Apart from there are some debugging techniques using which the errors in the script can be identified and then handled.
- JavaScript is a scripting language which is very much similar to a programming language. It supports the arrays, functions and control statements. Hence debugging the javascript is just similar to debugging any programming language.
- The standard traditional method of debugging the JavaScript is by use of **alert()** to display the values of corresponding variables. But it is a complex method of debugging as within the loop if we write the alert statement then each time the popup window will appear to display the corresponding messages.
- However we can make use of some other API methods for debugging the javascript.
- Another way is to use the **log()** function **console API**. Following is a simple script that illustrates how to debug the values of the variable using the **log()** function of **console**.

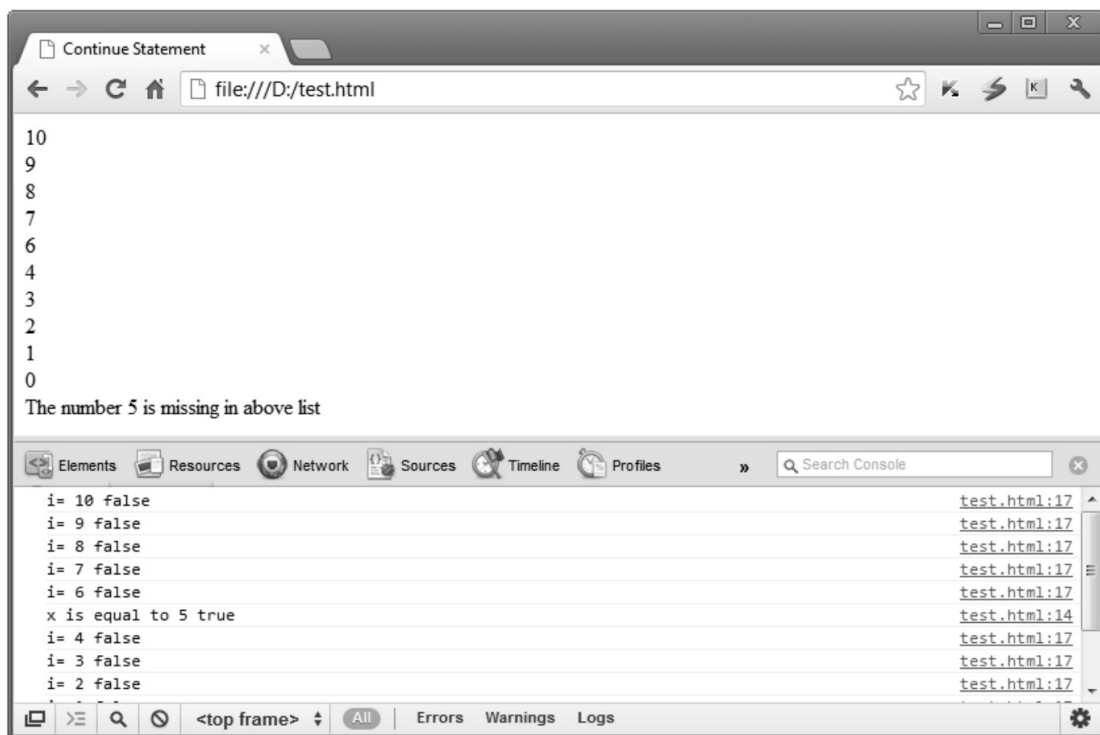
```
<!DOCTYPE html>
<html>
<head>
<title>Continue Statement</title>
</head>
<body>
<script type="text/javascript">
for(i=10;i>=0;i--)
{
  if(i==5)
  {
    x=i;
    console.log('x is equal to 5', x == 5);
    continue;
  }
  console.log('i= ' + i, i == 5);
  document.write(i);
  document.write("<br>");
}
}
```

Here `x==5` is true hence it will return **true** along with the message.

```
document.write("The number "+x+" is missing in above list" );  
</script>  
</body>  
</html>
```

Just open the above code in Google Chrome. Press F12, click on the Console to get the debugging *done using the log function. Here is the snapshot for the same.*

Output



Part - II : DOM

2.19 Introduction to Document Object Model (DOM)

SPPU : Dec.-19, Marks 5

- The Document Object Modeling (DOM) is for defining the standard for accessing and manipulating HTML, XML and other scripting languages.
- It is the **W3C recommendation** for handling the structured documents.
- Normally the structured information is provided in XML, HTML and many other documents.
- Hence DOM provides the standard set of programming interfaces for working with XML and XHTML and JavaScript.

What is DOM ?

Document Object Model (DOM) is a set of platform independent and language neutral application programming interface (API) which describes how to access and manipulate the information stored in XML, HTML and JavaScript documents.

Review Question

1. Explain Document Object Model (DOM) with suitable example.

SPPU : Dec.-19, Marks 5

2.20 DOM History and Levels

- Various levels of DOM are enlisted in the following Table -

Level	Description
DOM 0	This model is supported by the early browsers. This level could support JavaScript. This version was implemented in Netscape 3.0 and Internet Explorer 3.0 browsers.
DOM 1	This version was issued in 1998 which was focused on XHTML and XML.
DOM 2	This version was issued in 2000 that could specify the style sheet. It also supports the event model and traversal within the documents.
DOM 3	This is the current release of DOM specification published in 2004. This version could deal with XML with DTD and schema, document validations, document views and formatting.

2.21 The Document Tree

SPPU : April-18, May-19, Marks 5

- Basically DOM is an **Application Programming Interface (API)** that defines the interface between HTML document and application program. That means, suppose application program is written in Java and this Java program wants to access the elements of HTML web document then it is possible by using a set of application programming interfaces (API) which belongs to the DOM.
- The DOM contains the **set of interfaces** for the document tree node type. These interfaces are similar to the Java or C++ interfaces. They have **objects, properties and methods** which are useful for respected node type of the web document.
- The documents in DOM are represented using a tree like structure in which every element is represented as a node. Hence the tree structure is also referred as DOM tree.

- **For example :** Consider following XHTML document.

```
<html>
  <head>
    <title>This is My Web Page </title>
  </head>
  <body>
    <h1>Hello Friends </h1>
    <h2>How are you?</h2>
    </h3>See you</h3>
  </body>
</html>
```

- The DOM tree will be

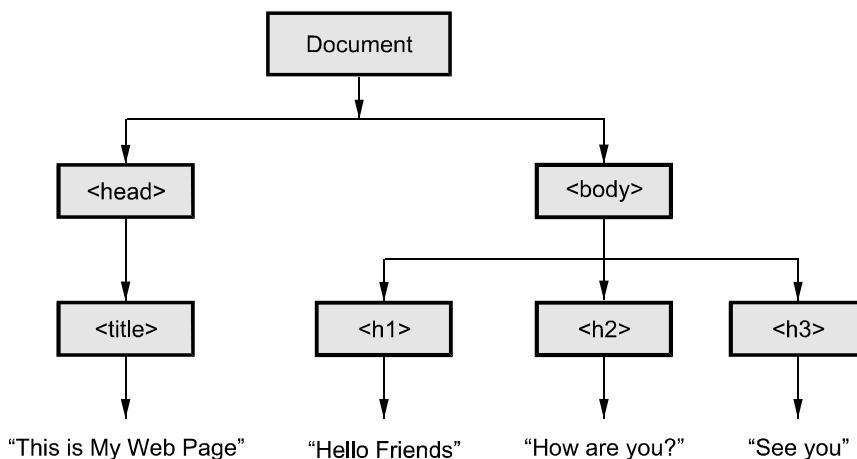


Fig. 2.21.1 DOM structure for simple web document

- We can describe some basic terminologies used in DOM tree as follows -
 1. Every element in the DOM tree is called **node**.
 2. The topmost single node in the DOM tree is called the **root**.
 3. Every child node must have a parent **node**.
 4. The bottommost nodes that have no children are called **leaf nodes**.
 5. The nodes that have the common parent are called **siblings**.

Review Question

1. What is the purpose of DOM node tree ? Draw DOM node tree for simple HTML page.

SPPU : April-18, May-19, Marks 5

2.22 Modifying Element Style

SPPU : May-18, Dec.-19, March-20, Marks 5

We can access or change the contents of document by using various methods. Some commonly used properties and methods of DOM are as follows :

Methods

Method	Meaning
getElementById	This method is used to obtain the specific element which is specified by some id within the script.
createElement	This method is used to create an element node.
createTextNode	Useful for creating a text node.
createAttribute	Useful for creating attribute.
appendChild	For adding a new child to specified node, this method is used.
removeChild	For removing a child node of a specific node, this method is used.
getAttribute	This method is useful for returning the specified attribute value.
setAttribute	This method is useful for setting or changing the specified attribute to the specified value.

Properties

Property	Meaning
attributes	This property is used to get the attribute nodes of the node.
parentNode	This DOM property is useful for obtaining the parent node of the specific node.
childNodes	This DOM property is useful for obtaining the child nodes of the specific node.
innerHTML	It is useful for getting the text value of a node.

2.22.1 Accessing Elements using DOM

- There are several ways by which we can access the elements of the web document.
- To understand these methods of accessing we will consider one simple web document as follows.

```
<html >
<head>
  <title>This is My Web Page </title>
</head>
<body>
<form name="form1">
```

```
<input type="text" name="myinput"/>
</form>
</body>
</html>
```

Method 1

- Every XHTML document element is associated with some address. This address is called **DOM address**.
- The document has the collection of **forms** and **elements**. Hence we can refer the text box element as

```
var Dom_Obj=document.forms[0].elements[0];
```

- But this is not the suitable method of addressing the elements. Because if we change the above script as

```
...
<form name="form1">
    <input type="button" name="mybutton"/>
    <input type="text" name="myinput"/>
</form>
...
```

then index reference gets changed. Hence another approach of accessing the elements is developed.

Method 2

- We can access the desired element from the web document using JavaScript method **getElementById**. The element access can be made as follows -

```
var Dom_Obj=document.getElementById("myinput");
```

- But if the element is in particular group, that means if there are certain elements on the form such as radio buttons or check boxes then they normally appear in the groups. Hence to access these elements we make use of its index. Consider the following code sample

```
<form id="Food">
    <input type="checkbox" name="vegetables" value="Spinach" />Spinach
    <input type="checkbox" name="vegetables" value="FenuGreek" />FenuGreek
    <input type="checkbox" name="vegetables" value="Cabbage" />Cabbage
</form>
```

- For getting the values of these checkboxes we can write following code.

```
var Dom_obj=document.getElementById("Food");
for(i = 0 ; i < Dom_Obj.vegetables.length ; i++)
    document.write(Dom_Obj.vegetables[i] + "<br/>");
```

2.22.2 Modifying Elements using DOM

Appending an element Using DOM

We can add new node in the HTML document using the DOM method **appendChild** method

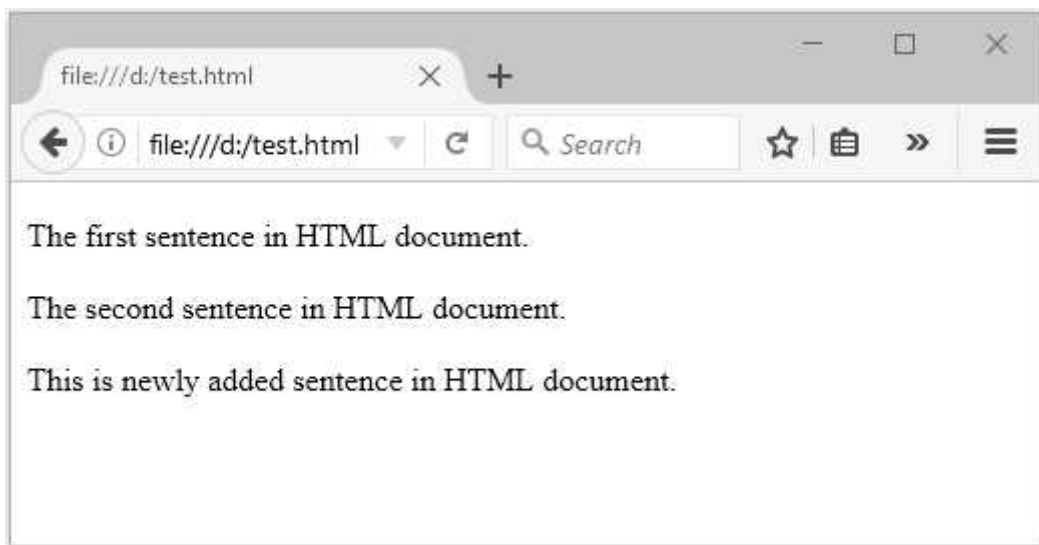
Following example illustrates the idea.

```
<!DOCTYPE html>
<html>
<body>

<div id="div1">
<p id="p1">The first sentence in HTML document.</p>
<p id="p2">The second sentence in HTML document.</p>
</div>

<script>
var pNode = document.createElement("p");
var node = document.createTextNode("This is newly added sentence in HTML document.");
pNode.appendChild(node);
var element = document.getElementById("div1");
element.appendChild(pNode);
</script>
</body>
</html>
```

Output



Inserting an element Using DOM

We can insert a node in between the nodes by using `appendChild` and `insertBefore` method. For example -

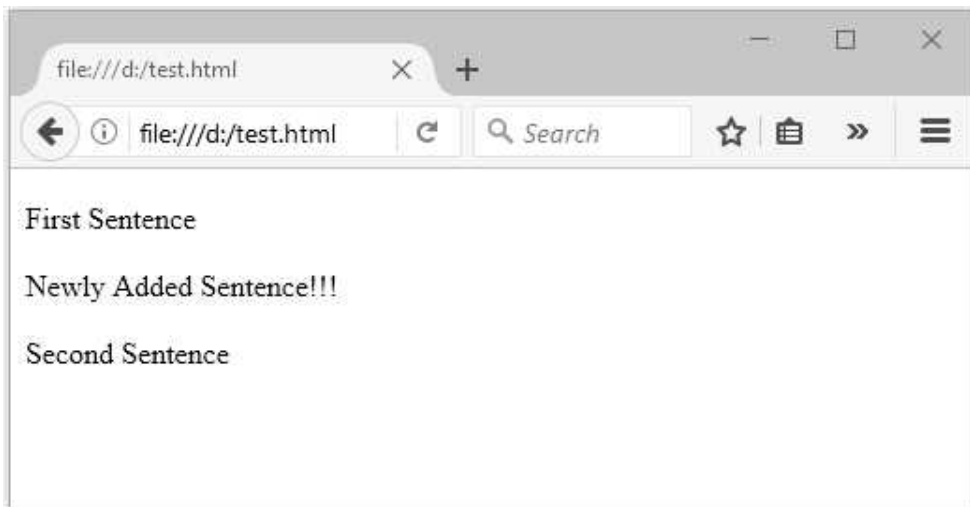
```
<!DOCTYPE html>
<html>
<body>

<div id="div1">
<p id="p1">First Sentence</p>
<p id="p2">Second Sentence</p>
</div>

<script>
var pNode = document.createElement("p");
var node = document.createTextNode("Newly Added Sentence!!!");
pNode.appendChild(node);
var element = document.getElementById("div1");
var nextnode = document.getElementById("p2");
element.insertBefore(pNode,nextnode);
</script>

</body>
</html>
```

Output



Removing an element Using DOM

We can remove or delete particular from HTML document using DOM's `removeChild` method.

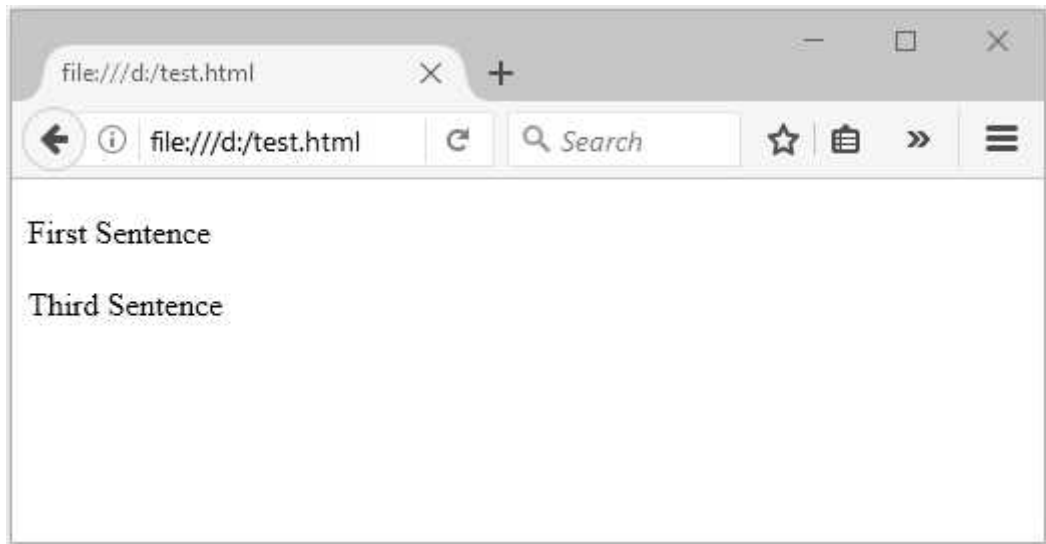
Here is the illustration.

```
<!DOCTYPE html>
<html>
<body>

<div id="div1">
<p id="p1">First Sentence</p>
<p id="p2">Second Sentence</p>
<p id="p3">Third Sentence</p>
</div>

<script>
var parentnode = document.getElementById("div1");
var node = document.getElementById("p2");
parentnode.removeChild(node);
</script>
</body>
</html>
```

Output



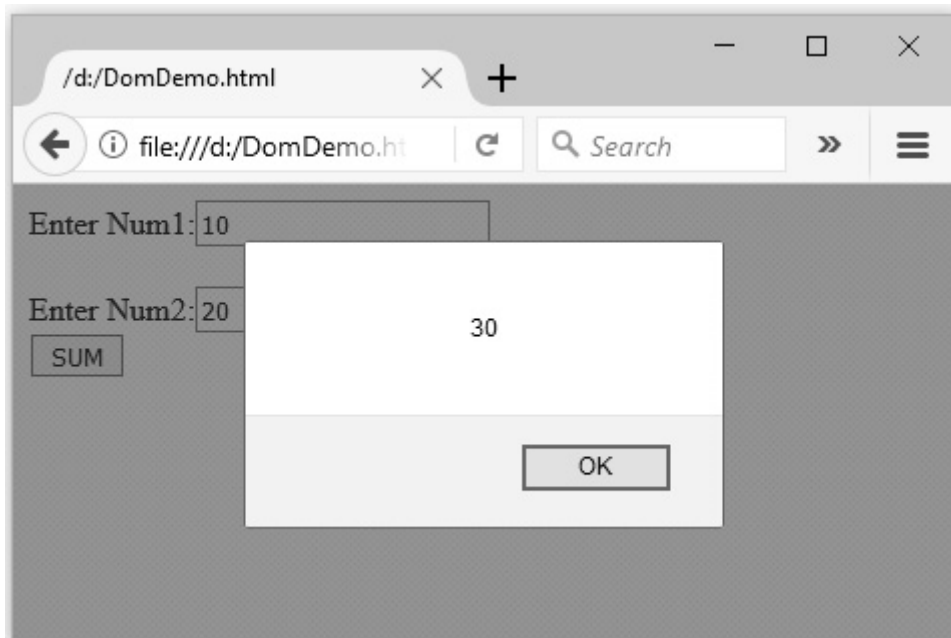
Example 2.22.1 Write a JavaScript to display sum of two elements. Make use of appropriate DOM method.

Solution :

```
<!doctype html>
<html>
```

```
<head>
<script type="text/javascript">
function getSum()
{
    var number1=document.getElementById("number1").value;
    var number2=document.getElementById("number2").value;
    var num1=Number(number1);
var num2=Number(number2);
    alert(num1+num2);
}
</script>
</head>
<body>
<form>
Enter Num1:<input type="text" id="number1" /><br/> <br/>
Enter Num2:<input type="text" id="number2" /><br/>
<input type="button" value="SUM" onclick="getSum()"/>
</form>
</body>
</html>
```

Output

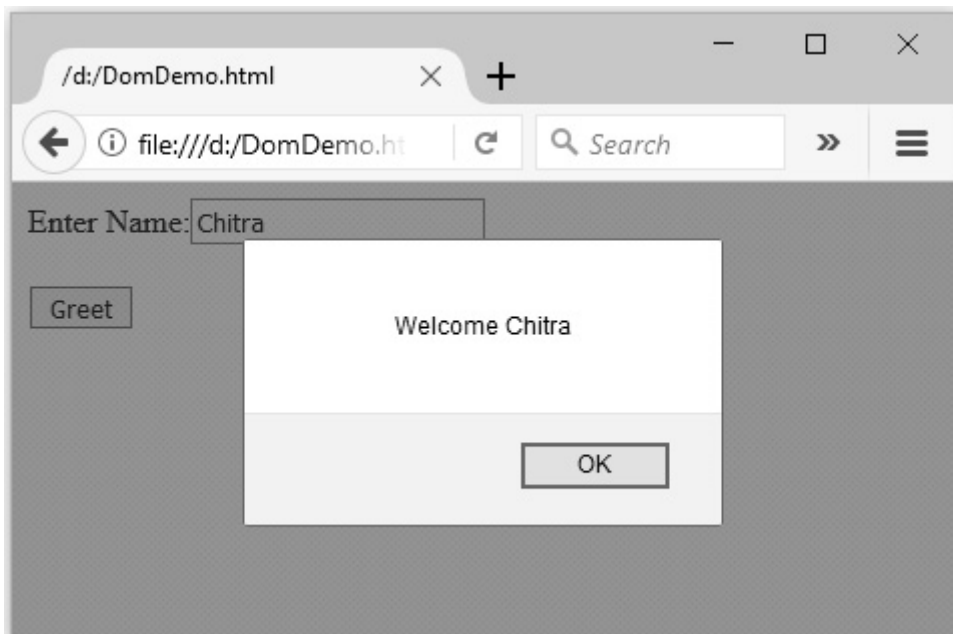


Example 2.22.2 Write a JavaScript to welcome the user by his/her name when he enters his/her name in textbox. Make use of appropriate DOM method.

Solution :

```
<!Doctype html>
<html>
<head>
<script type="text/javascript">
function display()
{
    var name=document.getElementsByName("user");
    alert("Welcome "+name[0].value);
}
</script>
</head>
<body>
<form>
Enter Name:<input type="text" name="user" /><br/> <br/>
<input type="button" value="Greet" onclick="display()"/>
</form>
</body>
</html>
```

Output



Review Questions

1. Write a program of your choice to demonstrate the use of properties of DOM.

SPPU : May-18, Marks 5

2. Explain the following document object properties with proper syntax.

i) `document.getElementById()`ii) `document.getElementsByClassName()`iii) `document.getElementsByName()`.**SPPU : Dec.-19, March-20, Marks 5****2.23 DOM Event Handling****SPPU : April-18, Dec.-18, May-19, March-20, Marks 5**

- **Definition of Event** : Event is an activity that represents a change in the environment. For example mouse clicks, pressing a particular key of keyboard represent the events. Such events are called intrinsic events.
- **Definition of Event Handler** : Event handler is a script that gets executed in response to these events. Thus event handler enables the web document to respond the user activities through the browser window.
- JavaScript support this special type of programming in which events may occur and these events get responded by executing the event handlers. This type of programming is called event-driven programming.
- Events are specified in lowercase letters and these are case-sensitive.
- **Event Registration** : The process of connecting event handler to an event is called event registration. The event handler registration can be done using two methods -
 - Assigning the tag attributes
 - Assigning the handler address to object properties.

Internet Programming

On occurrence of events the tag attribute must be assigned with some user defined functionalities. This will help to execute certain action on occurrence of particular event.

Commonly used events and tag attributes are enlisted in the following table -

Events	Intrinsic event attribute	Meaning	Associated tags
blur	onblur	Losing the focus.	<button> <input> <a> <textarea> <select>

change	onchange	On occurrence of some change.	<input> <textarea> <select>
click	onclick	When user clicks the mouse button.	<a> <input>
dblclick	ondblclick	When user double clicks the mouse button.	<a> <input> <button>
focus	onfocus	When user acquires the input focus.	<a> <input> <select> <textarea>
keyup	onkeyup	When user releases the key from the keyboard.	Form elements such as input,button,text,textarea and so on.
keydown	onkeydown	When user presses the key down	Form elements such as input,button,text,textarea and so on.
keypress	onkeypress	When user presses the key.	Form elements such as input,button,text,textarea and so on.
mousedown	onmousedown	When user clicks the left mouse button.	Form elements such as input,button,text,textarea and so on.
mouseup	onmouseup	When user releases the left mouse button.	Form elements such as input,button,text,textarea and so on.
mousemove	onmousemove	When user moves the mouse.	Form elements such as input,button,text,textarea and so on.
mouseout	onmouseout	When the user moves the mouse away from some element.	Form elements such as input,button,text,textarea and so on.
mouseover	onmouseover	When the user moves the mouse away over some element.	Form elements such as input,button,text,textarea and so on.
load	onload	After getting the document loaded.	<body>

reset	onreset	When the reset button is clicked.	<form>
submit	onsubmit	When the submit button is clicked.	<form>
select	onselect	On selection.	<input> <textarea>
unload	onunload	When user exits the document.	<body>

The use of these tag attributes for handling the events is illustrated by following code sample.

```
<input type = "button" name = "My_button"
onclick = "My_fun()" ; />
```



Tag attribute Event handler

That means when the user clicks the **button** then as an event handler a user defined function **My_fun()** gets called. Basically in this function user writes the instructions that need to be executed on the button click event.

Example 2.23.1 *Discuss the properties of mouse events associated with DOM2 with an example.*

Solution : Various types of mouse events associated with DOM2 are click, mousedown, mouseup, mouseout and mouseover.

Events	Intrinsic event attribute	Meaning	Associated tags
mousedown	onmousedown	When user clicks the left mouse button.	Form elements such as input, button, text, textarea and so on.
mouseup	onmouseup	When user releases the left mouse button.	Form elements such as input, button, text, textarea and so on.
mousemove	onmousemove	When user moves the mouse.	Form elements such as input, button, text, textarea and so on.
mouseout	onmouseout	When the user moves the mouse away from some element.	Form elements such as input, button, text, textarea and so on.
mouseover	onmouseover	When the user moves the mouse away over some element.	Form elements such as input, button, text, textarea and so on.

click	onclick	When user clicks the mouse button.	<a> <input>
dblclick	ondblclick	When user double clicks the mouse button.	<a> <input> <button>

Following is an example in which image can be dragged and dropped anywhere in the HTML document. Accordingly the co-ordinates will be displayed.

```

<html>
<head>
<title> Drag and Drop Demo</title>
<script type = "text/javascript">
  var X_offset, Y_offset, Item;
  function Catch_function(e)

  {
    Item = e.currentTarget;
    var posX = parseInt(Item.style.left);
    var posY = parseInt(Item.style.top);
    X_offset = e.clientX - posX;
    Y_offset = e.clientY - posY;
    document.addEventListener("mousemove", Move_function, true);
    document.addEventListener("mouseup", Drop_function, true);
    e.stopPropagation();
    e.preventDefault();

  }
  function Move_function(e)
  {
    Item.style.left = (e.clientX - X_offset) + "px";
    Item.style.top = (e.clientY - Y_offset) + "px";
    e.stopPropagation();
  }
  function Drop_function(e)
  {
    document.removeEventListener("mouseup", Drop_function, true);
    document.removeEventListener("mousemove", Move_function, true);
    e.stopPropagation();
  }
</script>

```

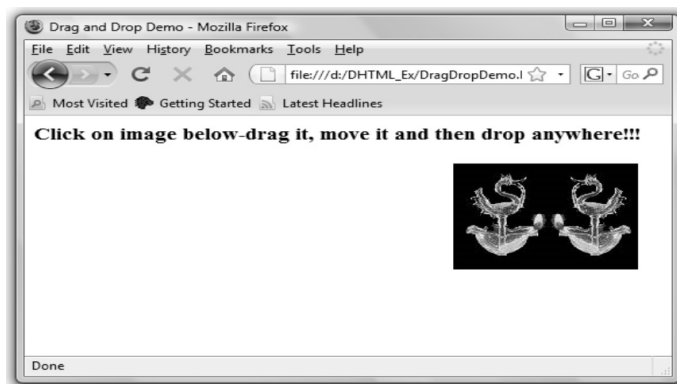
Registering the mouse event with the help of **addEventListener** function.

```

</head>
<body>
  <h3>
    Click on image below-drag it, move it and then drop anywhere!!!
    <br /> <br />
    
  </h3>
</body>
</html>

```

Output



2.23.1 Handling Events from the Body Elements

To understand how events works in JavaScript let us put some Form components on the JavaScript. The **onload** event gets activated as soon as the web page gets loaded in the browser's window. Following script along with its output illustrate the **onload** tag attribute.

JavaScript[OnloadDemo.html]

```

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Demo of onload Tag Attibute</title>
  <script type="text/javascript">
    function my_fun()
    {

```

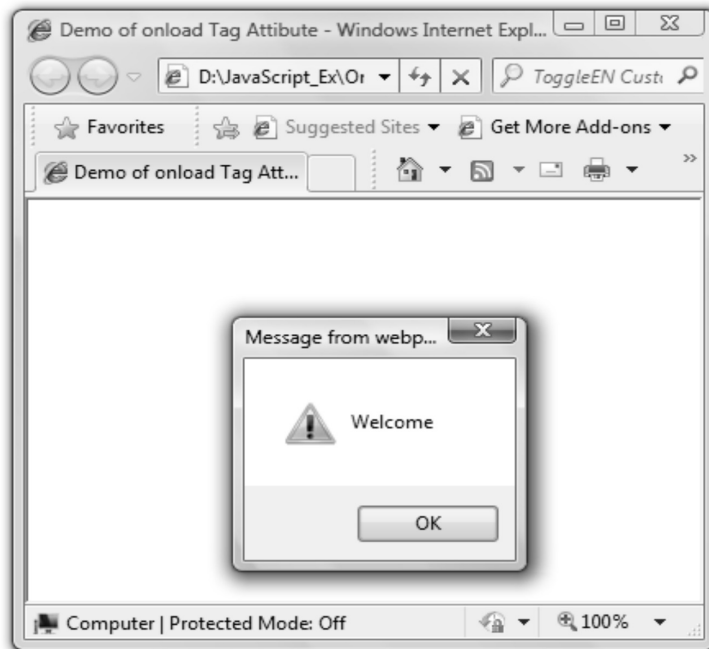


```
//This message will be displayed on page loading
```

```
alert("Welcome");  
}  
</script>  
</head>  
<body onload="my_fun()">  
</body>  
</html>
```

When web document gets loaded on the browser window then **my_fun()** will be called

Output



Example 2.23.2 Write an HTML page and also provide a JavaScript for accepting a user ID and password from the user to ensure that input is not empty.

SPPU : April-18, In Sem, Marks 5

Solution :

```
<!DOCTYPE html>  
<html>  
<head>  
  <script type="text/javascript">  
    function my_fun()  
    {  
      var mypwd=document.getElementById("pwd");
```

```
var myuname=document.getElementById("uname");
if(myuname.value=="")
{
    alert("You have not entered the name");
    myuname.focus();
    return false;
}
if(mypwd.value=="")
{
    alert("You have not entered the password");
    mypwd.focus();
    return false;
}
else
{
    alert("Congratulations!!!");
    return true;
}
}
</script>
</head>
<body>
    <center>
        <form id="form1">
            <label> Enter your name
                <input type="text" value="" id="uname" />
            </label>
            <label> Enter your password
                <input type="password" value="" id="pwd" />
            </label>
            <br/><br/>
            <input type="submit" onsubmit="my_fun();" />
            <input type="reset" value="Reset" name="reset" /> <br/>
        </form>
    </center>
</body>
</html>
```

Example 2.23.3 Explain with example how to access values of textbox of form using JavaScript ?

SPPU : March-20, Marks 5

Solution :

```
<!DOCTYPE html>
<html>
```

```
<head>
<script type = "text/javascript">
function getNum()
{
    var num = document.getElementById("Data").value;
    var a = Number(num);
    alert(a);
}
</script>
</head>
<body>
<form>
    Enter some Number: <input type="text" id="Data"/> <br/> <br/>
    <input type = "button" value="Click Me" onclick='getNum()'/>
</form>
</body>
</html>
```

Review Question

1. Write a program of your choice to demonstrate DOM.

SPPU : Dec.-18, May-19, March-20, Marks 5

2.24 Form Validation in JavaScript

SPPU : Dec.-18, Marks 5

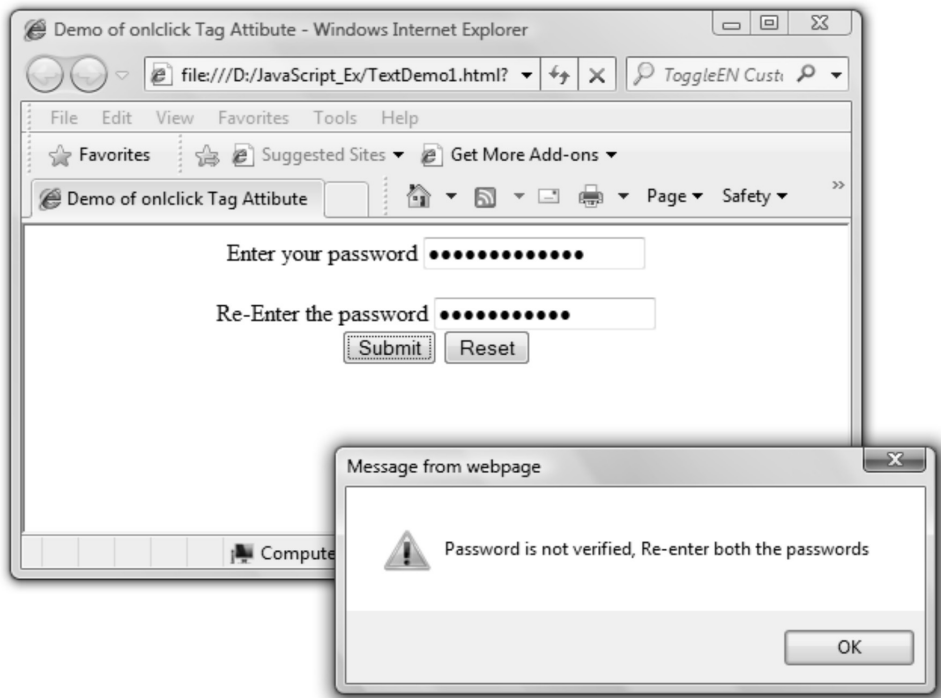
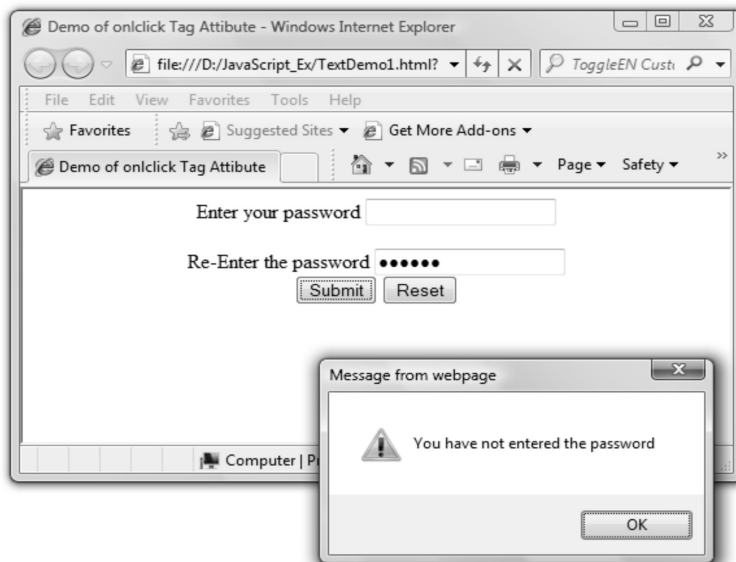
- Various control objects are placed on the form. These control objects are called **widgets**.
- These widgets used in JavaScript are - Textbox, Push button, Radio button, Checkbox and so on.
- In JavaScript the **validation of these widgets** is an important task.
- Let us understand the validation of form elements with the help of examples -

Example 2.24.1 Write a JavaScript for password verification.

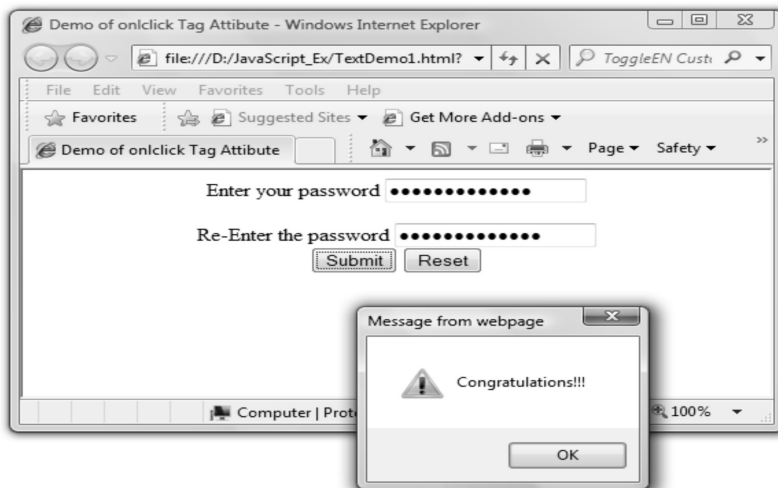
Solution :

```
<!DOCTYPE html>
<html>
<head>
    <title>Demo of onclick Tag Attribute</title>
<script type="text/javascript">
function my_fun()
{
    var mypwd=document.getElementById("pwd");
```

```
var my_re_pwd=document.getElementById("re_pwd");
if(mypwd.value=="")
{
    alert("You have not entered the password");
    mypwd.focus();
    return false;
}
if(mypwd.value!=my_re_pwd.value)
{
    alert("Password is not verified, Re-enter both the passwords");
    mypwd.focus();
    mypwd.select();
    return false;
}
else
{
    alert("Congratulations!!!!");
    return true;
}
}
</script>
</head>
<body>
    <center>
        <form id="form1">
            <label> Enter your password
            <input type="password" value="" id="pwd" />
            </label>
            <br/> <br/>
            <label> Re-Enter the password
            <input type="password" value="" id="re_pwd" onblur="my_fun();"/>
            </label> <br/>
            <input type="submit" value="Submit" name="submit" onsubmit="my_fun();"/>
            <input type="reset" value="Reset" name="reset"/> <br/>
        </form>
    </center>
</body>
</html>
```

Output (Run1)**Output (Run2)**

Output(Run3)



Example 2.24.2 Write JavaScript to validate a form consisting of Name, Age, Address, EmailID, hobby (check box), Gender (radio box), country (Drop down menu).

Solution : ApplicationForm.html

```

<html>
<head>
<title>The Student Registration Form</title>
<script type=text/javascript>
function validate()
{
    var i;
    var name_str=document.my_form.name;
    var phoneID=document.my_form.ph_txt;
    var ph_str=document.my_form.ph_txt.value;
    var str=document.my_form.Email_txt.value;
    if((name_str.value==null) || (name_str.value==""))
    {
        alert("Enter some name")
        return false
    }
    if(document.my_form.Age_txt.value=="")// validating age
    {
        alert("Enter Some Age")
        return false
    }
    if((document.my_form.Age_txt.value<"5")&&(document.my_form.Age_txt.value>"21"))

```

Validating Name

```
{
  alert("Invalid Age")
  return false
}
```

```
if(ph_str.length<1 || ph_str.length>11)
{
  alert("Invalid length of Phone Number")
  return false
}
for (i = 0; i < ph_str.length; i++)
{
  var ch = ph_str.charAt(i);
  if (((ch < "0") || (ch > "9"))){
    alert("Invalid Phone Number")
  }
}
```

Validating Phone Number

```
phoneID.focus()
return false
}
}
```

```
var index_at=str.indexOf("@")
var len=str.length
var index_dot=str.indexOf(".")
var emailID=document.my_form.Email_txt
if ((emailID.value==null) || (emailID.value==""))
{
  alert("Please Enter your Email ID")
  emailID.focus()
  return false
}
```

Validating Email ID

```
if (str.indexOf("@")==-1)
{
  alert("Invalid E-mail ID")
  return false
}
if (str.indexOf(".")=-1 || str.indexOf(".")=0
    || str.indexOf(".")=index_at)
{
  alert("Invalid E-mail ID")
  return false
}
```

Validating Email ID

```
if (str.indexOf("@",(index_at+1))!=-1)
{
  alert("Invalid E-mail ID")
  return false
}
```

```
        if (str.indexOf(" ") != -1)

        {
            alert("Invalid E-mail ID")
            return false
        }
        if (!document.my_form.group1[0].checked && !document.my_form.group1[0].checked)
        {
            alert("Please Select Sex");
            return false;
        }
        if (!document.my_form.group1[0].checked && !document.my_form.group1[0].checked)
        {
            alert("Please Select Sex");
            return false;
        }
        return true
    }
</script>
</head>
<body bgcolor=aqua>
<center><h3>Application Form</h3></center>
<form name=my_form onsubmit=validate()>
<strong>Name: </strong>
<input type=text name=name><br/>

<strong>Age: </strong>
<input type=text name=Age_txt><br/>

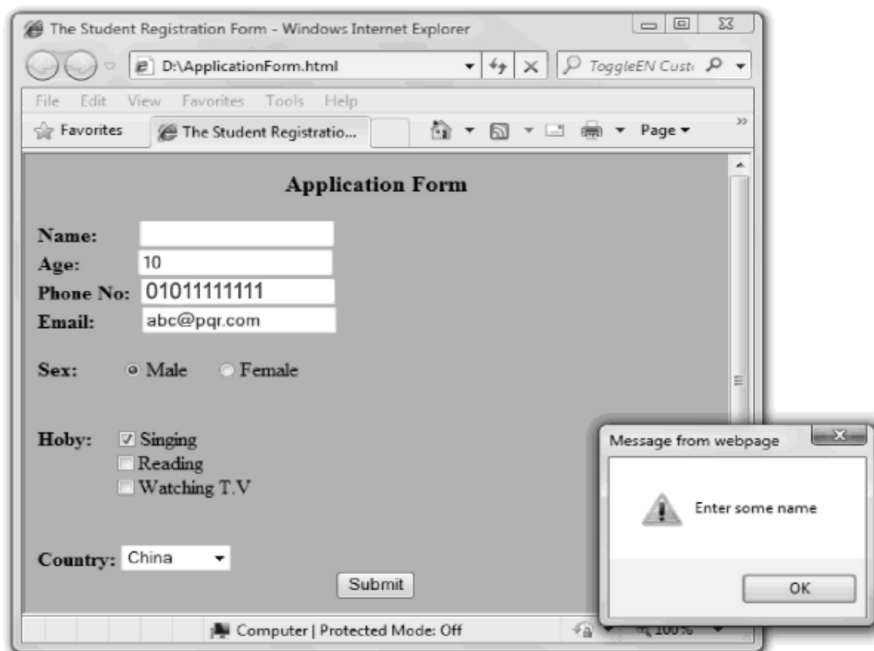
<strong>Phone No:</strong>
<input type=text name=ph_txt><br/>
<strong>Email: </strong>
<input type=text name=Email_txt><br/><br/>
<strong>Sex: </strong>
<input type="radio" name="group1" value="Male">Male
<input type="radio" name="group1" value="Female">Female<br/><br/><br/>
<strong>Hoby: </strong>
<input type="checkbox" name="option1" value="Singing">Singing<br/>
<input type="checkbox" name="option1" value="Reading">Reading<br/>
<input type="checkbox" name="option1" value="T.V.">Watching T.V<br/>
<br/><br/>
<strong>Country:</strong>
<select name="My_Menu">
```



```
<option value="India">India</option>
<option value="China">China</option>
<option value="Shrilanka">Shrilanka</option>
</select>
<center>
<input type=submit value=Submit> </br>
</center>

</body>
</html>
```

Output



Example 2.24.3 Design a HTML form for validation the users with fields user name and password and ok button which should receive the input from the user and responses as authorized or invalid user name or invalid password.

Solution :

```
<head>
<title>HTML FORM</title>
<script type=text/javascript>
function validate()
{
var user=document.form1.username;
```

```
    var pass=document.form1.password;
    if((user.value==null) || (user.value==""))
    {
        alert("Enter some user name");
    }
    if((pass.value==null) || (pass.value==""))
    {
        alert("Enter some password");

    }
    if(user.value=="admin")
    {
        alert("Valid user name");

    }
    if(pass.value=="password")
    {
        alert("Valid password");
    }
    else
        alert("Invalid username/password");
    }
</script>
</head>
<body>
    <form id="form1" name="form1" onsubmit=validate(>
    <table width="510" border="0" align="center">
    <tr>    <td>Username:</td>
    <td><input type="text" name="username"/></td>
    </tr>
    <tr>    <td>Password</td>
    <td><input type="password" name="password"/></td>
    </tr>
    <tr>
    <td>&nbsp;   </td>
    <td><input type="submit" name="button" value="OK" /></td>
    </tr>
    </table>
    </form>
</body>
</html>
```

Example 2.24.4 Write a Javascript to validate radio button, operator field and check box.

Solution : JavaScript Program

```

<html>
<head>
<script LANGUAGE="JavaScript">

function ValidateForm(form)
{
    if ((form.gender[0].checked == false ) && ( form.gender[1].checked == false ))
    { alert ( "Please choose your Gender: Male or Female" );
      return false;
    }
    if ((form.status[0].checked == false ) && ( form.status[1].checked == false ))
    { alert ( "Please choose your choice: AC or non AC" );
      return false;
    }
    if ( form.course.selectedIndex == 0 )
    { alert ( "Please select Some Course." );
      return false;
    }
    return true;
}
</script>
</head>
<body>
<br>
<form>
Your Gender: <input type="radio" name="gender" value="Male"> Male
<input type="radio" name="gender" value="Female"> Female
<br/>
Choice: <input type="checkbox" name="status"> AC
<input type="checkbox" name="status"> Non AC
<br/>
Course:
<select name="course">
  <option value="">Select an Option:</option>
  <option value="Computer">Computer</option>
  <option value="Mechanical">Mechanical</option>
  <option value="E&Tc">E&Tc</option>
</select>
<br/> <br/>
<input type="button" name="SubmitButton" value="Submit"

```

Validating Radio Button

Validating Checkbox

Validating Option

```
onClick="ValidateForm(this.form)">
<input type="reset" value="Reset">
</form>
</body>
</html>
```

Example 2.24.5 *Design a form to accept workshop registration details from participants and validate any five fields using JavaScript ? (Assume suitable structure)*

SPPU : Dec.-18, Marks 5

Solution :

```
<html>
<head>
<title>The Workshop Registration Form</title>
<script type="text/javascript">
function validate()
{

    var i;
    var name_str=document.my_form.name;
    var phoneID=document.my_form.ph_txt;
    var ph_str=document.my_form.ph_txt.value;
    var str=document.my_form.Email_txt.value;
    if((name_str.value==null) || (name_str.value==""))//validating name
    {
        alert("Enter some name")
        return false
    }

    if(document.my_form.Age_txt.value=="")// validating age
    {
        alert("Enter Some Age")
        return false
    }

    if((document.my_form.Age_txt.value<"5")&&(document.my_form.Age_txt.value>"21"))
    {
        alert("Invalid Age")
        return false
    }

    if(ph_str.length<1 || ph_str.length>11) //validating phone number
    {
        alert("Invalid length of Phone Number")
        return false
    }
}
```

```
}
for (i = 0; i < ph_str.length; i++)
{
    var ch = ph_str.charAt(i);
    if (((ch < "0") || (ch > "9"))){
        alert("Invalid Phone Number")

        phoneID.focus()
        return false
    }
}

var index_at=str.indexOf("@")
var len=str.length
var index_dot=str.indexOf(".")
var emailID=document.my_form.Email_txt
if ((emailID.value==null) || (emailID.value==""))
{
    alert("Please Enter your Email ID")
    emailID.focus()
    return false
}
if (str.indexOf("@")==-1) //validating email ID
{
    alert("Invalid E-mail ID")
    return false
}
if (str.indexOf(".")=-1 || str.indexOf(".")=0
    || str.indexOf(".")==index_at)
{
    alert("Invalid E-mail ID")
    return false
}

if (str.indexOf("@",(index_at+1))!=-1)
{
    alert("Invalid E-mail ID")
    return false
}
if (str.indexOf(" ")!=-1)
{
    alert("Invalid E-mail ID")
```

```
        return false
    }
    if (!document.my_form.group1[0].checked && !document.my_form.group1[0].checked)
    {
        alert("Please Select Sex");
        return false;
    }
    if (!document.my_form.group1[0].checked && !document.my_form.group1[0].checked)
    {
        alert("Please Select Sex");
        return false;
    }
    return true
}
</script>
</head>
<body bgcolor=aqua>
<center><h3>Registration Form</h3></center>
<form name=my_form onsubmit=validate()>
<strong>Name: </strong>
<input type=text name=name><br/>

<strong>Age: </strong>
<input type=text name=Age_txt><br/>

<strong>Phone No:</strong>
<input type=text name=ph_txt><br/>
<strong>Email: </strong>
<input type=text name=Email_txt><br/><br/>
<strong>Sex: </strong>
<input type="radio" name="group1" value="Male">Male
<input type="radio" name="group1" value="Female">Female<br/><br/><br/>
<strong>Hoby: </strong>
<input type="checkbox" name="option1" value="Singing">Singing<br/>
<input type="checkbox" name="option1" value="Reading">Reading<br/>
<input type="checkbox" name="option1" value="T.V.">Watching T.V<br/>
<br/><br/>
<strong>Country:</strong>
<select name="My_Menu">
<option value="India">India</option>
```

```
<option value="China">China</option>
<option value="Shrilanka">Shrilanka</option>
</select>
<center>
<input type=submit value=Submit> </br>
</center>
</body>
</html>
```

Part - III : jQuery

2.25 jQuery

SPPU : May-19, Marks 5

2.25.1 Introduction to jQuery

- JQuery is a fast and concise library created by **John Rosig** in 2006. The moto of this web document is **Write less do more**. This scripting is very easy to learn. The JQuery is actually a **Javascript library**.
- JQuery runs **almost all the browsers**.
- It adds up many **advanced, cross-browser functions** to standard

Advantages of using jQuery

1. It works on almost all the platforms - This is called as **cross platform compatibility**.
2. It has large and advanced set of functionalities.
3. It is **lightweight** about 19 kB in size.
4. It supports AJAX technology.
5. It offers **event handling** to capture a wide variety of events such as button clicks, mouse move and so on.
6. It supports plenty of built in **animation effects** to make the web document live and eye catching.
7. It supports **DOM manipulations**. That means it is easy to select DOM elements, traverse them and modifying their content using jQuery.

Disadvantages of using jQuery

1. Without the use of **jquery-1.10.2.min.js** either locally or from the internet we can not make use of the library functions of jQuery.
2. The jQuery coding must be done in support of JavaScript, AJAX, ASP, PHP and so on to get the advantage of its library functionality.

2.25.2 Basic Syntax

Basically the jQuery code is written in following form.

```
$(selector).action()
```

The \$ sign denotes the access to jQuery.

The (selector) finds the HTML elements such as <p>, <div>, <h1>,<form><button> and so on.

The **action()** denotes the actions to be performed on selector elements.

Document ready event : All the jQuery methods are written in document ready event.

2.25.3 Loading jQuery

- The **load()** method loads data from a server and puts the returned data into the selected element.

- **Syntax**

```
$(selector).load()
```

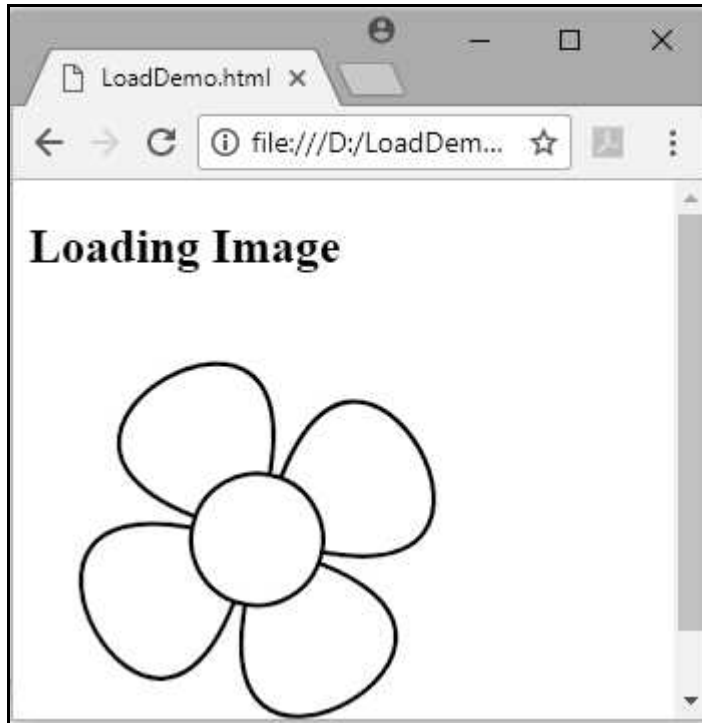
- **Example**

jQuery Document[LoadDemo.html]

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/1.11.3/jquery.min.js"> </script>
<script>
$(document).ready(function()
{
    $("img").load();
});
</script>
</head>
<body>
<h2> Loading Image</h2>

</body>
</html>
```


Output



We can load some external file using the load function for that purpose we need to pass URL to the load function.

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("#div1").load("input.html");
    });
});
</script>
</head>
<body>
<div id="div1"><h2>Loading File</h2></div>
<button>Click to load file</button>
</body>
</html>
```

2.25.4 Selecting Elements

1. The element selector

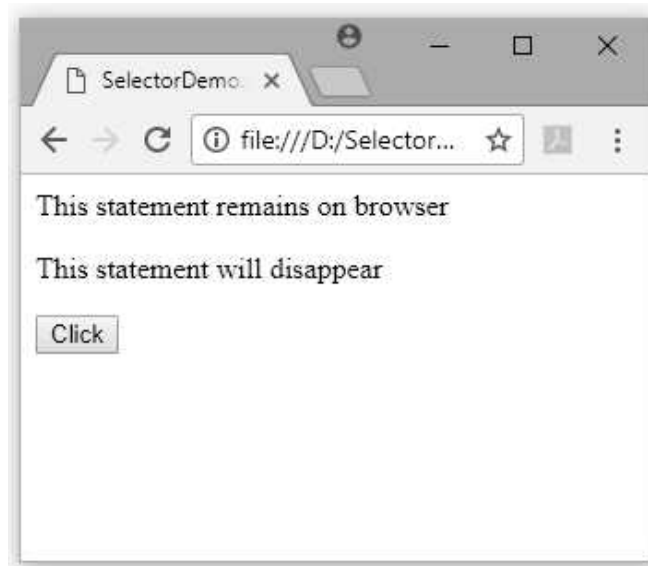
As we know hat, the selectors can be the basic HTML elements such as<p>, <div>, <h1>, <form><button> and so on.

Let us understand jQuery coding with the help of simple example in which the HTML element <p> is used. Following code can be written using a simple text editor like notepad. Save the code with any suitable file name and extension **.html**

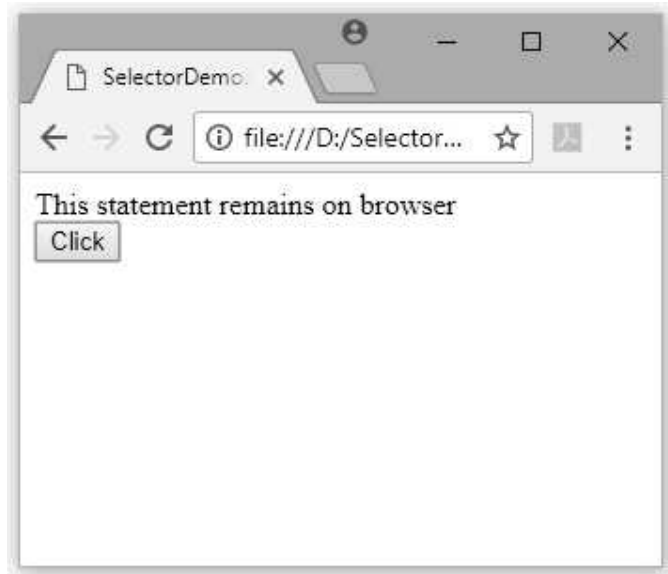
SelectorDemo.html

```
<html>
<head>
<script src="http://ajax.googleapis.com/ajax/libs/jquery/1.10.2/jquery.min.js">
</script>
<script>
$(document).ready(function()
{
    $("button").click(
function()
    {
        $("p").hide();
    }
    );
});
</script>
</head>

<body>
<div>This statement is remains on browser</div>
<p>This statement will disappear</p>
<button>Click</button>
</body>
</html>
```

Output

On clicking the button we will get

**Script Explanation :**

In the above code, within the **ready** function the jQuery code is written.

For the button element selector the action is **click**. This action invokes the `<p>` element and the action associated with this element is **hide**. This means that when user clicks the **button** the text written within the `<p> </p>` will be hidden or disappeared.

2. Use of * as a selector

The * means all. If we include selector as * then the corresponding action will be associated with all the HTML elements. The sample code for demonstrating this action is as follows -

```
<html>
<head>
<script src="http://ajax.googleapis.com/ajax/libs/jquery/1.10.2/jquery.min.js">
</script>
<script>
$(document).ready(function()
{
  $("button").click(
function()
{
  $("*").hide();
}
);
});
</script>
</head>

<body>
<h1> This is a header </h1>
<div>This second statement</div>
<p>This is another statement</p>

<button>Click</button>
</body>
</html>
```

Output

On clicking the click button all the contents of the web page will be hidden and black document will be displayed.

3. The id selector

The **id** can be associated with the HTML element. In the following example we have used <div> element to which the **id** named **myid** is associated. On button click event the function is invoked to hide the sentence written using the <div id="myid">.

To find the element with particular id write # and then name of the id.

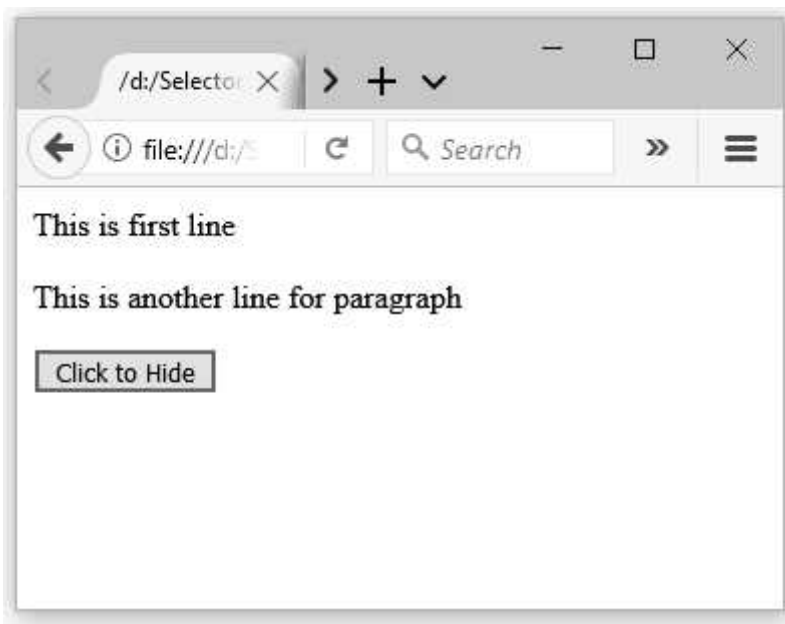
```
<html>
<head>
```

```
<script src="http://ajax.googleapis.com/ajax/libs/jquery/1.10.2/jquery.min.js">
</script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $("#myid").hide();
  });
});
</script>
</head>
<body>
<div>This is first line</div>
<p>This is another line for paragraph</p>
<div id="myid">This sentence will be hidden on clicking the button</div>
<button>Click to Hide</button>
</body>
</html>
```

Output



On clicking the Hide button we get following effect



4. The .class selector

The class selector is also used to find the specific html element. The element is written along with class="classname" and to find this element write a dot character followed by **classname**. Following example illustrates this

```
<html>
<head>
<script src="http://ajax.googleapis.com/ajax/libs/jquery/1.10.2/jquery.min.js">
</script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $(".myclass").hide();
    });
});
</script>
</head>
<body>
<div>This is first line</div>
<p>This is another line for paragraph</p>
<div class="myclass">This sentence will be hidden on clicking the button</div>
<button>Click to Hide</button>
</body>
</html>
```

Output

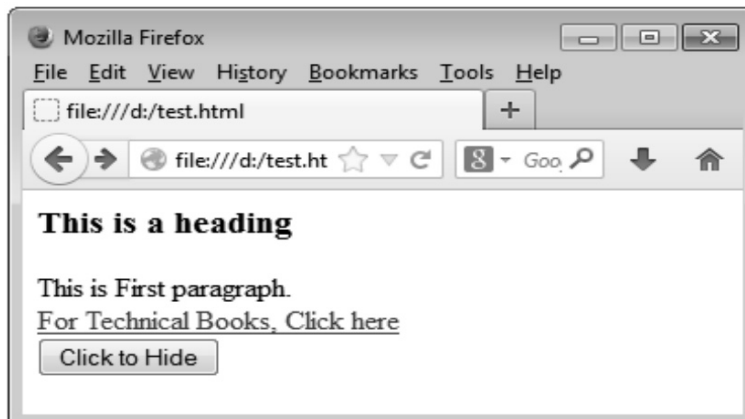
Note : The output will be the same as above

5. The href selector

The href element is used along with the anchor tag in order to introduce hyperlink in the web document. We can navigator to the desired web page using such hyperlinks. In the following program on button click, the hyperlink can be hidden.

```
<html>
<head>
<script src="http://ajax.googleapis.com/ajax/libs/jquery/1.10.2/jquery.min.js">
</script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $('[href]').hide();
  });
});
</script>
</head>

<body>
<h3>This is a heading</h3>
<div>This is First paragraph.</div>
<div>
<a href="https://www.vtubooks.com">For Technical Books, Click here</a>
</div>
<button>Click to Hide</button>
</body>
</html>
```

Output

6. The tr selector

The element **tr** is used to define the rows of the table. We can denote this selector along with the css style. It is demonstrated by following web document

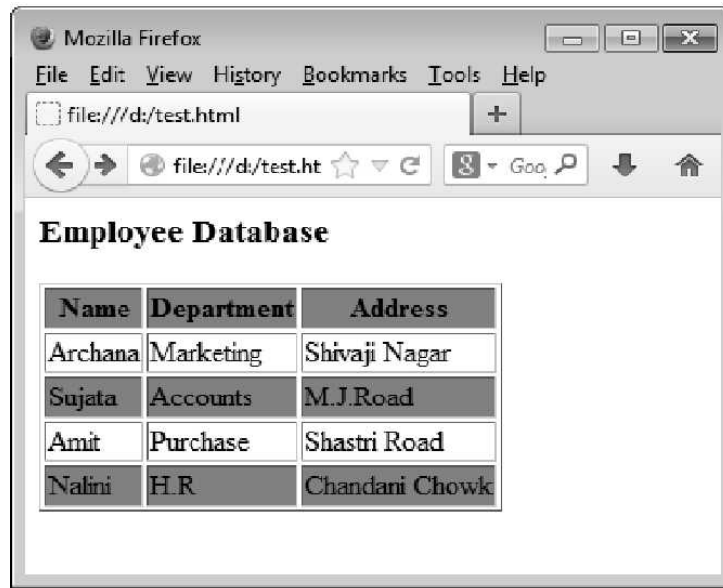
```
<html>
<head>
<script src="http://ajax.googleapis.com/ajax/libs/jquery/1.10.2/jquery.min.js">
</script>
<script>
$(document).ready(function(){
  $("tr:even").css("background-color","grey");
});
</script>
</head>

<body>
<h3>Employee Database</h3>
<table border="1">
<tr>
<th>Name</th>
<th>Department</th>
<th>Address</th>
</tr>
<tr>
<td>Archana</td>
<td>Marketing</td>
<td>Shivaji Nagar</td>
</tr>

<tr>
<td>Sujata</td>
<td>Accounts</td>
<td>M.J.Road</td>
</tr>
<tr>
<td>Amit</td>
<td>Purchase</td>
<td>Shastri Road</td>
</tr>

<tr>
<td>Nalini</td>
<td>H.R</td>
<td>ChandaniChowk</td>
</tr>

</body>
</html>
```


Output**2.25.5 Changing Styles**

To change the style the `.css()` function is used. The syntax is

```
$(selector).css({"css property name":"css property value"});
```

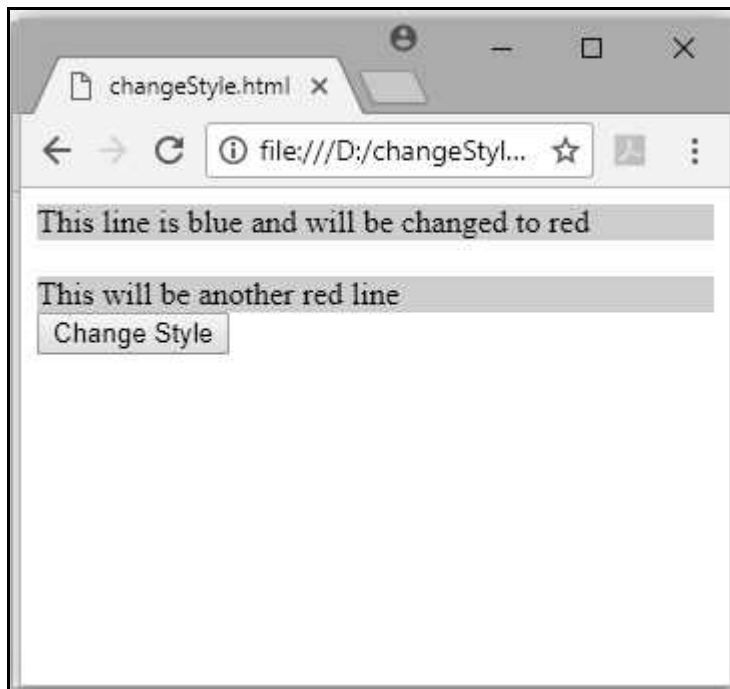
For example -

jQuery Document[changeStyle.html]

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js">
</script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("div").css("background-color", "red");
    });
});
</script>
</head>
<body>
<div style="background-color:blue">This line is blue and will be changed to red</div> <br/>
```

```
<div style="background-color:yellow">This will be another red line</div>
<button>Change Style</button>
</body>
</html>
```

Output



2.25.6 Creating Elements

- The jQuery **after()** method inserts content AFTER the selected HTML elements. The syntax is

```
$(Selector).after("Some Text");
```

- The jQuery **before()** method inserts content BEFORE the selected HTML elements. The syntax is

```
$(Selector).before("Some Text");
```

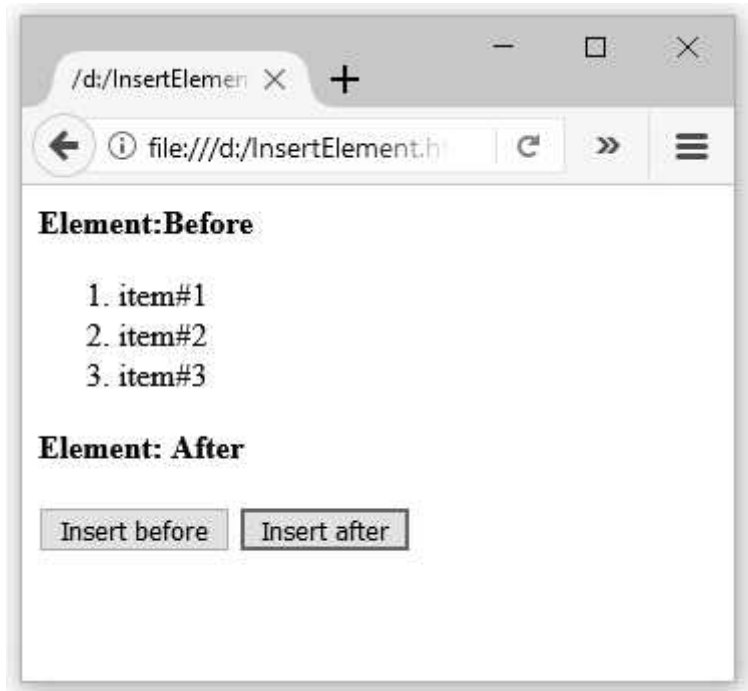
Example

jQuery Document[InsertElement.html]

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
<script>
```

```
$(document).ready(function(){
    $("#button1").click(function(){
        $("ol").before("<b>Element:Before</b>");
    });
    $("#button2").click(function(){
        $("ol").after("<b>Element: After</b>");
    });
});
</script>
</head>
<body>
<ol>
    <li>item#1</li>
    <li>item#2</li>
    <li>item#3</li>
</ol>
<br/> <br/>
<button id="button1">Insert before</button>
<button id="button2">Insert after</button>
</body>
</html>
```

Output



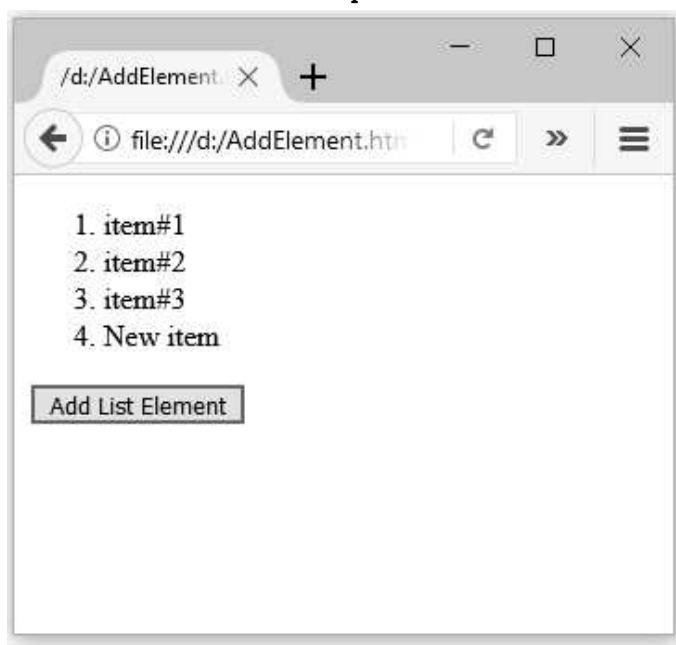
2.25.7 Appending Elements

We can add new element at the end of existing elements using the append function. The appending of elements can be well understood by following example.

jQuery Document[AddElement.html]

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
    $("#button").click(function(){
        $("ol").append("<li>New item</li>");
    });
});
</script>
</head>
<body>
<ol>
<li>item#1</li>
<li>item#2</li>
<li>item#3</li>
</ol>
<button id="button">Add List Element</button>
</body>
</html>
```

Output



2.25.8 Removing Elements

To remove elements from the DOM there are two commonly used methods - `remove` and `empty`. jQuery `.empty()` method removes all the child element of the matched element where `remove()` method removes set of matched elements from DOM.

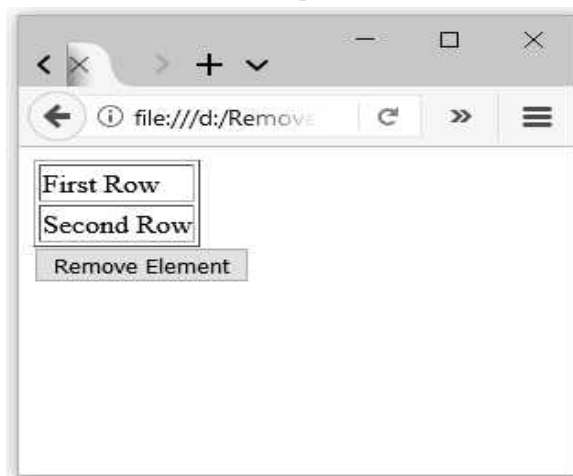
```
$( ".content" ).empty();
```

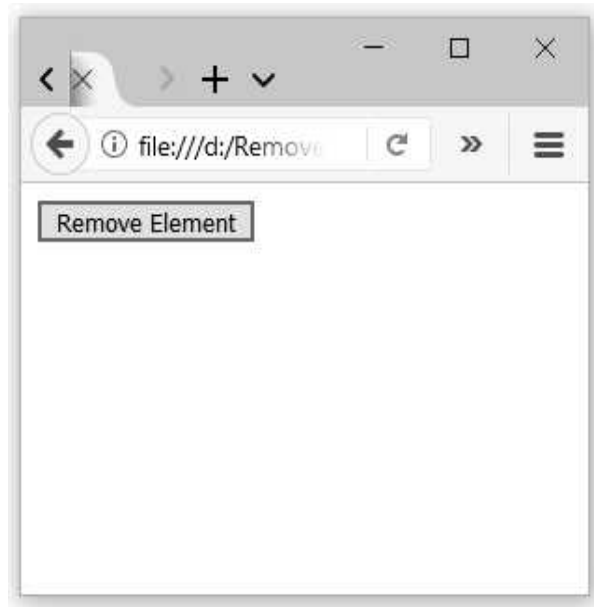
```
$( ".content" ).remove();
```

Example of `.remove()`

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("#tbl").remove();
    });
});
</script>
</head>
<body>
<table id="tbl" border='1'>
<tr><td> First Row</td> </tr>
<tr><td> Second Row </td> </tr>
</table>
<button>Remove Element</button>
</body>
</html>
```

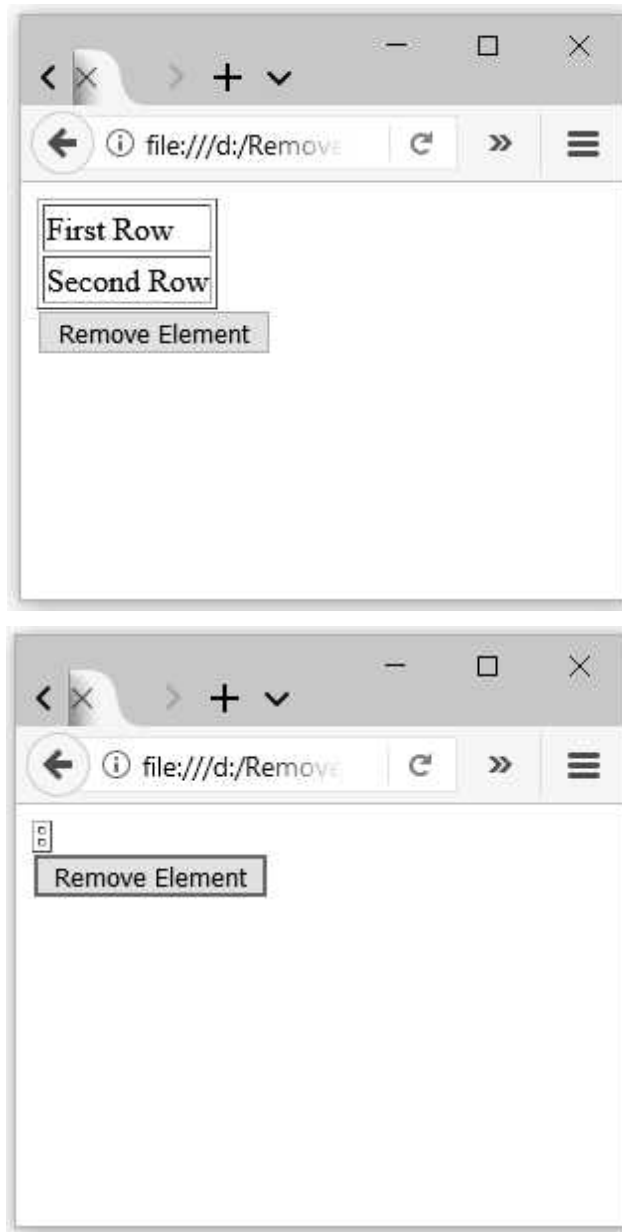
Output





Example of .empty()

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("#tbl tr td").empty();
    });
});
</script>
</head>
<body>
<table id="tbl" border='1'>
<tr><td> First Row</td> </tr>
<tr><td> Second Row </td> </tr>
</table>
<button>Remove Element</button>
</body>
</html>
```

Output**Review Question**

1. When to use jQuery ? What are the advantages of using jQuery over JavaScript.

SPPU : May-19, Marks 5

Part - IV : Angular JS**2.26 Overview of Angular JS****SPPU : May-18, 19, Dec.-18, 19, Marks 8**

- Angular JS is a powerful JavaScript framework.
- It is an open source front-end enabled web application framework.
- It is licensed under Apache license version 2.0.
- It extends HTML DOM with additional attributes. It is more responsive to user actions.

Advantages

Following are the advantages of Angular JS.

1. **Built by Google Engineers** : The Angular JS is maintained by dedicated Google Engineers. That means - there is huge community for development and enhancement.
2. **MVC Support** - It is based on Model View Controller (MVC) architecture which provides separation of model, view and controller components.
3. **Intuitive** - It is more intuitive as it makes use of HTML as a declarative language. Moreover, it is less brittle for reorganizing.
4. **Comprehensive** : AngularJS is a comprehensive solution for rapid front-end development. It does not need any other plugins or frameworks.
5. **Rich Features** : There is a wide range of features supported by Angular JS such as Data binding, dependency injection, enterprise level testing and so on.
6. **Unit Testing** : The Angular JS code is unit testable.
7. **Reusable Code** : Angular JS support for using the reusable components.
8. **Less Code** : It support for less code and more functionality.

Disadvantages

1. As Angular JS is based on JavaScript framework, it is not secure.
2. There are multiple ways to do the same thing with AngularJS. Sometimes, it can be hard for novices to say which way is better for a task.

Features of Angular JS

There are core features Angular JS that are widely used by web developers. These are -

1. **Data Binding** : It allows the automatic synchronization between model and view components.

2. **Scope** : There are some objects from model that can be correlated to controller and view.
3. **Controller** : These are basically JavaScript functions that are bound to particular scope.
4. **Directives** : The directives is designed to give HTML new functionality.
5. **Filters** : This feature allows to select subset from array of items.
6. **Routing** : This feature allows to switch between multiple views.
7. **Services** : There is a support for many built in services for Angular JS
8. **Dependency Injection** : It is a software design pattern that helps the developer to develop and understand the application easily.

2.26.1 MVC with Angular JS

Concept of MVC

The MVC stands for Model, view and controller. It is a pattern for the architectural framework. It consists of three parts -

1. **Model** : This part of the architecture is responsible for managing the application data. This module responds to the request made from **view**. The **model** gives instructions to **controller** to update when the response is made.
2. **View** : This part takes care of the presentation of data. The data is presented in desired format with the help of **view**. This is a script based system using JSP, ASP, PHP and so on.
3. **Controller** : The controller receives input, validates it, and then performs business operations that modify the state of the data model. The **controller** basically responds to user request and performs interaction with **model**. Refer Fig. 2.26.1.

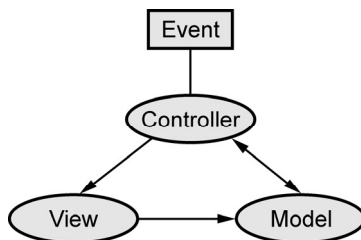


Fig. 2.26.1 MVC architecture

MVC in Angular JS

- Modules in AngularJS serve as containers to help you organize your application into logical units.

- Modules tell AngularJS how an application is configured.
- In angular JS the Module are represented using the directive **<ng-app>** and module name.
- The **scope object** binds the **view** with **controller**.
- In AngularJS it is possible to have multiple views for single page application.
- The **ng-view** and **ng-template** and **ng-route** directives support the concept of views.
- The use of controller in AngularJS is to control the flow of data. The **ng-controller** directive is used for this purpose. Refer Fig. 2.26.2.

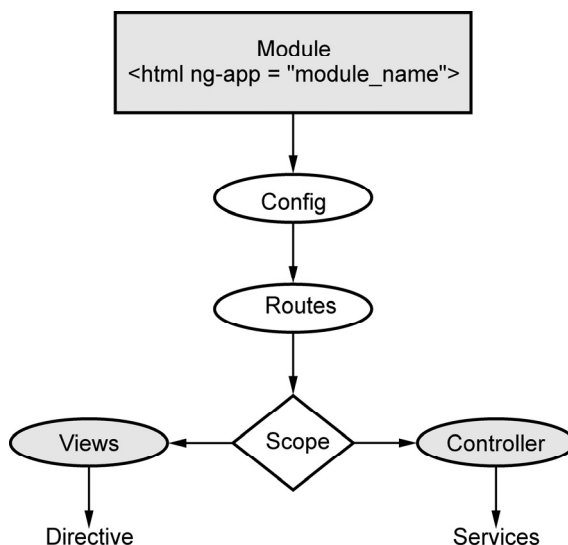


Fig. 2.26.2 MVC with angular JS

2.26.2 Directives

- In Angular JS, the capacity of HTML is extended using the **ng-directives**.
- The directives are specified with **ng** prefix.
- The three commonly used ng-directives are
 1. **ng-app** : This directive defines an AngularJS application.
 2. **ng-model** : This directive binds the value of HTML controls to application data. These controls can be input, select, textarea and so on.
 3. **ng-bind** : This directive binds application data to HTML view.

Let us now learn and understand how to develop Angular JS application.

Step 1 : Create an Angular JS code as follows in a Notepad. Save it using the extension .htm or .html

AngularJS Document[FirstApp.html]

```
<html>
  <head>
    <title>My First AngularJS Application</title>
    <script src =
"https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
  </head>
  <body>
    <h1>AngularJS Application</h1>
    <div ng-app = "">
      <p>Enter your Name: <input type = "text" ng-model = "name"></p>
      <p>Welcome <span ng-bind = "name"></span></p>
    </div>
  </body>
</html>
```

Output



Script Explanation :

1. The AngularJS script is written within <html> tags. The AngularJS is based on JavaScript framework. The framework for AngularJS is loaded using the <script> tag. It is as follows -

```
<script src =  
"https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
```

2. Defining np-app directive : The np-app directive is defined in above script as follows -

```
<div ng-app = "">  
...  
...  
</div>
```

3. Defining np-model directive : The **textbox** control is modeled using np-model directive

```
<p>Enter your Name: <input type = "text" ng-model = "name"></p>
```

4. Defining np-bind : We can bind the value entered in the textbox with **name** using np-bind directive.

```
<p>Welcome <span ng-bind = "name"></span></p>
```

Hence, we can see on the output screenshot, that whatever name entered by the user gets bound with the **Welcome** message.

The above output can be viewed by opening any popular web browser and by entering the URL of corresponding AngularJS script.

Various Directives supported by AngularJS are -

1. **ng-app**
2. **ng-init**
3. **ng-model**
4. **ng-repeat**

Let us discuss these directives with suitable scripting examples

1. ng-app Directive :

This directive starts the AngularJS application. It basically defines the **root element**. It initializes the application automatically when the web page containing the AngularJS code is loaded. For example -

```
<div ng-app="">  
...  
</div>
```

2. ng-init Directive :

The ng-init directive is used for initialization of AngularJS data.

For example :

DirectiveDemo1.html

```
<html>  
<script src = "https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>  
<body>  
  <div ng-app="" ng-init="msg='I love my country!!'">
```

```
        <h3>{{msg}}</h3>
    </div>
</body>
</html>
```

Output



3. **ng-model** : The ng-model directive binds the value of HTML control to application data. These controls can be input, select, textarea and so on. For example -

AngularJS Document[FirstApp.html]

```
<html>
  <head>
    <title>My First AngularJS Application</title>
    <script src =
"https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"> </script>
  </head>
  <body>
    <h1>AngularJS Application</h1>
    <div ng-app = "">
      <p>Enter your Name: <input type = "text" ng-model = "name"> </p>
      <p>Welcome <span ng-bind = "name"> </span> </p>
    </div>
  </body>
</html>
```

Output

AngularJS Application

Enter your Name:

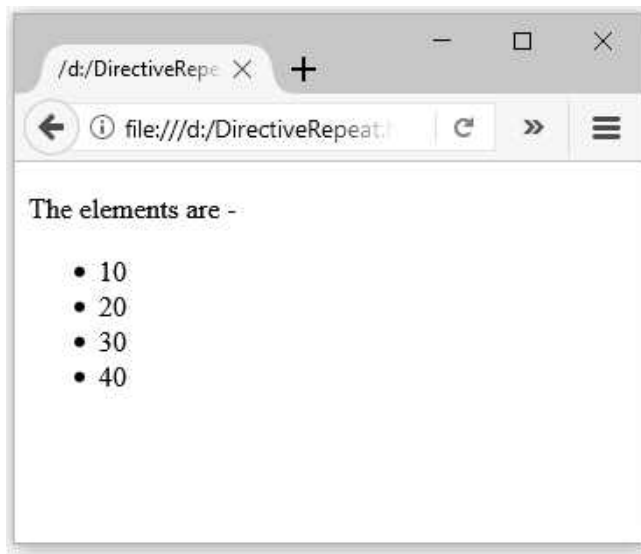
Welcome Mr.XYZ

Name in text box are represented using ng-model directive and is linked with application data using ng-bind

4. ng-repeat : The ng-repeat element directive repeats the HTML element.

AngularJS Document [DirectiveRepeat.html]

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>
<div ng-app="" ng-init="nums=[10,20,30,40,50]">
  <p>The elements are -</p>
  <ul>
    <li ng-repeat="i in nums">
      {{ i }}
    </li>
  </ul>
</div>
</body>
</html>
```

Output**2.26.3 Expression**

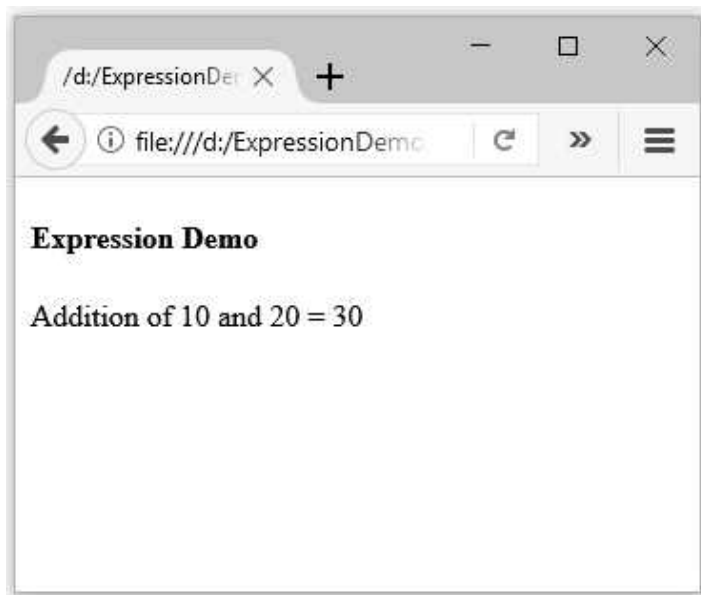
- In AngularJS the expressions are written within `{{ }}`. For example - `{{2 + 3}}` is expression.
- AngularJS binds data to HTML using Expressions.
- AngularJS solves the expression and return the result of evaluation.
- The expressions in AngularJS are just similar to JavaScript expressions. The expression contains literals, variables and operators.

AngularJS Document[ExpressionDemo.html]

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>
<div ng-app="">
  <h4>Expression Demo</h4>
  <p> Addition of 10 and 20 = {{ 10+20 }} </p>
</div>

</body>
</html>
```

Output



Now from above code if we remove `ng-app` directive then the expression won't be evaluated. For instance -

<pre><body> <div> <h4>Expression Demo</h4> <p> Addition of 10 and 20 = {{ 10+20 }}</p> </div> </body> </html></pre>	A screenshot of a web browser window showing the output of an AngularJS expression. The browser title is '/d:/ExpressionDemo'. The address bar shows 'file:///d:/ExpressionDemo'. The page content displays 'Expression Demo' followed by 'Addition of 10 and 20 = {{ 10+20 }}'.
---	--

We can assign the values to variables and then evaluate an expression. This initialization of variable can be using **ng-init**. For example -

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>
<div ng-app="" ng-init="a=10;b=20">
  <h4>Expression Demo</h4>
  <p> Addition = {{ a+b }}</p>
  <p> Multiplication = {{ a*b }}</p>
</div>
</body>
</html>
```


Output

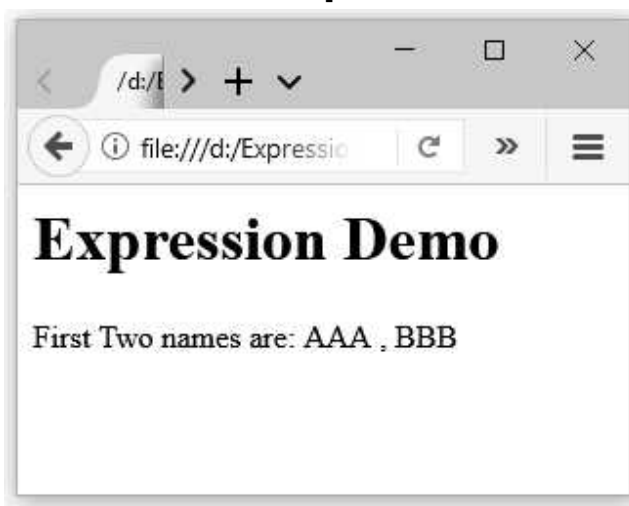
The output will be displayed as 30 and 200

Similarly we can assign strings to variables and can display them. For example

AngularJS Document[ExpressionDemo1.html]

```
<html>
<script src = "https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"> </script>
<body>
  <h1>Expression Demo</h1>

  <div ng-app = "" ng-init = "names = {first:'AAA',second:'BBB',third:'CCC'}">
    <p>First Two names are: {{names.first + " , " + names.second}}</p>
  </div>
</body>
</html>
```

Output**2.26.4 Controllers**

- The use of controller in AngularJS is to control the flow of data. The **ng-controller** directive is used for this purpose.
- The controller is JavaScript object containing properties and functions.
- Each controller accept \$scope as a parameter which refers to the application/module that controller is to control.
- We can use controller with properties, methods and external files.

Controller with Properties

The controller is defined with properties. These properties are then assigned with some values. It is illustrated by following example

AngularJS Document[ControllerDemo.html]

```
<!DOCTYPE html>
<html>
<script src="http://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>

<div ng-app="MyApp" ng-controller="MyController">

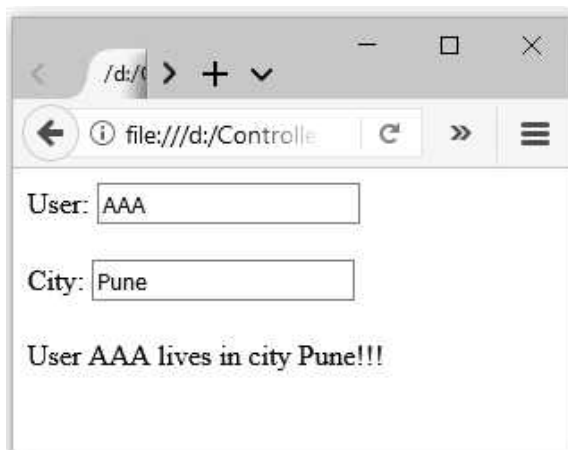
User: <input type="text" ng-model="user"> <br/> <br/>
City: <input type="text" ng-model="city"> <br/>
<br/>
User {{user + " "}} lives in city {{city + "!!!"}}

</div>

<script>
var app = angular.module('MyApp', []);
app.controller('MyController', function($scope)
{
    $scope.user = "AAA";
    $scope.city = "Pune";
});
</script>
</body>
</html>
```

Properties

Output



Script Explanation :

In above script,

1. We have used two directives **div-app** and **div-controller** which are defined with some suitable attribute name. In above case **div-app** attribute is "MyApp" and **div-controller** attribute is "MyController".
2. The **MyController** function is basically a JavaScript function and AngularJS will invoke the controller using **\$scope** object.
3. The controller creates two properties namely - 'user' and 'city'. Using **\$scope** object these properties are assigned with values 'AAA' and 'Pune'
4. The **ng-model** binds these input fields to the controller properties.

Controller with Method

In above script the controller has two properties. But we can define a method for the controller. Following example illustrates how to use controller method.

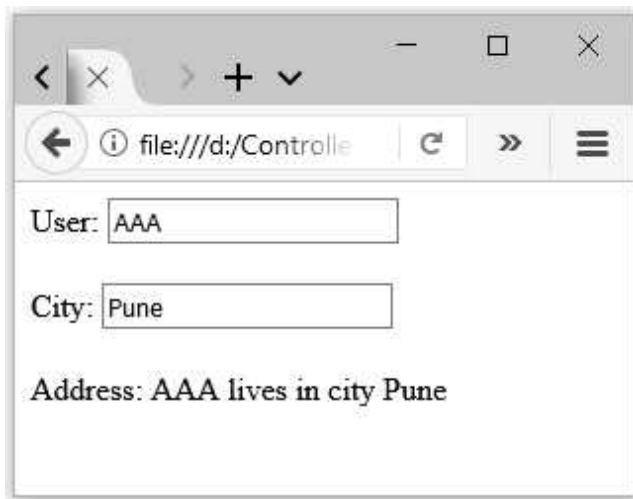
AngularJS Document[ControllerMethodDemo.html]

```
<!DOCTYPE html>
<html>
<script src="http://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>
<div ng-app="MyApp" ng-controller="MyController">
  User: <input type="text" ng-model="user"> <br/> <br/>
  City: <input type="text" ng-model="city"> <br/>
  <br/>
  Address: {{address_function()}}
</div>
</script>

var app = angular.module('MyApp', []);
app.controller('MyController', function($scope) {
  $scope.user = "AAA";
  $scope.city = "Pune";
  $scope.address_function=function() {
    return $scope.user + " lives in city " + $scope.city;
  };
});
</script>
</body>
</html>
```

} Properties

← Method definition

Output**Script Explanation :**

In above script

1. The two properties **user** and **city** are defined. Along with them, a method named **Address_function()** is invoked.
2. This function returns the string containing values to the properties.
3. This function is then bounded to HTML.

Controller with external File

We can write and save the controller in some external file. This file can be then invoked in the HTML document. Here is the illustration.

Step 1 : Create a file in Notepad. Name it with extension .js.

External File[MyController.js]

```
angular.module('MyApp', []).controller('MyController', function($scope) {  
    $scope.user = "AAA";  
    $scope.city = "Pune";  
    $scope.address_function=function() {  
        return $scope.user+" "+$scope.city;  
    }  
});
```

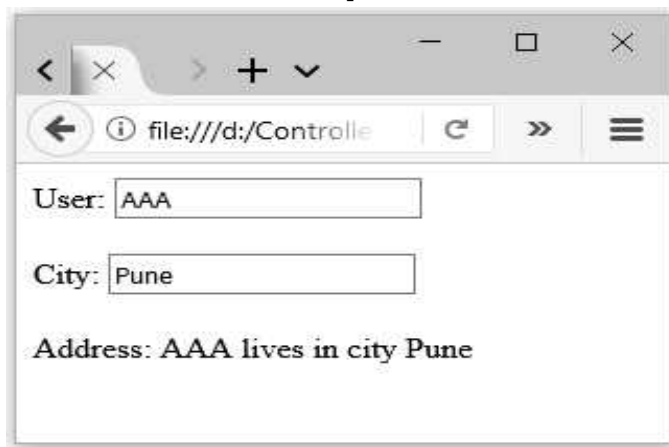
Step 2 : Create a HTML document invoking the external js file created in above step.

HTML Document[ControllerFileDemo.html]

```
<!DOCTYPE html>
<html>
<script src="http://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>

<div ng-app="MyApp" ng-controller="MyController">

User: <input type="text" ng-model="user"> <br/> <br/>
City: <input type="text" ng-model="city"> <br/>
<br/>
Address:{{user + " lives in " + city}}
</div>
<script src="MyController.js"> </script>
</body>
</html>
```

Output**2.26.5 Filters**

- Filters are used to modify the data.
- These are used along with expressions or directives using pipe (|) character.
- Commonly used filters are -

Filter	Purpose
uppercase	Converts the text to uppercase.
lowercase	Converts the text to lower case.
currency	Converts the number into currency format.
orderBy	Arranges the elements of the array based on criteria.
filter	Selects the subset of items from array.

1. Example Demonstrating uppercase filter

AngularJS Document[FilterDemo1.html]

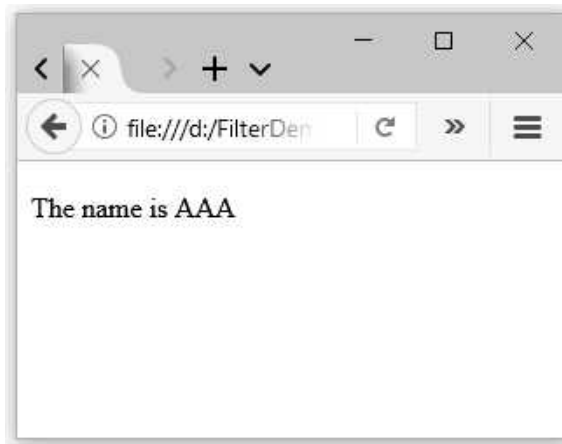
```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"> </script>
<body>

<div ng-app="MyApp" ng-controller="MyController">
<p>The name is {{ user | uppercase }}</p>

</div>

<script>
angular.module('MyApp', []).controller('MyController', function($scope) {
    $scope.user = "aaa"
});
</script>
</body>
</html>
```

Output



2. Example Demonstrating lowercase Filter

AngularJS Document[FilterDemo2.html]

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"> </script>
<body>

<div ng-app="MyApp" ng-controller="MyController">
```

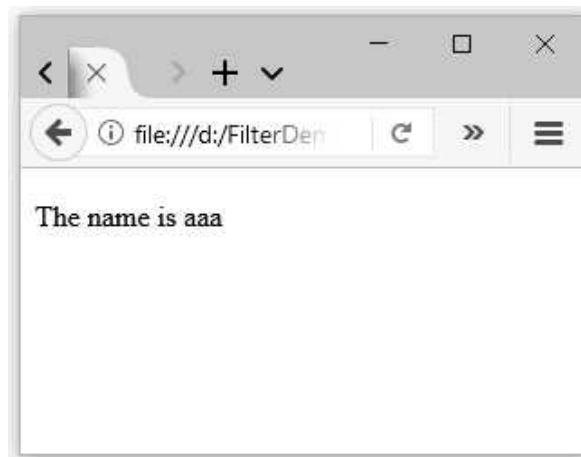
```
<p>The name is {{ user | lowercase }}</p>

</div>

<script>
angular.module('MyApp', []).controller('MyController', function($scope) {
    $scope.user = "AAA"
});
</script>

</body>
</html>
```

Output



3. Example Demonstrating currency Filter

AngularJS Document[FilterDemo3.html]

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>

<div ng-app="MyApp" ng-controller="MyController">
<p>The amount is {{ amount | currency }}</p>

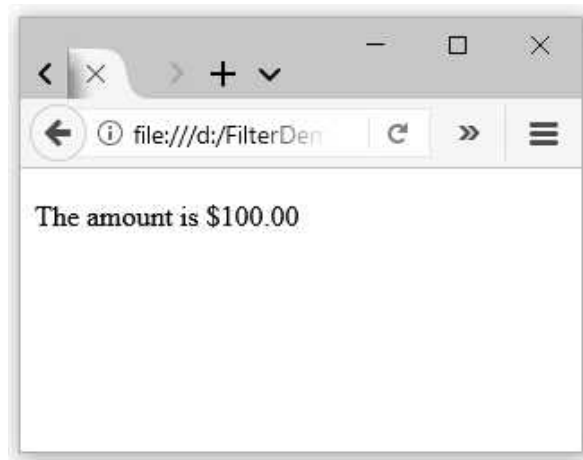
</div>

<script>
angular.module('MyApp', []).controller('MyController', function($scope) {
    $scope.amount = 100
});
```

```
</script>

</body>
</html>
```

Output



4. Example Demonstrating filter Filter

AngularJS Document[FilterDemo4.html]

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>

<div ng-app="MyApp" ng-controller="MyController">
<p>The cities having character 'u' are </p>
<ul>
  <li ng-repeat="cty in city | filter:'u'">
    {{cty}}
  </li>
</ul>
</div>

<script>
angular.module('MyApp', []).controller('MyController', function($scope) {
  $scope.city = [
    'Pune',
    'Mumbai',
    'Delhi',
    'Chennai',
    'Surat',
```



```
        'Kolkatta',  
        'Dehradun'  
    ];  
  
});  
</script>  
  
</body>  
</html>
```

Output



The above code can be modified in such a way that user is allowed to type a character in textbox. The list of cities containing the character typed by the user can be displayed.

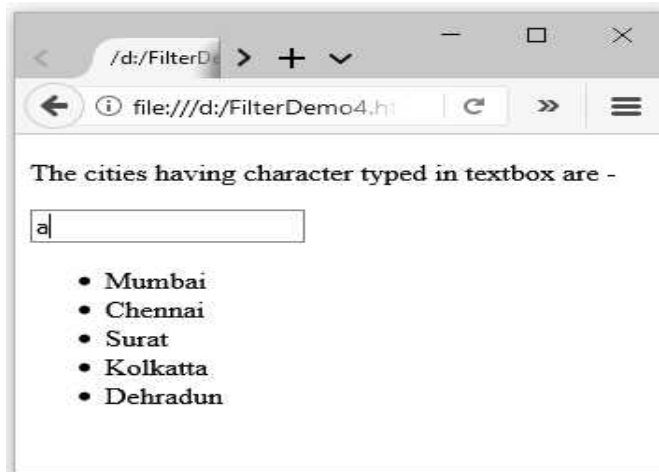
AngularJS Document[FilterDemo4.html]

```
<!DOCTYPE html>  
<html>  
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>  
<body>  
  
<div ng-app="MyApp" ng-controller="MyController">  
<p>The cities having characters typed in textbox are -</p>  
<p><input type="text" ng-model="ch"></p>  
<ul>  
  <li ng-repeat="cty in city | filter:ch">  
    {{cty}}  
  </li>  
</ul>  
</div>
```

```
<script>
angular.module('MyApp', []).controller('MyController', function($scope) {
    $scope.city = [
        'Pune',
        'Mumbai',
        'Delhi',
        'Chennai',
        'Surat',
        'Kolkatta',
        'Dehradun'
    ];
});
</script>

</body>
</html>
```

Output



5. Example Demonstrating orderBy Filter

AngularJS Document[FilterDemo5.html]

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"> </script>
<body>

<div ng-app="MyApp" ng-controller="MyController">
<h3>The names are listed in ascending order of city name </h3>
```

```
<ul>
  <li ng-repeat="i in personinfo | orderBy:'city'">
    {{i.name + " lives in " + i.city}}
  </li>
</ul>
</div>

<script>
angular.module('MyApp', []).controller('MyController', function($scope) {
  $scope.personinfo = [
    {name:'AAA',city:'Pune'},
    {name:'BBB',city:'Banglore'},
    {name:'CCC',city:'Chennai'},
    {name:'DDD',city:'Delhi'},
    {name:'EEE',city:'Mumbai'},
  ];

});
</script>

</body>
</html>
```

Output



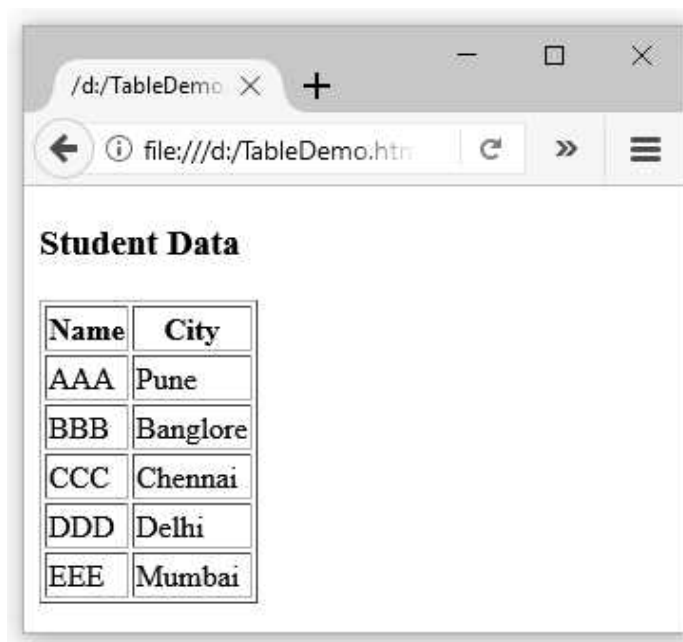
2.26.6 Tables

The **ng-repeat** directive is used for displaying each row of the table.

Following script demonstrates how to display the information in tabular form

AngularJS Document[TableDemo.html]

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>
<div ng-app="MyApp" ng-controller="MyController">
<h3>Student Data</h3>
<table border="1">
  <tr>
    <th>Name</th><th>City</th>
  </tr>
  <tr ng-repeat="p in personinfo">
    <td>{{p.name}}</td>
    <td>{{p.city}}</td>
  </tr>
</table>
</div>
<script>
angular.module('MyApp', []).controller('MyController', function($scope) {
  $scope.personinfo = [
    {name:'AAA',city:'Pune'},
    {name:'BBB',city:'Banglore'},
    {name:'CCC',city:'Chennai'},
    {name:'DDD',city:'Delhi'},
    {name:'EEE',city:'Mumbai'},
  ];
});
</script>
</body>
</html>
```

Output

We can display the above table using the ascending order of **city names** and for that purpose we need to use the filter orderBy. The illustrative code is as follows -

AngularJS Document

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>

<div ng-app="MyApp" ng-controller="MyController">
<h3>Student Data</h3>
<table border="1">
  <tr>
    <th>Name</th><th>City</th>
  </tr>
  <tr ng-repeat="p in personinfo | orderBy:'city'">
    <td>{{p.name}}</td>
    <td>{{p.city}}</td>
  </tr>
</table>
</div>

<script>
angular.module('MyApp', []).controller('MyController', function($scope) {
```

```

$scope.personinfo = [
    {name:'AAA',city:'Pune'},
    {name:'BBB',city:'Banglore'},
    {name:'CCC',city:'Chennai'},
    {name:'DDD',city:'Delhi'},
    {name:'EEE',city:'Mumbai'},
];

});
</script>
</body>
</html>

```

Tables with CSS

It is possible to apply the CSS style to the table. For instance following script shows the odd rows in blue and even rows in yellow color

AngularJS Document[TableCSSDemo.html]

```

<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<style>
table, tr, td {
border: 2px solid;
padding: 5px;
}
table tr:nth-child(odd) {
background-color: blue;
}
table tr:nth-child(even) {
background-color: yellow;
}

</style>
<body>
<div ng-app="MyApp" ng-controller="MyController">
<h3>Student Data</h3>
<table>
  <tr>
  </tr>
  <tr ng-repeat="p in personinfo">
    <td>{{p.name}}</td>
    <td>{{p.city}}</td>
  </tr>

```

Table with border of 2px and padding of 5 px

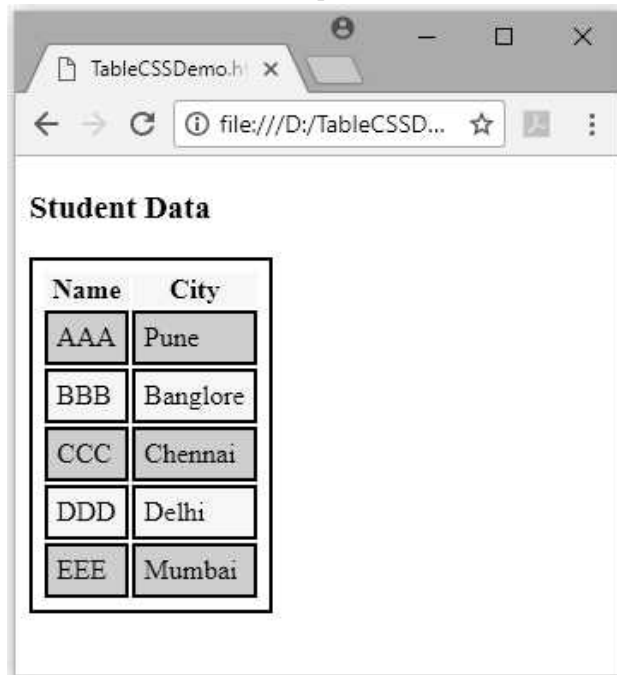
Setting the style: odd rows are having background color blue

Setting the style: even rows are having background color yellow

Displaying table with ng-repeat directive

```
</table>
</div>

<script>
angular.module('MyApp', []).controller('MyController', function($scope) {
    $scope.personinfo = [
        {name:'AAA',city:'Pune'},
        {name:'BBB',city:'Banglore'},
        {name:'CCC',city:'Chennai'},
        {name:'DDD',city:'Delhi'},
        {name:'EEE',city:'Mumbai'},
    ];
});
</script>
</body>
</html>
```

Output

Student Data

Name	City
AAA	Pune
BBB	Banglore
CCC	Chennai
DDD	Delhi
EEE	Mumbai

2.26.7 Modules

- We can write the script in AngularJs using different modules. Basically the modules help in separation of logic such as controller, service, application etc.
- The modules can be defined in separate **.js** files.

- Most commonly there are two modules - application module and controller module.
- Let us understand how to use modules in AngularJs with the help of example.
- First consider the following code without module -

AngularJS Document

```
<!DOCTYPE html>
<html>
<script src="http://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>

<div ng-app="MyApp" ng-controller="MyController">
  User: <input type="text" ng-model="user"><br/><br/>
  City: <input type="text" ng-model="city"><br/>
  <br/>
  Address: {{address_function()}}
</div>
<script>
var app = angular.module('MyApp', []);
app.controller('MyController', function($scope) {
  $scope.user = "AAA";
  $scope.city = "Pune";
  $scope.address_function=function() {
    return $scope.user + " lives in city " + $scope.city;
  };
});
</script>
</body>
</html>
```

Now we will create three modules from above code -

1. Application module
2. Controller Module
3. Driver Module or Use Module

Step 1 : Creating Application Module

For application module we will simply declare the application module using **angular.module** function. We've passed an empty array to it. This array generally contains dependent modules.

AngularJS Document[MyAppDemo.js]

```
var App=angular.module('MyApp', []);
```


Step 2 : Creating Controller Module

The controller **MyController** is declared using **App.controller** function.

AngularJS Document[MyControllerDemo.js]

```
App.controller('MyController', function($scope) {  
    $scope.user = "AAA";  
    $scope.city= "Pune"  
    $scope.address_function=function() {  
        return $scope.user+" "+$scope.city;  
    }  
});
```

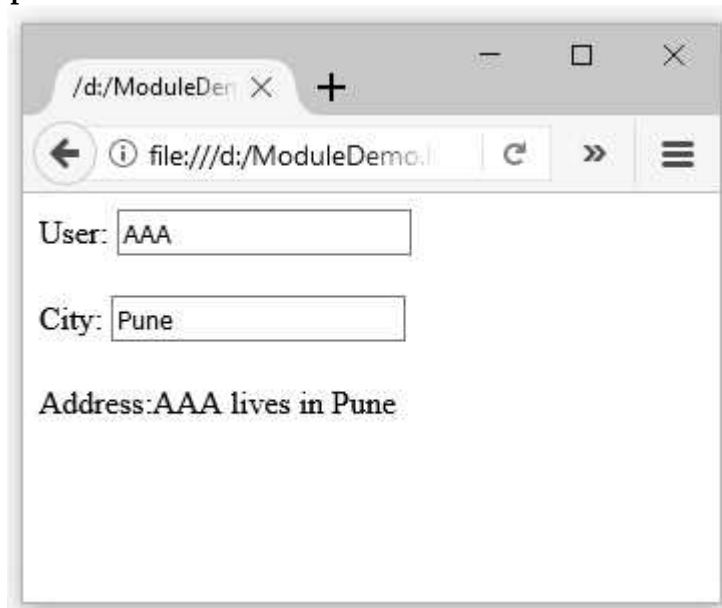
Step 3 : Creating Driver or use Module

This module will invoke the above modules and will display the result. The two scripts defined in above steps are included in this document using the `<script src>` tag.

Then using **ng-app** and **ng-controller** directives the **MyAppDemo.js** and **MyController.js** modules are mapped.

AngularJS Document[ModuleDemo.html]

```
<!DOCTYPE html>  
<html>  
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"> </script>  
  <script src = "MyAppDemo.js"> </script>  
  <script src = "MyControllerDemo.js"> </script> } Invoking external modules  
  <body>  
    <div ng-app="MyApp" ng-controller="MyController">  
      User: <input type="text" ng-model="user"> <br/> <br/>  
      City: <input type="text" ng-model="city"> <br/>  
      <br/>  
      Address:{{user + " lives in " + city}}  
    </div>  
  </body>  
</html>
```

Step 4 : The output can be viewed as**2.26.8 Forms**

- Form is a collection of various controls.
- AngularJS support two types of controls - **data binding** and **validation controls**.
- Various controls supported by AngularJS forms are -
 - Input
 - Select
 - Button
 - textarea

1. Input Control

User can type some text in the text box and the same text can be displayed on the AngularJS document. For this binding we use inside the <form> tag.

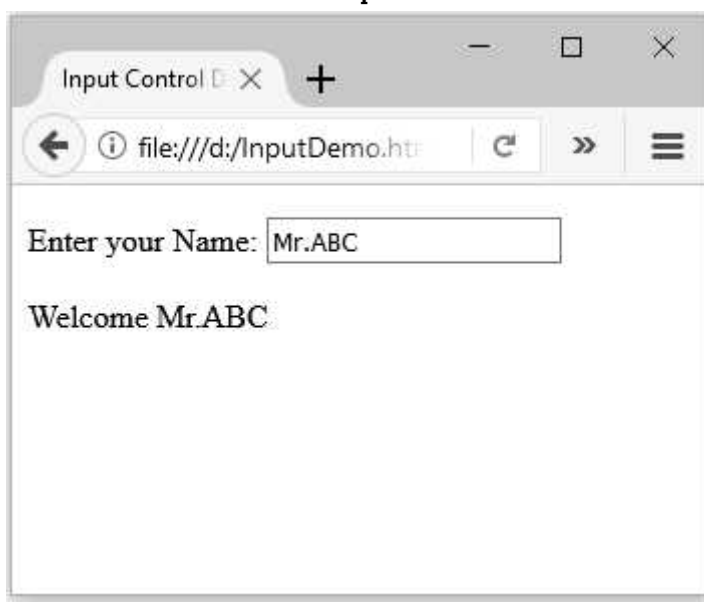
```
<input type="text" ng-model ="name">
```

Following example illustrates the use of input control. When user types the text inside the text box, it will be displayed along with the Welcome message.

AngularJS Document[InputDemo.html]

```
<html>  
<head>  
<title>Input Control Demo</title>
```

```
<script src =  
"https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>  
</head>  
<body>  
  <div ng-app = "">  
    <form>  
      <p>Enter your Name: <input type = "text" ng-model = "name"></p>  
    </form>  
    <p>Welcome {{name}}</p>  
  </div>  
</body>  
</html>
```

Output

2. Checkbox

We can have checkbox components on the Web document using AngularJS. For having checkbox, the input type must be "checkbox"

```
<input type="checkbox" ng-model="checked">
```

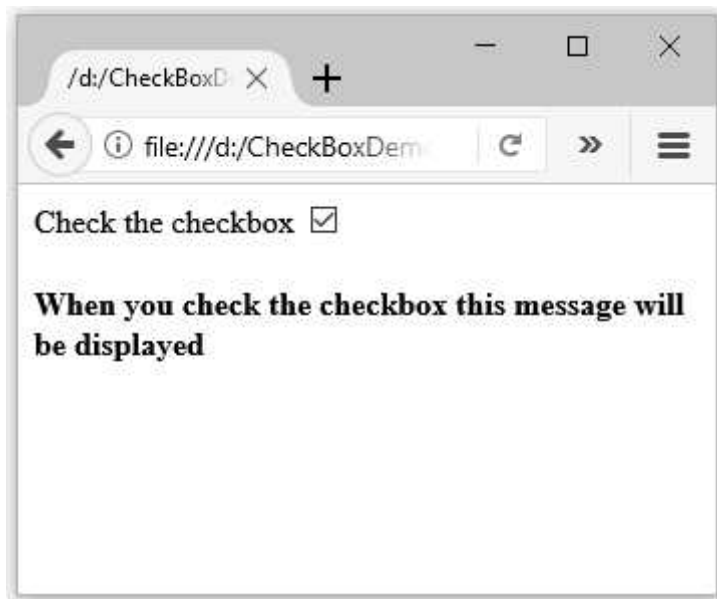
Here is an illustrative example for checkbox

AngularJS Document[CheckBoxDemo.html]

```
<!DOCTYPE html>  
<html>  
<script src="http://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>  
<body>  
<div ng-app="">
```

```
<form>
  Check the checkbox
  <input type="checkbox" ng-model="chked">
</form>
<h4 ng-show="chked">When you check the checkbox this message will be displayed</h4>
</div>
</body>
</html>
```

Output



3. Radio Button

The radio button is basically a group of buttons which allows you to choose one value at a time.

Let us see an illustrative script that uses Radio button control

AngularJS Document[RadioButtonDemo.html]

```
<!DOCTYPE html>
<html>
<script src="http://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body ng-app="">
  <form>
    Select the color:<br/>
    <input type="radio" ng-model="color" value="r">Red <br/>
    <input type="radio" ng-model="color" value="g">Green <br/>
    <input type="radio" ng-model="color" value="b">Blue <br/>
```

```
</form>
<div ng-switch="color">
  <div ng-switch-when="r">
    <p> You have chosen Red Color </p>
  </div>
  <div ng-switch-when="g">
    <p> You have chosen Green Color </p>
  </div>
  <div ng-switch-when="b">
    <p> You have chosen Blue Color </p>
  </div>
</div>
</body>
</html>
```

Output



Script Explanation

1. We have used `<form> </form>` tag inside which the `<input type="radio">` is taken for display of radio button.
2. The `ng-model` directive is used to bind radio buttons with HTML document, the `ng-model` is created for a group "color". the values are set as "r", "g" or "b".
3. In this script we have taken `ng-switch` directive to hide and show HTML sections depending on the value of the radio buttons. When the value of particular radio button is matched, then corresponding message is displayed.

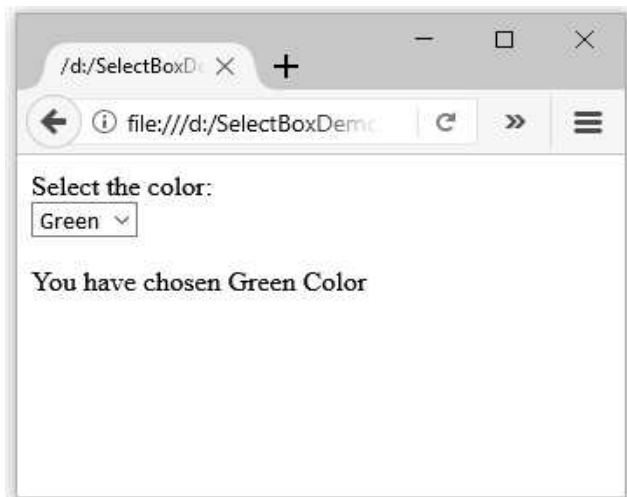
4. The selectbox

The dropdown list can be created using `<select>` `<option>` tags. For example -

AngularJS Document[SelectBoxDemo.html]

```
<!DOCTYPE html>
<html>
<script src="http://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body ng-app="">
  <form>
    Select the color:<br/>
    <select ng-model="color">
      <option value="r">Red <br/>
      <option value="g">Green <br/>
      <option value="b">Blue <br/>
    </select>
  </form>
  <div ng-switch="color">
    <div ng-switch-when="r">
      <p> You have chosen Red Color </p>
    </div>
    <div ng-switch-when="g">
      <p> You have chosen Green Color </p>
    </div>
    <div ng-switch-when="b">
      <p> You have chosen Blue Color </p>
    </div>
  </div>
</body>
</html>
```

Output



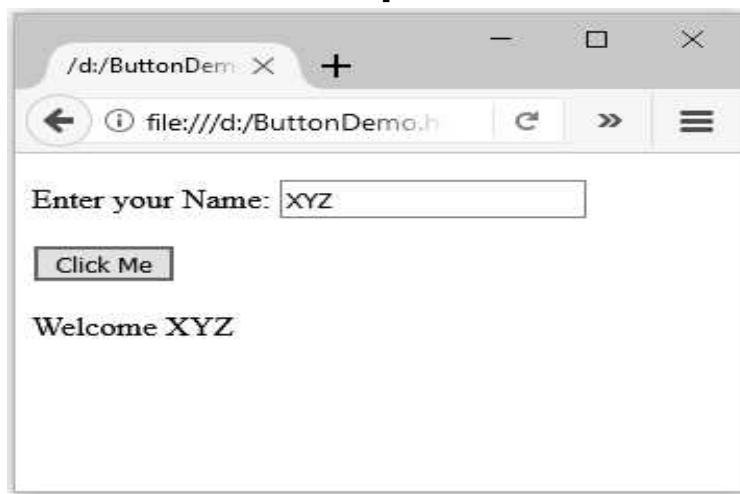
5. The button

We can create a button control and invoke some functionality at its click event. For example

AngularJS Document[ButtonDemo.html]

```
<html>
<script src =
"https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"> </script>
<body>
<div ng-app = "MyApp" ng-controller="MyController">
<form>
  <p>Enter your Name: <input type = "text" ng-model = "name"> </p>
  <input type = "button" value = "Click Me" ng-click="display()"> </p>
</form>
<p>Welcome {{username}}</p>
</div>
<script>
var app = angular.module('MyApp', []);
app.controller('MyController', function($scope) {
  $scope.name = "AAA";
  $scope.display=function() {
    $scope.username=$scope.name;
  };
});
</script>
</body>
</html>
```

Output



In above script on **ng-click** event the function **display** is invoked. This function displays the name typed by the user in the input text box.

2.26.9 Includes

We cannot include one HTML document inside another HTML document. But using AngularJS we can include html document. Following program illustrates it

Step 1 : Create a simple HTML document as follows -

HTML Document[TestFile.html]

```
<html>
<body>
<h2> Hello!!!</h2>
</body>
</html>
```

Step 2 : Now create AngularJS script. Make use of ng-include directive.

AngularJS Document[IncludeDemo.html]

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body ng-app="">
<div ng-include="TestFile.html"></div>
</body>
</html>
```

Step 3 : Open the web browser and get the output as follows -



We can also define the input controls in one HTML document, and initialize the values to input control in another file. For example

Step 1 : Create an AngularJS document as follows -

AngularJS Document[IncludeDemo1.html]

```
<html>
  <script src =
"https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"> </script>
<body>

  <div ng-app = "MyApp" ng-controller="MyController">
  <div ng-include = "myFile.htm"> </div>
  <script>
    var App = angular.module("MyApp", []);

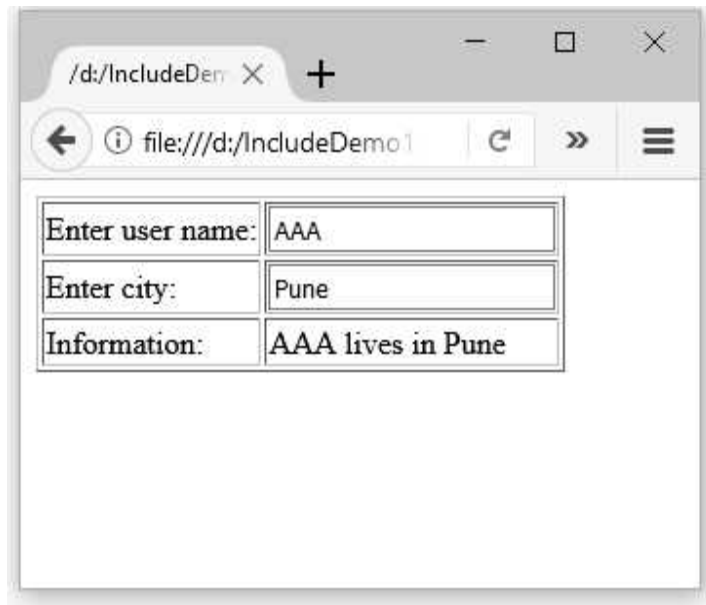
    App.controller('MyController', function($scope) {
      $scope.user = "AAA";
      $scope.city = "Pune";
      $scope.address_function=function() {
        return $scope.user + " lives in city " + $scope.city;
      };
    });
  </script>
</body>
</html>
```

Step 2 : The myFile.html document that is included in above step is as follows -

HTML Document : [myFile.html]

```
<table border = "1">
  <tr>
    <td>Enter user name:</td>
    <td><input type = "text" ng-model = "user"> </td>
  </tr>
  <tr>
    <td>Enter city: </td>
    <td><input type = "text" ng-model = "city"> </td>
  </tr>

  <tr>
    <td>Information: </td>
    <td>{{user + " lives in " + city}} </td>
  </tr>
</table>
```

Output**2.26.10 Views**

- View is basically the content which is shown to the user. User can wish to see the application in different views, accordingly that view can be displayed to the user.
- In AngularJS it is possible to have multiple views for single page application.
- The ng-view and ng-template and ng-route directives support the concept of views.
- The **\$routeProvider** services are required for supporting the application containing views.
- The **\$routeProvider** services are used to map the particular view with corresponding HTML page using templateUrl.
- The **ngRoute** module routes your application to different pages without reloading the entire application. It is used in **angular.module**.

Step 1 : Create the main page for invoking the view pages.

AngularJS Document[ViewDemo.html]

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular-route.js"></script>

<body ng-app="myApp">
```

```
<p><a href="#/">Main</a></p>

<a href="#!view1">First View</a><br/>
<a href="#!view2">Second View</a><br/>

<div ng-view></div>

<script>
var app = angular.module("myApp", ["ngRoute"]);
app.config(function($routeProvider) {
  $routeProvider
    .when("/", {
      templateUrl : "ViewDemo.html"
    })
    .when("/view1", {
      templateUrl : "view1.html"
    })
    .when("/view2", {
      templateUrl : "view2.html"
    });
});
</script>
</body>
</html>
```

Step 2 : Create view1.html page to display first view -

```
view1.html
<html>
<head>
<style>
h2 {
color:red;
}
</style>
</head>
<body>
<h2> This message is in RED Color </h2>
</body>
</html>
```

Step 3 : Create another web page for displaying second view.

```
view2.html
<html>
<head>
<style>
```

```
h2 {  
color:green;  
}  
</style>  
</head>  
<body>  
<h2> This message is in Green Color </h2>  
</body>  
</html>
```

Output



On clicking the first view or second view hyperlink the corresponding page will be displayed.

2.26.11 Scopes

The scope is an object that binds the HTML with JavaScript. In other words scope object binds view with controller. The view part is normally written in HTML and controller part is written as JavaScript.

Let us understand the use of scope object with the help of illustrative example.

AngularJS Document[ScopeDemo.html]

```
<html>  
<body>  
  <div ng-app = "MyApp" ng-controller = "MyController">  
    <p>Message: {{message}} <br/>Color: {{colormame}} </p>  
  
    <div ng-controller = "RedController">  
      <p>Message: {{message}} <br/>Color: {{colormame}} </p>  
    </div>  
  </div>
```

```
<div ng-controller = "BlueController">
  <p>Message: {{message}} <br/> Color: {{colorname}} </p>
</div>

</div>

<script src =
"https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"> </script>

<script>
  var App = angular.module("MyApp", []);

  App.controller("MyController", function($scope) {
    $scope.message = "In Color Demo controller";
    $scope.colorname = "No-Name";
  });

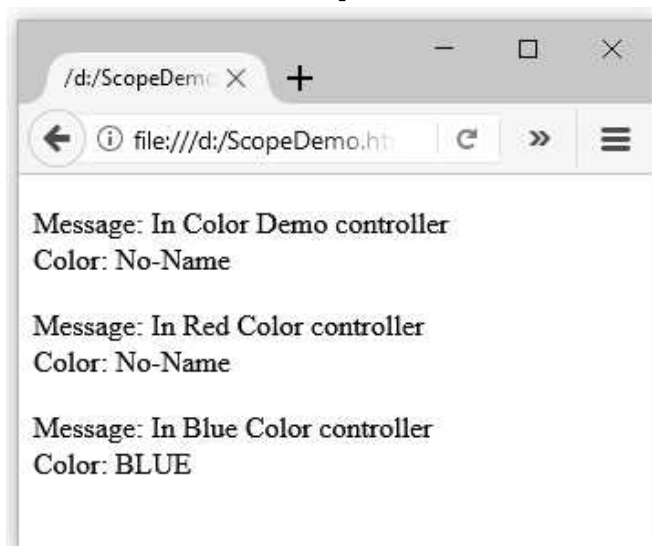
  App.controller("RedController", function($scope) {
    $scope.message = "In Red Color controller";
  });

  App.controller("BlueController", function($scope) {
    $scope.message = "In Blue Color controller";
    $scope.colorname = "BLUE";
  });

</script>

</body>
</html>
```

Output



Script Explanation :

In above code

1. We have created three controllers namely **MyController**, **RedController** and **BlueController**.
2. There are two properties namely **message** and **colorname**.
3. Note that **RedController** does not have **colorname** property; hence it inherits this value from **MyController**.
4. The **scope** object is used to define that values for these properties. Hence in HTML (view) part we can simply invoke the name of the property as **message** and **colorname** inside {{ }} and get the values assigned in respective controller.
5. Thus job of **scope** object is to bind the controller with view part of AngularJS document.

2.26.12 Services

The services are functions or objects that are intended to carry out specific task.

In AngularJS there are 30 built-in services used to carry out variety of tasks.

Built-in Services

Let us understand the use of services with the help of examples -

1. The \$http Service

This is a commonly used service in AngularJS. By using this service, the request can be made to the server and the application handles the response.

Step 1 : Create a simple web document as follows -

msg.html

Well Done!!!

Step 2 : Now create a web document that will make use of **\$http service** to get the message stored in above html file and to display the contents.

AngularJS Document[ServiceDemo1.html]

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>

<div ng-app="MyApp" ng-controller="MyController">
<p>Message obtained from a file is...</p>
<h1>{{message}}</h1>
```

```
</div>

<script>
var App = angular.module('MyApp', []);
App.controller('MyController', function($scope, $http) {
  $http.get("msg.html").then(function (response) {
    $scope.message = response.data;
  });
});
</script>

</body>
</html>
```

Output



2. The \$timeout Service

The \$timeout service is useful to execute certain task after some time interval. Following example displays one message on the browser and after 5 seconds another message will be displayed.

AngularJS Document[ServiceDemo2.html]

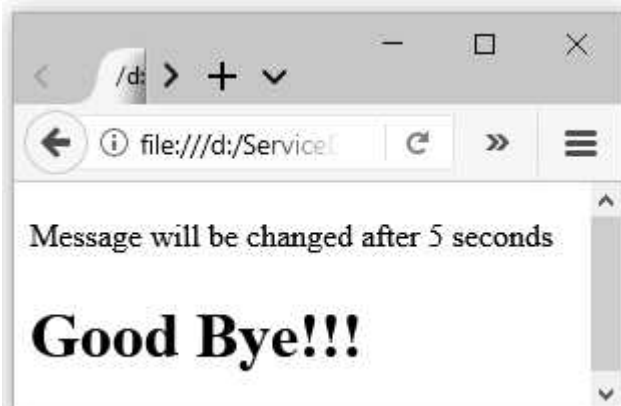
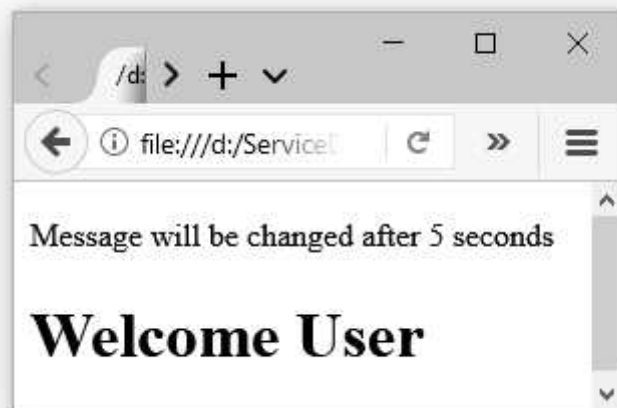
```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<body>
<div ng-app="MyApp" ng-controller="MyController">
```

```
<p>Message will be changed after 5 seconds</p>
<h1>{{message}}</h1>

</div>
<script>
var App = angular.module('MyApp', []);
App.controller('MyController', function($scope, $timeout) {
    $scope.message = "Welcome User";
    $timeout(function() {
        $scope.message = "Good Bye!!!";
    },5000);
});
</script>

</body>
</html>
```

Output



Creation of Custom Service

It is possible to create our own service. The service is normally created using two methods

1. Factory
2. Service

Step 1 : Using **factory method** we can define a factory and the method that performs specific task (i.e. service) is assigned to it.

For example

```
var App = angular.module("MyApp", []);
App.factory('AddService', function() {
  var factory = {};
  factory.addition = function(a, b) {
    return a + b
  }
  return factory;
});
```

Step 2 : Using **service method** we can define the service and a method is then assigned to it.

For example

```
App.service('CalculateService', function(AddService){
  this.add = function(a,b) {
    return AddService.addition(a,b);
  }
});
```

Step 3 : Then a **controller** is defined and the scope object and service method is invoked in it.

```
App.controller('MyController', function($scope, CalculateService) {
  $scope.add = function() {
    $scope.result = CalculateService.add($scope.number1,$scope.number2);
  }
});
```

The above three steps must be applied within the `<script> </script>`, which acts as a **controller**.

The **view** part is specified in `<div> </div>` tag.

Let us create a service for addition of two numbers.

AngularJS Document[ServiceDemo3.html]

```
<html>
  <script src =
    "https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"> </script>
```

```
<body>
  <h3> Addition of two numbers </h3>
  <div ng-app = "MyApp" ng-controller = "MyController">
    <p>Enter first number: <input type = "number" ng-model = "number1" /></p>
    <p>Enter second number: <input type = "number" ng-model = "number2" /></p>
    <button ng-click = "add()">ADDITION</sup></button>
    <p>Answer = {{result}}</p>
  </div>

  <script>
    var App = angular.module("MyApp", []);

    App.factory('AddService', function() {
      var factory = {};

      factory.addition = function(a, b) {
        return a + b
      }
      return factory;
    });

    App.service('CalculateService', function(AddService){
      this.add = function(a,b) {
        return AddService.addition(a,b);
      }
    });

    App.controller('MyController', function($scope, CalculateService) {
      $scope.add = function() {
        $scope.result = CalculateService.add($scope.number1,$scope.number2);
      }
    });
  </script>

</body>
</html>
```

Output

Addition of two numbers

Enter first number:

Enter second number:

Answer = 55

Example 2.26.1 Write a user defined service in AngularJS for reversing the given string.

Solution :

```
<html>
  <script src =
"https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"> </script>
<body>
  <h3> Reversing of String </h3>
  <div ng-app = "MyApp" ng-controller = "MyController">
    <p>Enter some string: <input type = "text" ng-model = "s" /> </p>
    <button ng-click = "reverse_str()">Click to Reverse</sup> </button>
    <p>Reversed String is: {{result}} </p>
  </div>

  <script>
    var App = angular.module("MyApp", []);

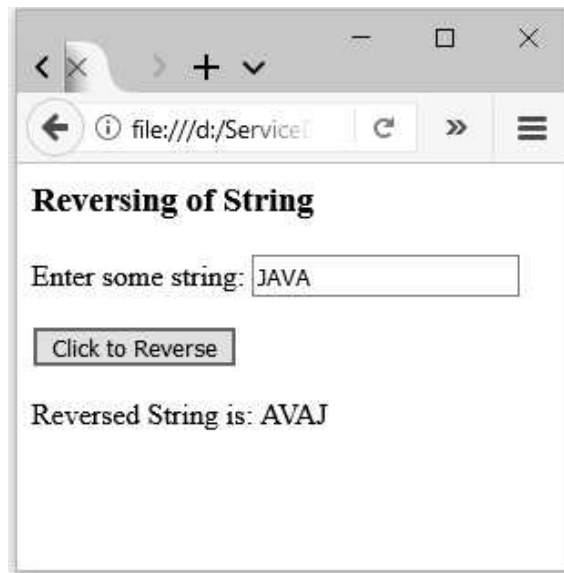
    App.factory('MyService', function() {
      var factory = {};
      factory.str_operation = function(str) {
        var r="";
        for(var i=str.length-1;i>=0;i--)
          r+=str[i];
      }
    });
  </script>
</body>
</html>
```

```
        return r;
    }
    return factory;
});

App.service('DisplayService', function(MyService){
    this.reverse_str = function(str) {
        return MyService.str_operation(str);
    }
});

App.controller('MyController', function($scope, DisplayService) {
    $scope.reverse_str = function() {
        $scope.result = DisplayService.reverse_str($scope.s);
    }
});
</script>
</body>
</html>
```

Output



2.26.13 Dependency Injection

- Dependency Injection is a software design pattern in which an object is given its dependencies, rather than the object creating them itself.
- AngularJS comes with a built-in dependency injection mechanism.

- Due to dependency injection mechanism, multiple different types of components which can be injected into each other's dependencies.
- **Advantage** of dependency injection is that it makes easier to reuse, configure and test the component in the application.
- There are core components that take part in dependency injection -
 1. value
 2. factory
 3. service
 4. Provider
 5. constant

Advantages of Dependency Injection

1. The creation and consumption of values and methods(dependencies) are separated.
2. Concurrent or independent development of dependencies is possible.
3. It facilitates the changing of dependencies when required.
4. It also allows injecting mock objects as dependencies for testing and maintenance.
5. It is possible to reuse and configure the object due to dependency injection.

Let us discuss how the dependency injection works for AngularJS application.

1. Value

The value is a simple object. It can be number, text, or some JavaScript object. The dependency injection can be achieved by passing values to factory, services or controller. The value can be defined using **value()** function.

For example -

```
//define a module
Var App = angular.module("myApp", []);

//create a object as 'str' and pass it a data.
App.value("str", "abc");

//inject this value in the controller
App.controller('MyController', function($scope, DisplayService,str) {
    $scope.message=str;
    $scope.result = DisplayService.reverse_str($scope.message);
});
```

2. factory

The **factory** function is used to return some value. Inside this factory function the values based on the demand of user or services are created. Typically factory function is injected within a service to utilize the methods. For example -

```
var App = angular.module("MyApp", []);  
//create factory function  
App.factory('AddService', function() {  
    var factory = {};  
    factory.addition = function(a, b) {  
        return a + b  
    }  
    return factory;  
});  
  
// inject factory in service named 'CalculateService'  
App.service('CalculateService', function(AddService){  
    this.add = function(a,b) {  
        return AddService.addition(a,b);  
    }  
});
```

3. service

The service is a JavaScript object which contains set of functions to perform some specific task. The services are injected into controller.

```
//creation of service named 'CalculateService'  
App.service('CalculateService', function(AddService){  
    this.add = function(a,b) {  
        return AddService.addition(a,b);  
    }  
});  
  
//Injection of service to controller 'MyController'  
App.controller('MyController', function($scope, CalculateService) {  
    $scope.add = function() {  
        $scope.result = CalculateService.add($scope.number1,$scope.number2);  
    }  
});
```

4. provider

The provider is a special factory method with **get()** function.

In AngularJS the provider is used internally to create services, factory during the configuration phase.

The **get()** function of provider is used to return the value or service or factory.

For example -

```
//define a module
var App = angular.module("MyApp", []);
...

//create a service using provider which defines a method addition to return addition of two numbers
App.config(function($provide) {
  $provide.provider('AddService', function() {
    this.$get = function() {
      var factory = {};

      factory.addition = function(a, b) {
        return a + b;
      }
      return factory;
    };
  });
});
```

5. constant

The values can not be injected into **module.config()** , hence the constants are used to pass the values to config. For example

```
App.constant("configparam","value");
```

2.26.14 Custom Directives

- In the simplest terms, a directive in AngularJS is a reusable component.
- Directives in AngularJS encapsulate all the behavioral properties and functionalities of an element thereby keeping all of the functionality grouped together.
- We can create directive using following steps -

Step 1 : Define Angular App

```
var mainApp = angular.module("myApp", []);
```

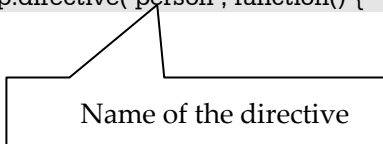
Step 2 : Now define directive

```
mainApp.directive("person", function() {
  var directive = {};
  directive.restrict = "EA";
  directive.template = "<h1> This is my first custom directive</h1>";
  return directive;
});
```

Code Explanation :

- The **directive** object is defined inside the

```
mainApp.directive("person", function() {
```



- We have define the directive object as

```
var directive = {};
```

The restrict 'EA' means the restriction is at the HTML element level as well as for the HTML element with attribute.

The legal restrict values are :

1. E for Element name
2. A for Attribute
3. C for Class
4. M for Comment

By default the value is EA, meaning that both Element names and attribute names can invoke the directive.

- Then we define the template for the directive element as

```
directive.template = "<h1> This is my first custom directive</h1>";
```

The **template** is basically the placeholder HTML code that can be used multiple times for the defined directive.

- Finally the created **directive** object is returned from the **mainApp.directive** function.

```
return directive;
```

Step 3 : Embed the directive in HTML page as

```
<body ng-app = "myApp">
```

```
  For element level: <person> </person>
```

```
  For attribute level: <div person> </div> <br/>
```

```
</body>
```

Code Explanation :

Note that we have used the directive named **person** as HTML element as well as attribute with <div> tag.

Step 4 : The complete code can be given as follows -

AngularJS Document[DirectiveDemo.html]

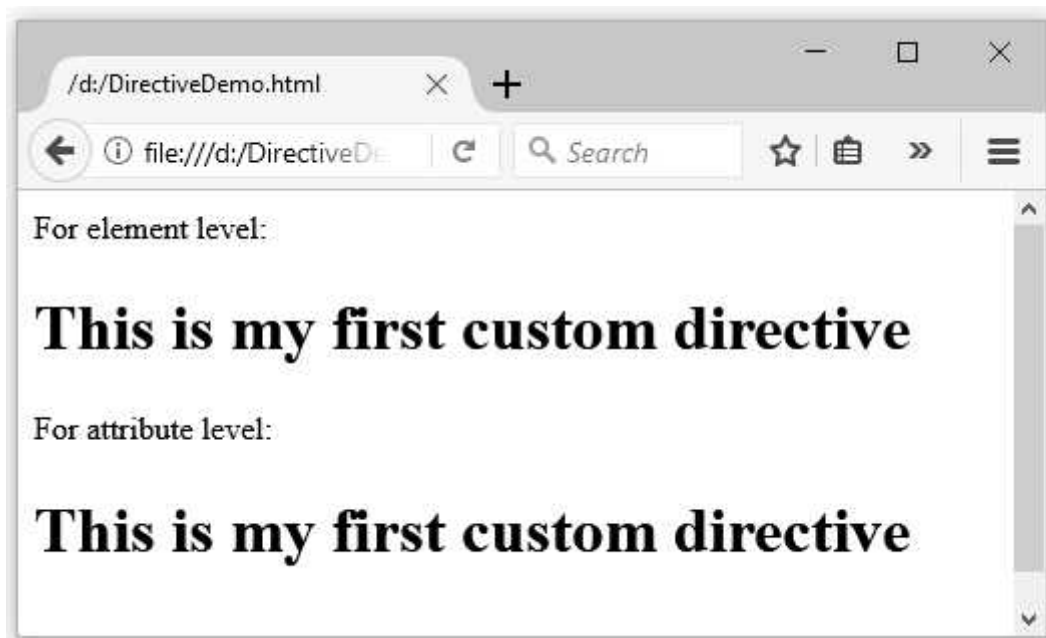
```
<html>
```

```
<body ng-app = "myApp">
```



```
For element level: <person></person>
For attribute level: <div person></div><br/>
<script src =
"https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js"></script>
<script>
var mainApp = angular.module("myApp", []);
mainApp.directive("person", function() {
var directive = {};
directive.restrict = "EA";
directive.template = "<h1> This is my first custom directive</h1>";
return directive;
});
</script>
</body>
</html>
```

Step 5 : Run the above code in Web browser



2.26.15 Internationalization

AngularJS has built-in support for internationalization of numbers and dates.

There are built-in filters such as **date**, **currency** and **number**.

date Filter

The date filter the date is following format -

- i) Short
- ii) medium

- | | |
|----------------|------------------|
| iii) fullDate | iv) shortDate |
| v) mediumDate | vi) longDate |
| vii) shortTime | viii) mediumTime |

currency Filter

The currency filter will use the currency symbol associated with the chosen locale when formatting a number as a currency. We can specify the currency symbol to the currency filter like this :

```
{{ curr | currency : '$' }} //dollars  
{{ curr | currency : '£' }} //pounds
```

number Filter

The number filter formats numbers according to the chosen locale. For example in Danish the thousand separators is specified by comma.

Setting Locale

For Internationalization we need to set the locale. The Danish locale can be set using following script

```
<script src = "https://code.angularjs.org/1.2.5/i18n/angular-locale_da-dk.js"></script>
```

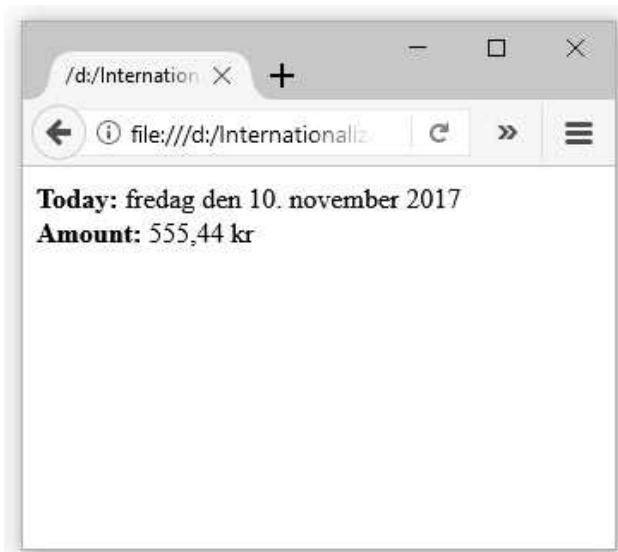
Example

Let us discuss AngularJS Document having display of date and currency in specific form.

AngularJS Document[Internationalization.html]

```
<!DOCTYPE html>  
<html>  
  <script  
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.2.5/angular.min.js"></script>  
  <script src="https://code.angularjs.org/1.2.5/i18n/angular-locale_da-dk.js"></script>  
<body ng-app="myapp">  
<div ng-controller="mycontroller">  
  <strong>Today:</strong> {{dt | date : "fullDate"}} <br/>  
  <strong>Amount:</strong> {{val | currency }}  
</div>  
  
<script>  
  var module = angular.module("myapp", []);  
  module.controller("mycontroller", function($scope) {  
    $scope.dt = new Date();  
    $scope.val = 555.44;  
  });
```

```
</script>  
</body>  
</html>
```

Output**Review Questions**

1. Does Angular JS support single page application via multiple views on a single page ? Justify your answer. **SPPU : May-18, Marks 6**
2. List and explain events supported by angular JS to enrich form filling and validation. **SPPU : May-18, Marks 4**
3. What are different ways to create service in angular JS ? Explain separation of concern in angular JS. **SPPU : May-18, Marks 6**
4. Create simple angular JS application to display "Hello, Input Name" using proper directive. **SPPU : Dec.-18, May-19, Marks 6**
5. List and explain various directives in angular JS. **SPPU : Dec.-18, Marks 4**
6. Write sample angular JS application which can demonstrate use of different expressions. **SPPU : May-19, Marks 8**
7. What is angular JS ? List and explain various directives in angular JS. **SPPU : Dec.-19, Marks 8**
8. Explain how angular JS integrates with HTML ? **SPPU : Dec.-19, Marks 8**

2.27 Multiple Choice Questions

Q.1 JavaScript is _____ language.

- | | |
|--|--|
| ☐ a programming | ☐ b scripting |
| ☐ c application | ☐ d none of these |

Q.2 JavaScript is designed for this purpose.

- ☐ a To apply particular style to HTML pages.
- ☐ b To perform server side scripting
- ☐ c To add interactivity to HTML pages.
- ☐ d To execute query to the backend database

Q.3 The script tag must be placed in _____.

- | | |
|---------------------------------|--|
| ☐ a head | ☐ b head and body |
| ☐ c body | ☐ d none of these |

Q.4 JavaScript code can be called using _____.

- | | |
|---|---|
| ☐ a triggering event | ☐ b preprocess |
| ☐ c function | ☐ d all of the above |

Q.5 Which of the following is not treated as error in JavaScript ?

- | | |
|---|--|
| ☐ a Missing Semicolons | ☐ b Not declaring variables |
| ☐ c Division by zero | ☐ d All of the above |

Q.6 Among the following which one is ternary operator ?

- | | |
|------------------------------|------------------------------|
| ☐ a + | ☐ b : |
| ☐ c - | ☐ d ? |

Q.7 Which of the following attribute can hold the JavaScript version ?

- | | |
|-------------------------------------|--|
| ☐ a Language | ☐ b Script |
| ☐ c Version | ☐ d None of the above |

Q.8 JavaScript was invented by _____.

- | | |
|--|---|
| ☐ a Sir Tim Barnard Lee | ☐ b W3 group |
| ☐ c James Gosling | ☐ d Brendan Rich |

Q.9 Integer variable is declared using following syntax in JavaScript _____.

- | | |
|-------------------------------------|---|
| ☐ a int num; | ☐ b Integer num; |
| ☐ c var num; | ☐ d integer num; |

Q.10 JavaScript variables are _____.

- | | |
|---|---|
| ☐ a case sensitive | ☐ b case non sensitive |
|---|---|

Q.11 JavaScript is _____.

- | | |
|-------------------------------------|--|
| ☐ a compiled | ☐ b interpreted |
|-------------------------------------|--|

Q.12 The nodes in HTML are _____.

- | | |
|---|--|
| ☐ a Subclass nodes | ☐ b HTML nodes |
| ☐ c Window nodes | ☐ d Element nodes |

Q.13 Which property is used to obtain browser vendor and version information ?

- | | |
|------------------------------------|--------------------------------------|
| ☐ a Model | ☐ b Version |
| ☐ c Browser | ☐ d Navigator |

Q.14 What is the result of the following code snippet ? window.location===document.location.

- | | |
|----------------------------------|---------------------------------|
| ☐ a False | ☐ b True |
| ☐ c 0 | ☐ d 1 |

Q.15 jQuery is a _____.

- | | |
|---|---|
| ☐ a Java program | ☐ b query language |
| ☐ c JavaScript library | ☐ d none of these |

Q.16 _____ is the sign used as a shortcut in jQuery.

- | | |
|-------------------------------|------------------------------|
| ☐ a \$ | ☐ b % |
| ☐ c @ | ☐ d & |

Q.17 _____ is the method used to set one or more style properties to the selected element.

- | | |
|--|---|
| ☐ a html() method | ☐ b style() method |
| ☐ c css() method | ☐ d all of the above |

Q.18 Which of the jQuery function prevents the code from running before the loading of the document finishes ?

- | | |
|---|--|
| ☐ a \$(document).load() | ☐ b \$(document).unload() |
| ☐ c \$(document).ready() | ☐ d \$(document).trim() |

Q.19 AngularJS expressions are written using _____.

- | | |
|---|---|
| ☐ a (expression) | ☐ b [expression] |
| ☐ c [[expression]] | ☐ d {{expression}} |

Q.20 Which directive initializes an AngularJS application ?

☐ a ng-init

☐ b ng-app

☐ c ngSrc

☐ d ng-model

Answers Keys for Multiple Choice Questions :

Q.1	b	Q.2	c	Q.3	b	Q.4	c	Q.5	c
Q.6	d	Q.7	a	Q.8	d	Q.9	c	Q.10	a
Q.11	b	Q.12	d	Q.13	d	Q.14	b	Q.15	c
Q.16	a	Q.17	c	Q.18	c	Q.19	d	Q.20	b

